Pós Graduação em Sistemas Computacionais Embebidos

Módulo de Inteligência Artificial em Sistemas Embebidos



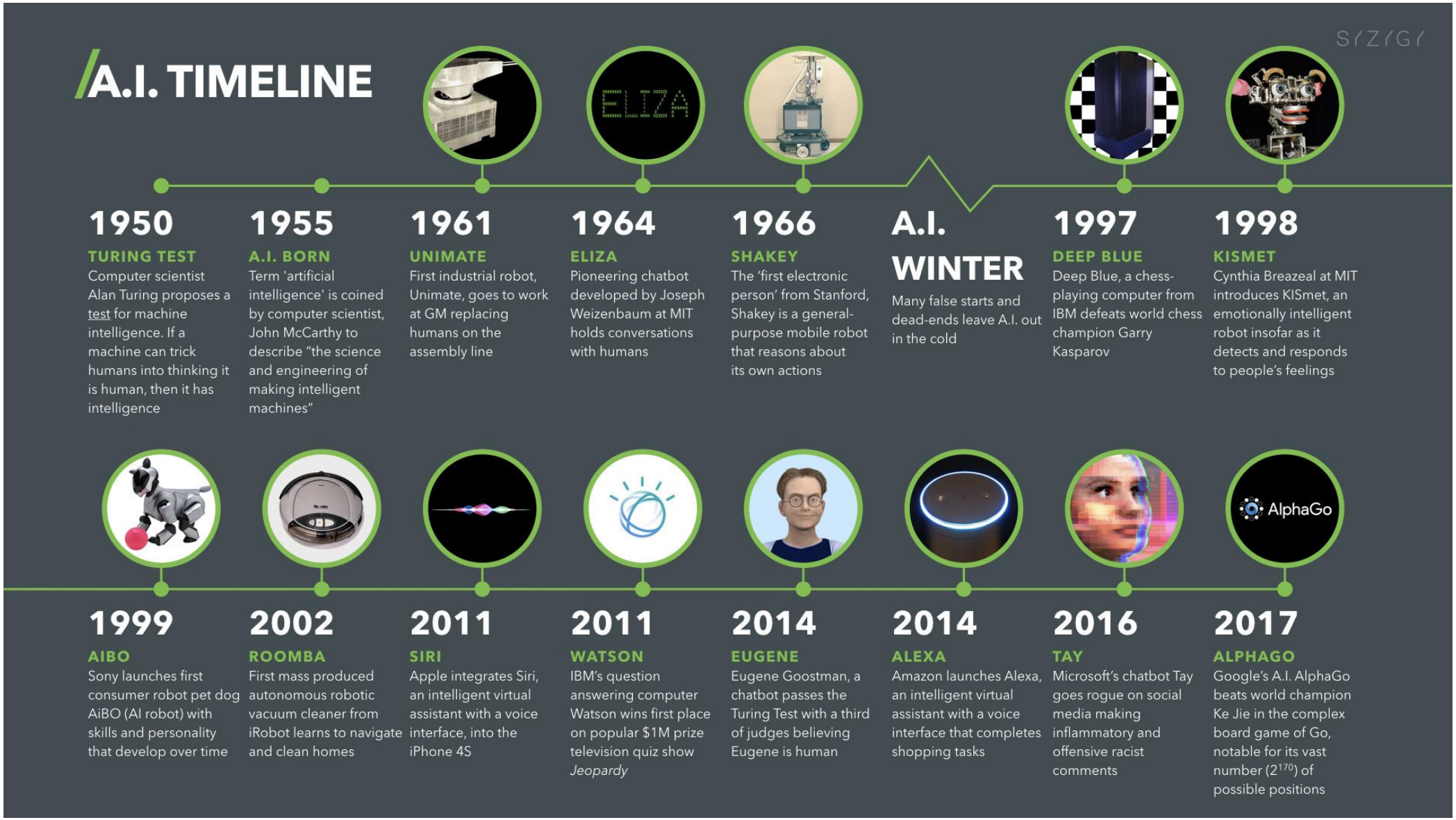
PGSCE-IARSEM

Introduction to the field of AI

2025/2026

Veríssimo Santos

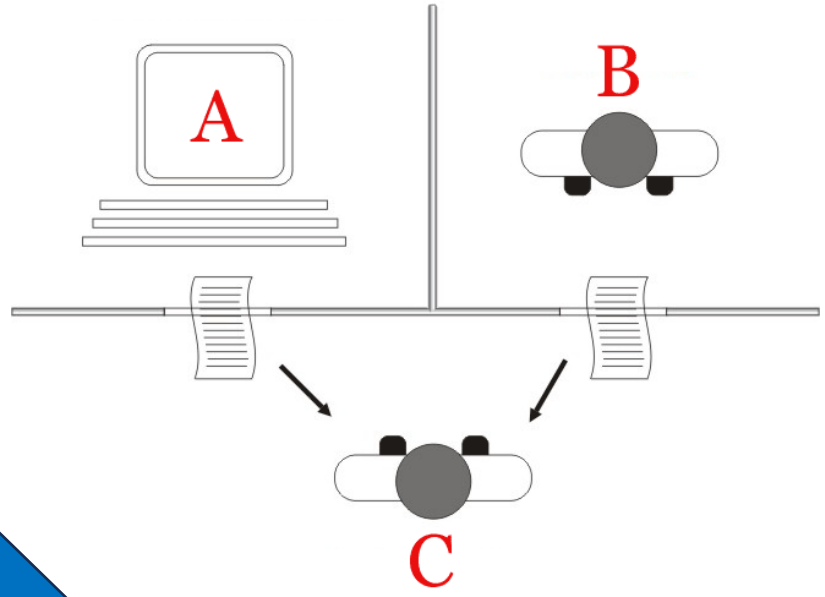
AI - Milestones



AI - Milestones

1950 - O Teste de *Turing*

- O Teste de Turing é um método de avaliação da capacidade de uma máquina de exibir um comportamento inteligente que seja equivalente, ou indistinguível, ao de um ser humano.
- **Como funciona na prática?**
- Formato original (Alan Turing, 1950) ***Imitation Game* / Jogo da Imitação**
- **E**nvolve três participantes em ambientes separados:



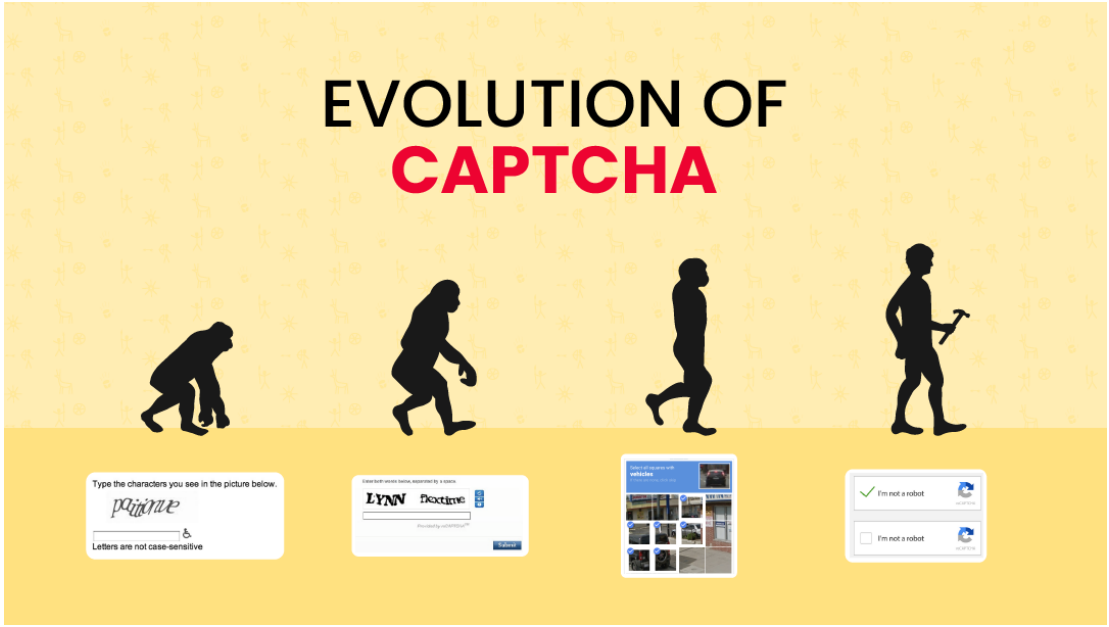
O Avaliador (C) / O Humano (B) / A Máquina (A)

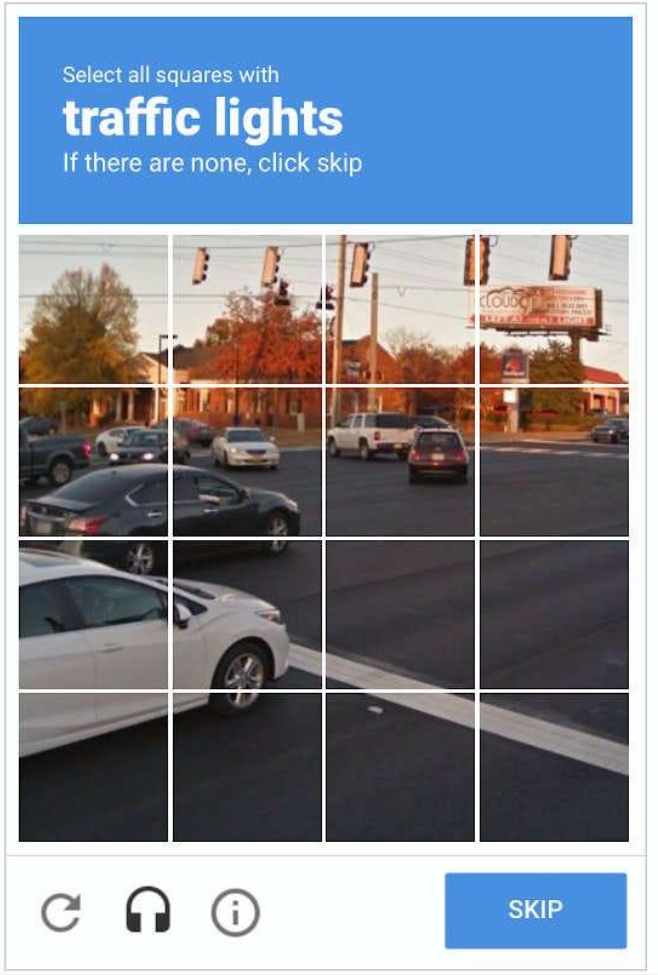
O avaliador (C) interage com os outros dois participantes exclusivamente por texto.
A máquina (A) tenta simular respostas humanas (incluindo falhas, hesitações ou falta de conhecimento em certos assuntos).
Se, após um determinado tempo de conversa, o avaliador (C) não conseguir identificar de forma confiável quem é a máquina (A) e quem é a pessoa real (B), considera-se que a máquina **passou no Teste de Turing**.

AI - Milestones

Exemplo de uso do Teste de *Turing*

- **CAPTCHA** - **C**ompletely **A**utomated **P**ublic **T**uring test to tell **C**omputers and **H**umans **A**part
- É um **Reverse Turing Test.**
- Porque neste caso o examinador é a máquina (servidor web) que utiliza o teste para perceber se quem está a ceder é um humano ou um bot.
- **O requisito neste caso é forçar a execução de uma tarefa, a qual só um humano consegue realizar de forma correta.**





AI - Milestones

The Dartmouth Conference

1956 US - Dartmouth Summer Research Project on Artificial Intelligence

Founding event of Artificial Intelligence as a field of scientific study

Objective: To bring together brilliant minds (mathematics, computer science, and cognitive psychology) to explore the possibility of creating "thinking machines."

Duration: *eight weeks* ***(brainstorming).***

Discussed Topics:

- Use of Language
- Neural Networks
- Abstractions and Concepts
- Problem Solving
- Self-improvement (Learning)
- Creativity

Organizers:

- **John McCarthy** (Dartmouth College)
- **Marvin Minsky** (Universidade de Harvard)
- **Nathaniel Rochester** (IBM)
- **Claude Shannon** (Bell Labs)

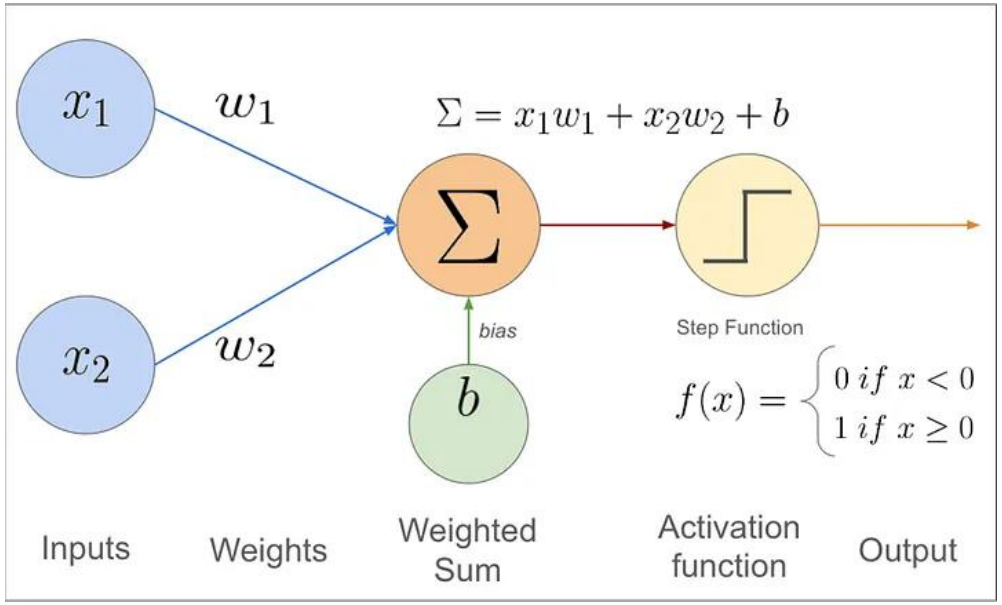
→ **Great contribution:**
They unified different areas of research (cybernetics, automata theory or complex information processing) under a single banner and common purpose. They set AI research directions for the following decades, sowing the technological seeds that culminated in today's artificial intelligence technologies.

AI - Milestones

1957 - Perceptron

- The **theoretical foundations emerged in 1943**, but the first operational and learning-capable neural network (the **Perceptron**) was **invented in 1957**.
- The **Perceptron is the first true trainable artificial neural network**. Initially, it was not just a software algorithm, but rather a physical machine built for image recognition, funded by the United States Navy.

The perceptron is a binary classifier that maps its input x (real) to the output f(x) (binary) as follows:

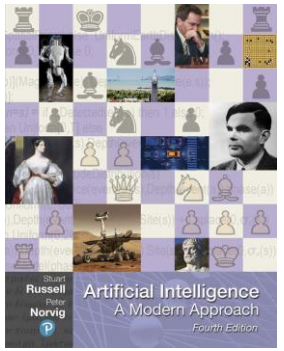


Artificial Intelligence - Definition

- What is Artificial Intelligence
- Defining AI is not an easy task:
  - Goals not totally defined and evolving
  - Vast spectrum => many perspectives

Definition:

"We call ourselves Homo sapiens—man the wise—because our intelligence is so important to us. For thousands of years, we have tried to understand how we think and act—that is, how our brain, a mere handful of matter, can perceive, understand, predict, and manipulate a world far larger and more complicated than itself. **The field of artificial intelligence, or AI, is concerned with not just understanding but also building intelligent entities—machines that can compute how to act effectively and safely in a wide variety of novel situations.**" *(Russell and Norvig)*



Artificial Intelligence - Definition

- In practical terms:

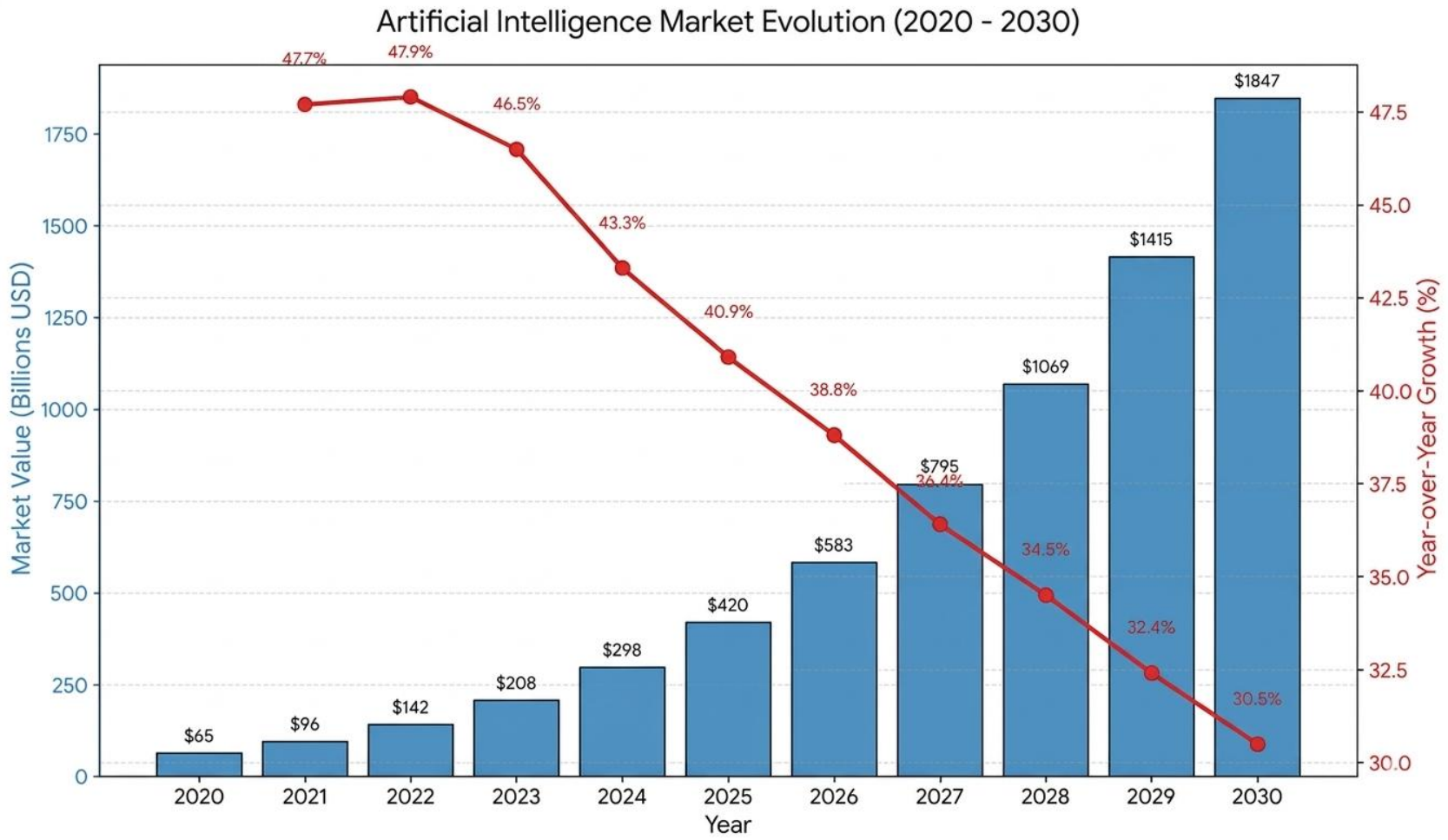
- Artificial Intelligence (AI) is a broad branch of computer science dedicated to creating systems and machines capable of performing tasks that would historically have required human intelligence.
- Instead of relying on pre-programmed instructions (as in conventional software), AI systems are designed to analyze data, recognize patterns, and make decisions or predictions more autonomously.

AI - Artificial Intelligence market value

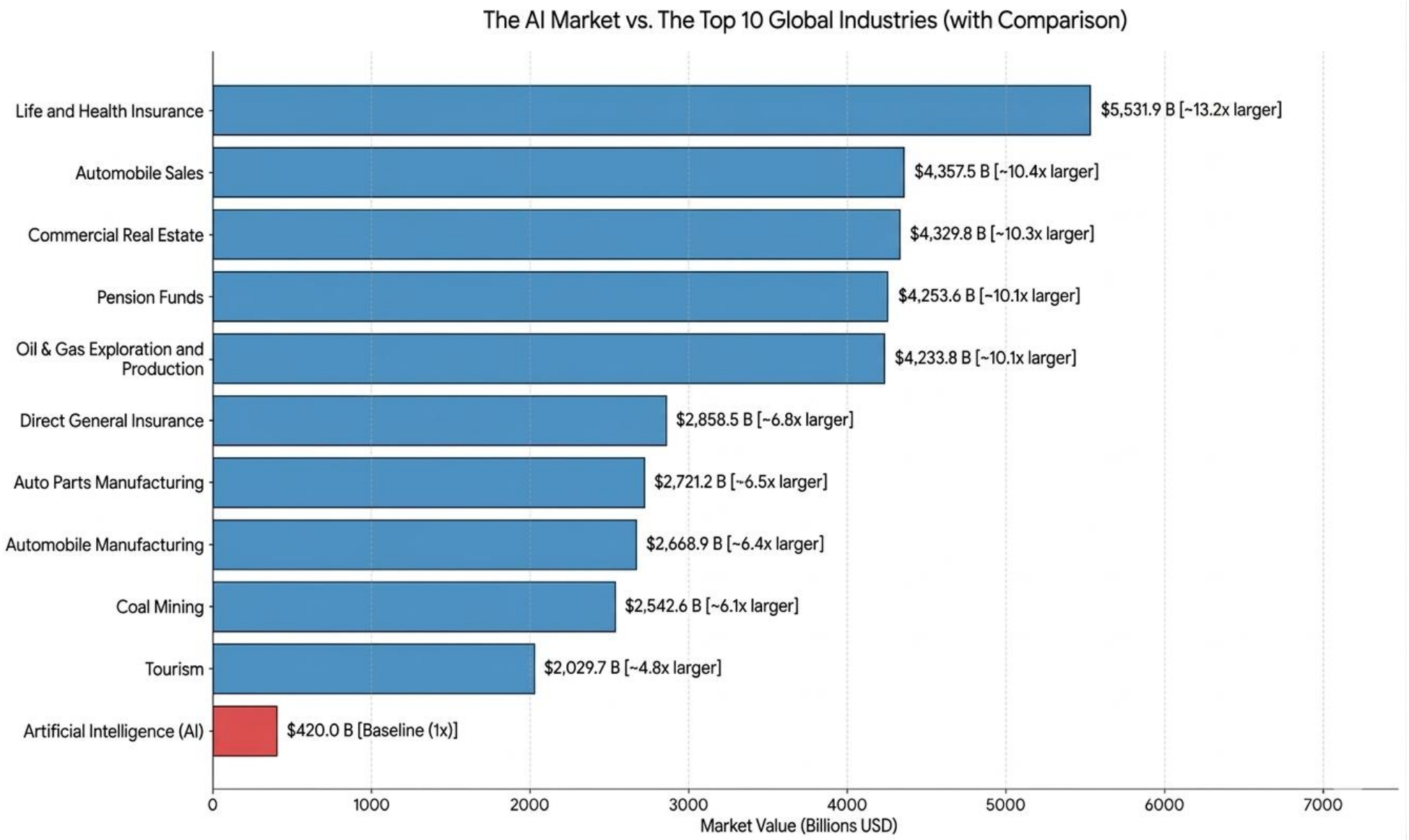
Trend **Fastest-growing field**

Market **Significant divergence between estimates !**

Year	AI Market Cap(USD Billions)	Growth compared to the previous year(%)
2020	$65,0	-
2021	$96,0	+47,7%
2022	$142,0	+47,9%
2023	$208,0	+46,5%
2024	$298,0	+43,3%
2025	$420,0	+40,9%
2026	$583,0	+38,8%
2027	$795,0	+36,4%
2028	$1.069,0	+34,5%
2029	$1.415,0	+32,4%
2030	$1.847,0	+30,5%



AI - Artificial Intelligence market vs biggest world markets



AI - Artificial Intelligence - Focus

1. **In terms of R&D the intellectual frontiers of AI are wide.**
2. **I.e., AI is an open field with terms of research and development.**
3. **It has been applied to most fields of human knowledge.**
4. **It can and probably will be used in ALL FIELDS OF HUMAN KNOWLEDGE.**

AI focus on:

General tasks:

- **Perception (Perceção):**
- Perception allows AI systems to interpret sensory data (e.g., images, sound, text) and convert it into structured information.
- It forms the foundation for higher-level AI tasks.
- Mimics human senses by processing raw data (e.g., recognizing faces in images, transcribing speech, Spatial Perception “Lidar”).
- Relies on feature extraction techniques, neural networks (shallow and deep) to identify patterns in unstructured data.
    - Ex: Computer vision systems use convolutional neural networks (CNNs) to detect objects in images.

AI - Artificial Intelligence - Focus

General tasks:

- **Learning (Aprendizagem):**
- Learning enables AI systems to improve performance over time by extracting patterns from data.
- Adapts to new information through **inductive reasoning** (generalizing from examples).
- Trains models on labeled datasets (e.g., classification/regression tasks).
- Uses layered neural networks for complex tasks like natural language processing.
- Data-driven, often producing "black-box" models that lack explicit interpretability.

- **Reasoning (Raciocínio):**
- Reasoning involves applying logic to derive conclusions, solve problems, or make decisions.
- Combines knowledge and rules to infer outcomes (e.g., diagnosing diseases from symptoms).
- Applies general rules to specific cases (e.g., "If A, then B").
- Selects the most plausible hypothesis from incomplete data.
- Uses formal logic and knowledge graphs for explainable decisions.
- Operates at the symbolic level, enabling transparency and multi-step problem-solving.

AI - Artificial Intelligence



Input:
Let $f(r) = \sum_{j=2}^{2008} \frac{1}{j^r} = \frac{1}{2^r} + \frac{1}{3^r} + \cdots + \frac{1}{2008^r}$. Find $\sum_{k=2}^{\infty} f(k)$.

LLEMMA 34B solution:
We have

$$\sum_{k=2}^{\infty} f(k) = \sum_{k=2}^{\infty} \sum_{j=2}^{2008} \frac{1}{j^k} = \sum_{j=2}^{2008} \sum_{k=2}^{\infty} \frac{1}{j^k} = \sum_{j=2}^{2008} \frac{1}{j^2} \sum_{k=0}^{\infty} \frac{1}{j^k} = \sum_{j=2}^{2008} \frac{1}{j^2} \frac{1}{1-\frac{1}{j}}$$
$$= \sum_{j=2}^{2008} \frac{1}{j(j-1)} = \sum_{j=2}^{2008} \left(\frac{1}{j-1} - \frac{1}{j} \right)$$
$$= \left(\frac{1}{1} - \frac{1}{2} \right) + \left(\frac{1}{2} - \frac{1}{3} \right) + \cdots + \left(\frac{1}{2007} - \frac{1}{2008} \right)$$
$$= 1 - \frac{1}{2008}$$
$$= \boxed{\frac{2007}{2008}}.$$

Final Answer: The final answer is $\frac{2007}{2008}$.

Write a short poem about sculpting clay

Sculpting clay,
A delicate art,
With skilled hands
And a steady heart.

To shape and mold
To create and form
A masterpiece
In the palm of your hand.

With patience and care
You bring your vision to life
A sculpture of beauty
In this humble material.

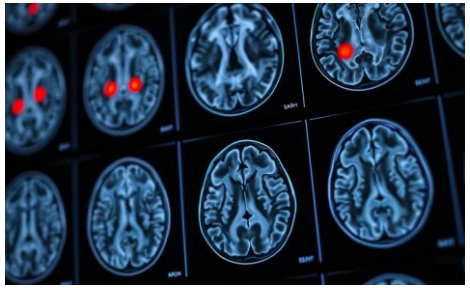
So let your hands dance
And your imagination roam
As you sculpt your clay
Into a work of art.

Specific tasks:

- **Playing chess;**
- **Proving mathematical theorems;**
- **Writing poetry;**
- **Driving cars;**
- **Generating artificial images;**
- **Diagnosing diseases,**
- **Etc.**







AI is relevant to any intellectual task; it is truly a universal field.

Artificial Intelligence - Research Approaches

Historically, researchers have pursued AI along different dimensions:

- Human vs. Rational
- Thought vs. Behavior

Which led to four approaches that have been followed:

- **Systems that act like humans:** *The Turing Test approach* **[VIDEO1][VIDEO2]** 
- **Systems that think like humans:** *The cognitive modelling approach*
- **Systems that think rationally:** *The laws of thought approach*
- **Systems that act rationally:** *The rational agent approach*

Very interesting and related current research directions pursued to establish the hypothesis that will lead to the expansion of the field.

Way to complex and primary results to be applied! Frequently a simpler and more practical taxonomy is used!

Artificial Intelligence Types

Types of Artificial Intelligence:

- **Narrow AI - Specific tasks**
  - Recognizing images (classification, object detection, segmentation, semantic segmentation, etc);
  - Translating languages;
  - Playing games.
  - In 2023 Large Language Models have established a new and higher baseline.
    - Enhanced Natural Language Processing
    - Excellent understanding and generation capability. Highly proficient.
- **General AI - Any intellectual human task**
  - It's a Holy Grail;
  - It-s a promising and a Challenge !

AI has achieved outstanding results in a narrow perspective. To achieve AGI (Artificial General Intelligence) there is a huge path to be followed.

Artificial Intelligence - Areas of Application

Ai has been applied in almost all field of knowledge.

Some notable areas of Application:

- **Healthcare & Life Sciences**
  - **1. Medical Diagnostics.**
  - **2. Drug Discovery.**
- **Finance & Business**
  - **3. Fraud Detection**
  - **4. Algorithmic Trading**
  - **5. Customer Service**
  - **6. E-commerce & Retail**
  - **7. Human Resources**
- **Transportation & Logistics**
  - **8. Autonomous Vehicles**
  - **9. Supply Chain Optimization**
- **Industry & Production**
  - **10. Predictive Maintenance**
  - **11. Precision Agriculture**
- **Technology & Security**
  - **12. Cybersecurity**
  - **13. Software Development**
  - **14. Natural Language Translation**
- **Daily Life & Entertainment**
  - **15. Smart Homes (IoT)**
  - **16. Entertainment & Streaming**
  - **17. Video Games**
  - **18. Generative Art & Content**
- **Science & Society**
  - **19. Climate Modeling & Environment:**
  - **20. Space Exploration**
- **Among many others**

Artificial Intelligence - Perspetivas futuras

AI Trends for 2026 @ YouTube
(https://www.youtube.com/watch?v=zt0JA5rxdfM)

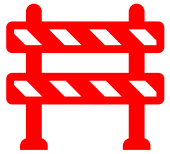
PGSCE-IARSEM

Data Science

2025/2026

Veríssimo Santos

Data Science - Slide Master



What is Data Science.

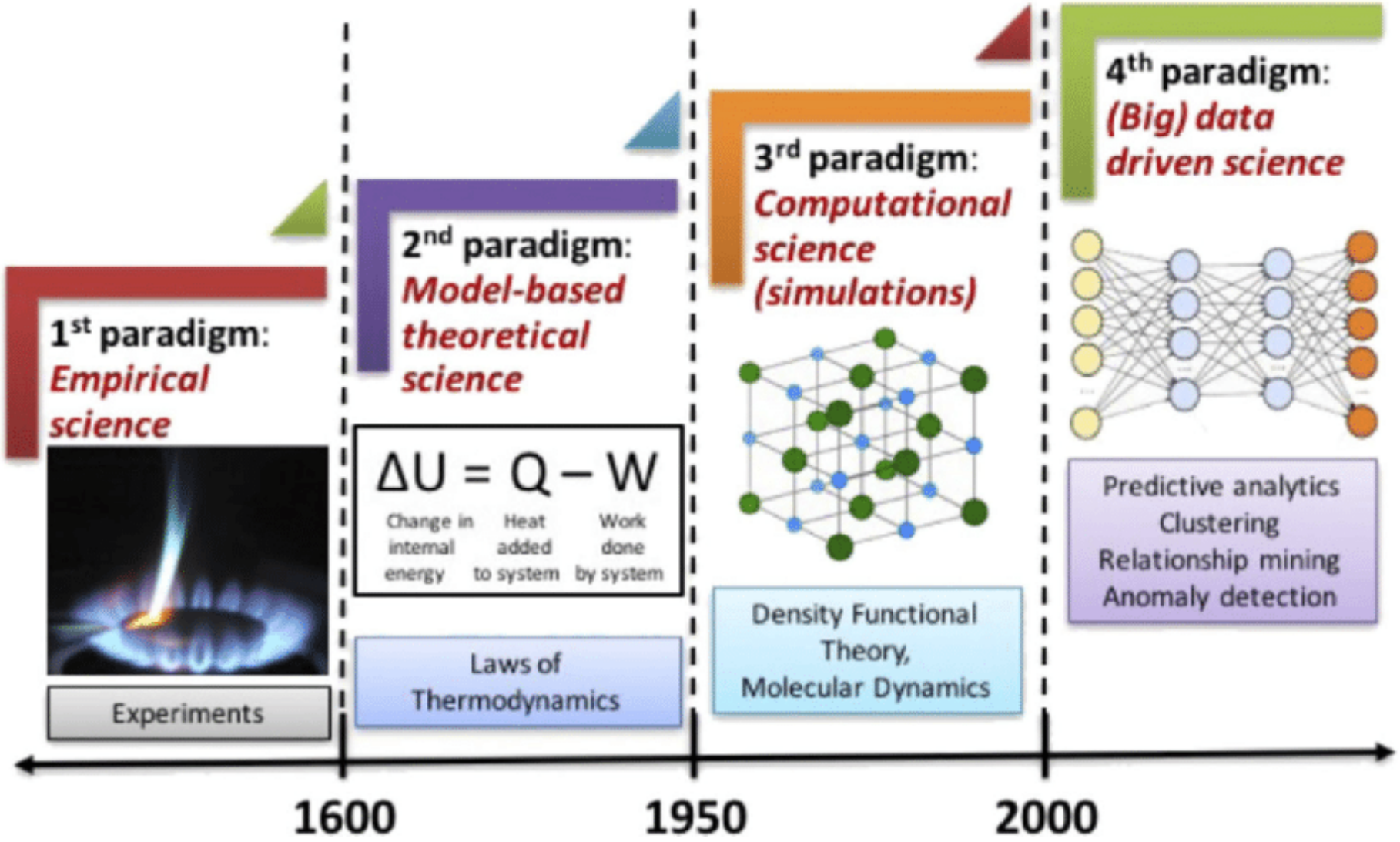
Evolução do campo.

Sub campos. Explicar cada um.

Principais conceitos de DS.

Prespectivas futuras.

Data Science - The paradigms of Science



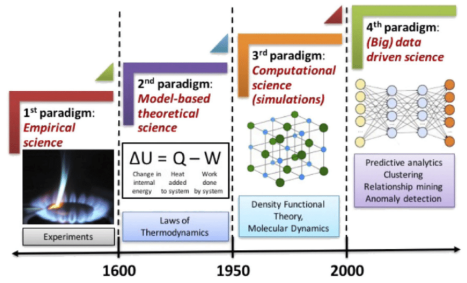
Data Science - The paradigms of Science

The First Paradigm – Empirical [Pre-1600s]

- Based on direct observation and experiments.
  - Science was based on direct observation of nature.
  - Early scientists, relied on empirical methods to describe natural phenomena.

  - **Impact on Fields:**
    - Biology & Medicine → Early classifications of plants and animals, anatomy studies, herbal medicine.
    - Astronomy → Naked-eye observations of the stars, planetary movements (e.g., Ptolemy's geocentric model).
    - Chemistry (Alchemy) → Primitive experiments with substances, leading to early chemical discoveries.
    - Physics → Basic mechanics and optics based on direct observation.

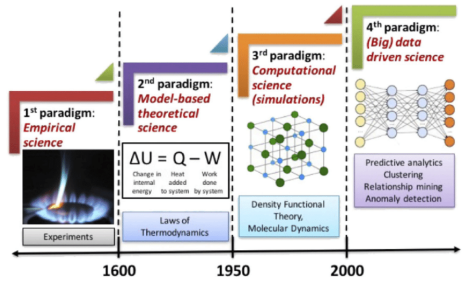
  - **Key Figures:**
    - Aristotle (Natural Philosophy)
    - Hippocrates (Medicine)
    - Ptolemy (Astronomy)



Data Science - The paradigms of Science

The Second Paradigm – Theoretical [1600s–1950s]

- Uses mathematical models and theories to explain natural phenomena.
  - The development of mathematical frameworks to explain natural processes.
  - Pioneered by figures like Isaac Newton and Galileo Galilei.
  - **Impact on Fields and Key Figures:**
    - **Physics** → ***Newton***'s laws, electromagnetism, relativity, quantum mechanics.
    - **Astronomy** → ***Kepler***'s laws of planetary motion, ***Newton***'s universal gravitation.
    - **Engineering & Mechanics** → Mathematical models for structures, fluid dynamics, aerodynamics.
    - **Economics & Social Sciences** → Theories of markets, probability, statistical models for human behavior.

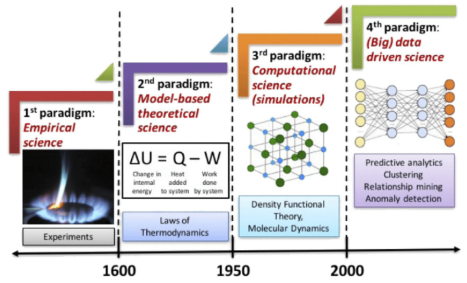


Data Science - The paradigms of Science

The Third Paradigm – Computational [1950s–2000s]

Simulations and large-scale computing became essential for research.

- The use of computers to solve complex equations and simulate real-world phenomena.
- **Impact on Fields:**
  - **Meteorology & Climate Science** → Weather prediction models, climate simulations.
  - **Engineering & Material Science** → Computational simulations for structures, aerodynamics, molecular modeling.
  - **Economics & Finance** → Predictive modeling, risk analysis, algorithmic trading.
  - **Genomics & Biology** → DNA sequencing, protein folding simulations.
  - **Physics & Chemistry** → Quantum simulations, computational chemistry, nuclear physics.
- **Key Figures:**
  - ***John von Neumann*** (Computational Modeling)
  - ***Edward Lorenz*** (Chaos Theory, Weather Models)

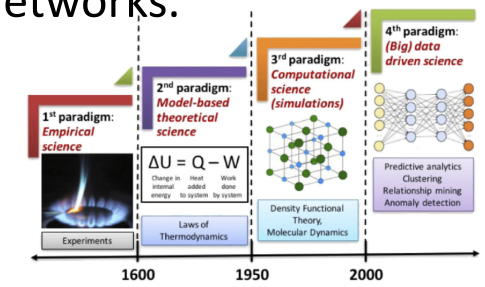


Data Science - The paradigms of Science

The Fourth Paradigm - Data-Intensive Science [2000s–present]

Massive amounts of data (Big Data) used drive new discoveries.

- Science now relies on analyzing massive datasets rather than just experiments or simulations.
- Enabled by AI, cloud computing, and large-scale data collection.
- **Impact on Fields:**
  - **Astronomy** → AI-driven sky surveys, automated planet detection.
  - **Healthcare & Genomics** → AI for disease diagnosis, personalized medicine, genome-wide studies.
  - **Social Sciences & Psychology** → Big Data analysis of human behavior, social network analysis.
  - **Business & Marketing** → Consumer analytics, recommendation systems, AI-driven decision-making.
  - **AI & Machine Learning** → Autonomous scientific discovery, robotics, neural networks.
  - Many other fields.

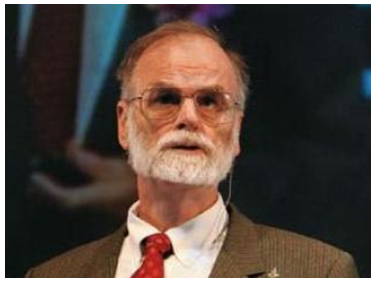


Data Science - The paradigms of Science

The Fourth Paradigm - Data-Intensive Science [2000s–present]

- **Key Figures:**

1. ***Jim Gray*** (Fourth Paradigm)
   2009 - The visionary behind the Fourth Paradigm.
   1998 A.M. Turing Awardee



(1)

2. ***Geoffrey Hinton*** (Deep Learning)
   The "Godfather of Deep Learning“;
   Decades of persistence researching artificial neural networks:
   Broader scientific contribution that laid the structural groundwork for the current AI revolution.
   2018 A.M. Turing Awardee



(2)

3. ***Fei-Fei Li*** (Computer Vision, AI)



AI4ALL

TIME's 2025 Person of the Year
The 2025 Queen Elizabeth Prize for Engineering



(3)

Data Science

- Computing enabled the **information age**, and this originated an immensity of data.
- These **massive amounts of data** created huge challenges for their storage and processing.
- The continuous evolution of computing and science generated opportunities not only to better manage this data but mostly to extract knowledge from it.

Data is the new gold!

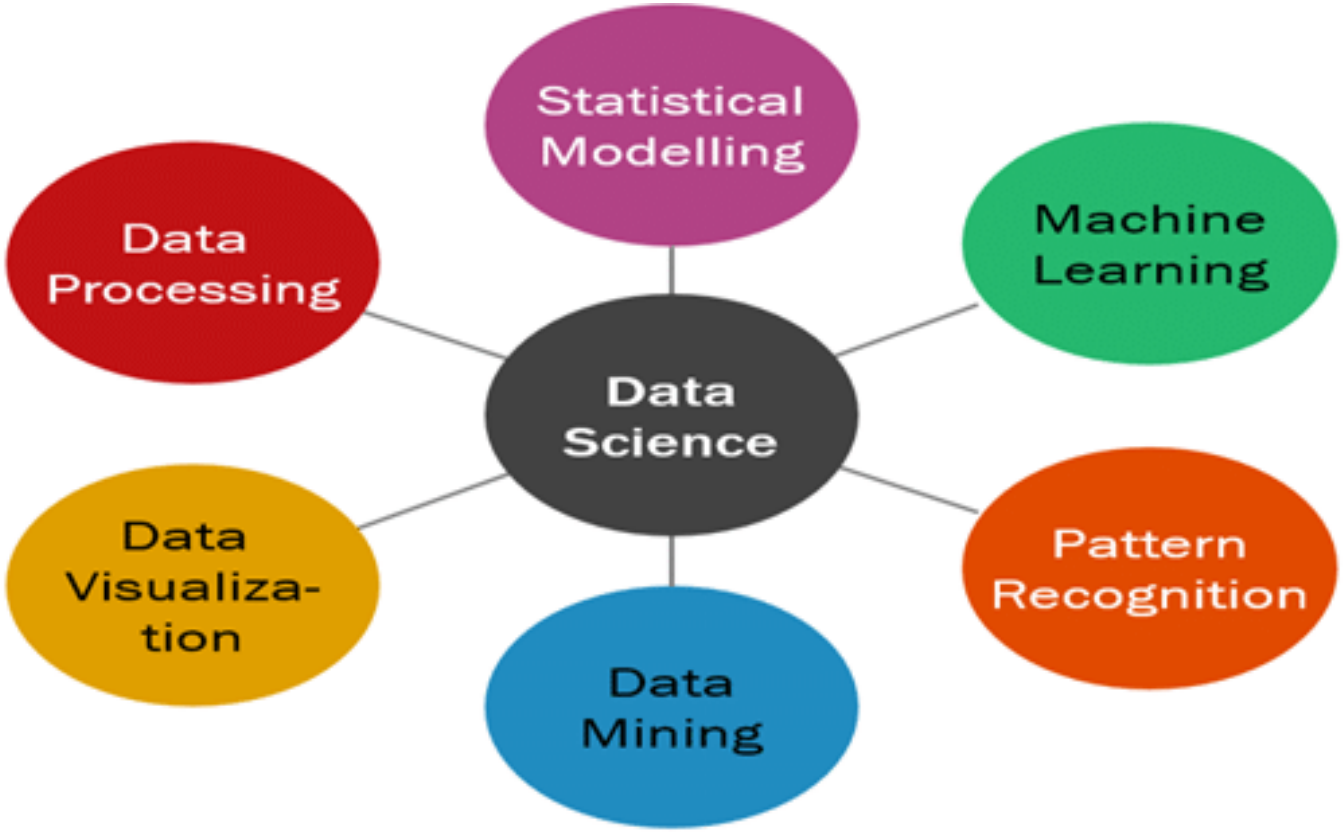


Data Science

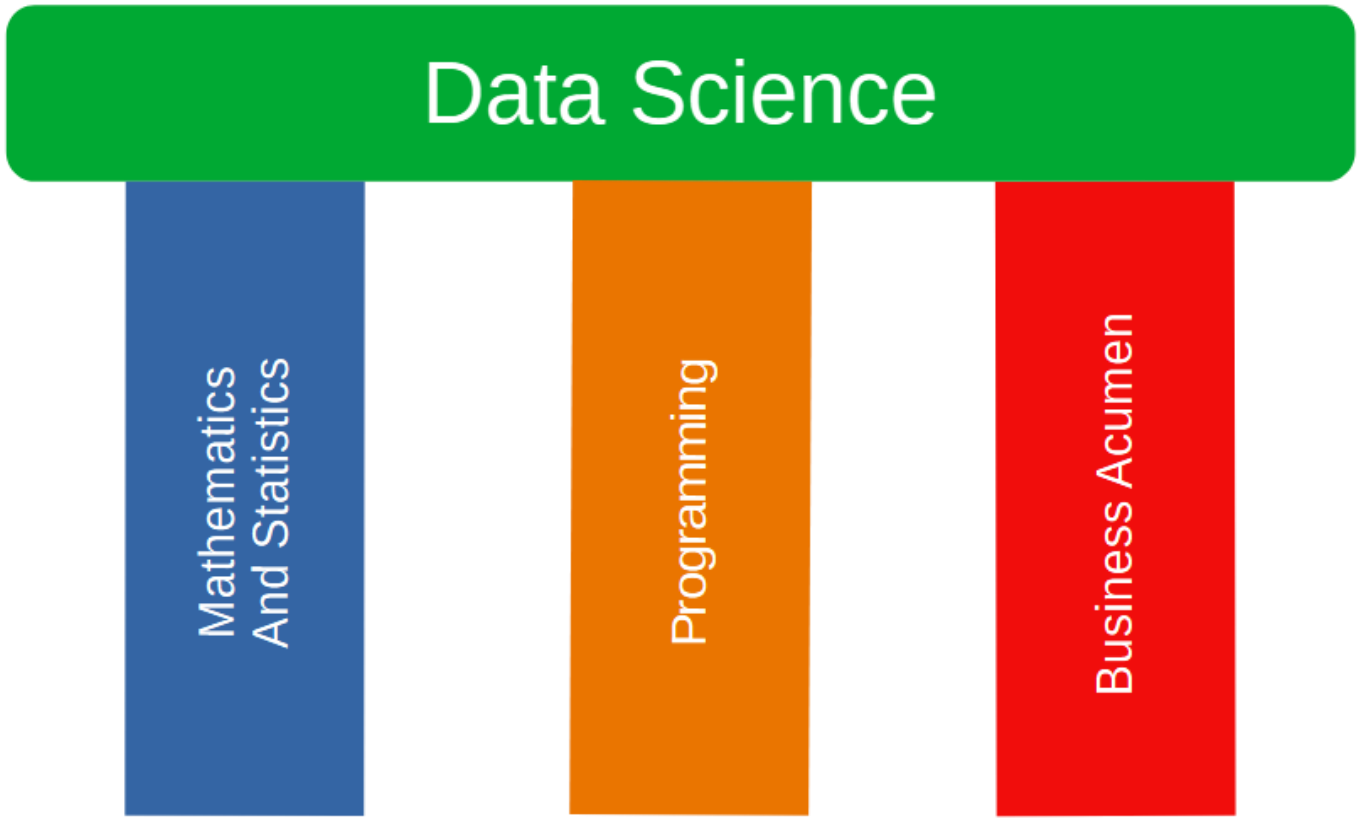
What is Data Science

- Broad interdisciplinary field that aims to extract or extrapolate knowledge and insights from noisy, structured, and unstructured data.
- Aims to create an in-depth understanding of data by finding patterns, performing analysis and making predictions to make sense of data)
- **Data Science uses (among others):**
  - Math;
  - Statistics;
  - Programming;
  - Advanced analytics;
  - Artificial intelligence;
  - And **Machine learning**.

Data Science Subfields



The Three Pillars of Data Science



Data Science Process



Data Engineers

Data Analysts

Machine Learning Engineers

Data Scientists

PGSCE-IARSEM

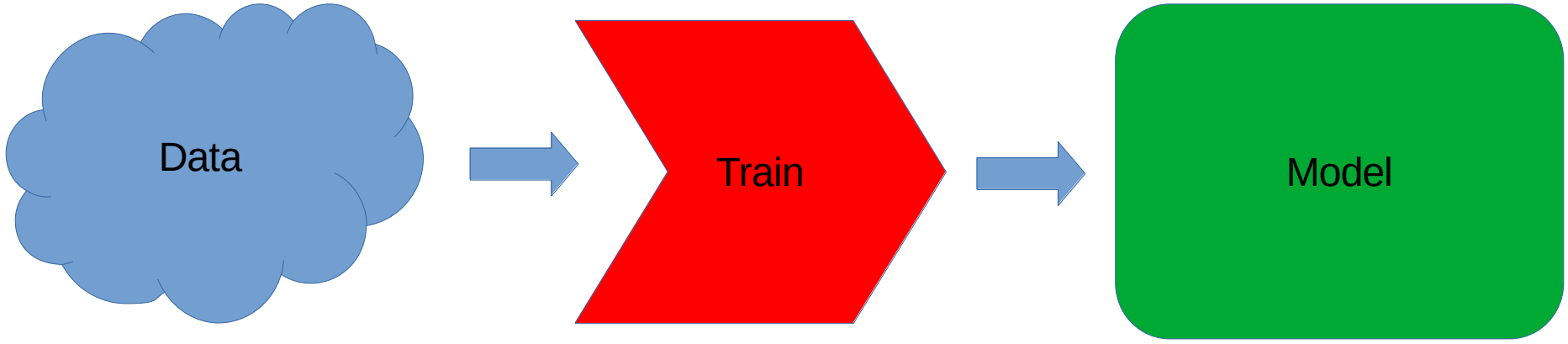
Machine Learning (ML)

2025/2026

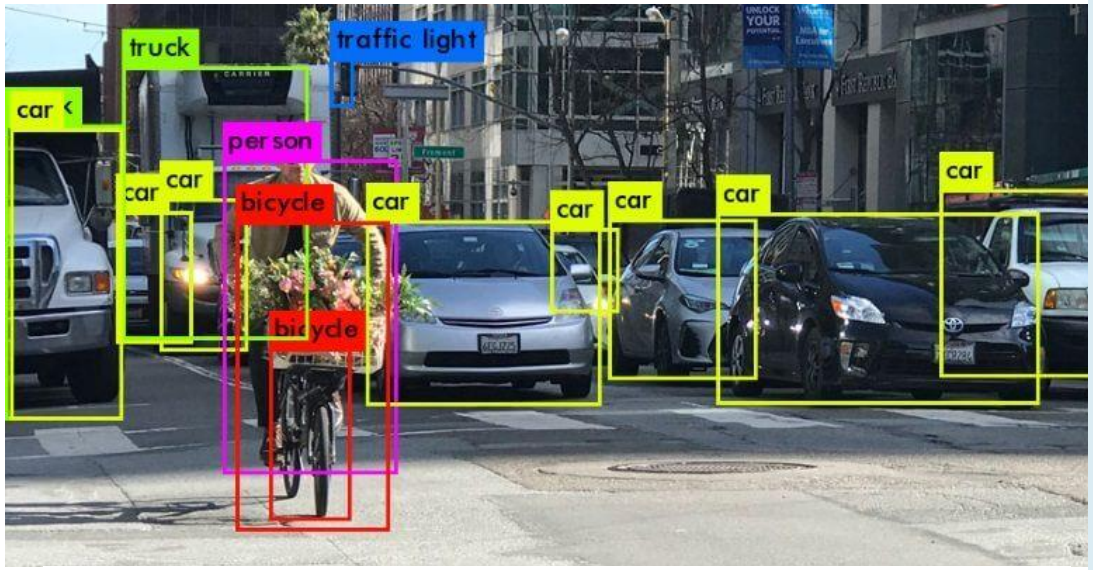
Veríssimo Santos

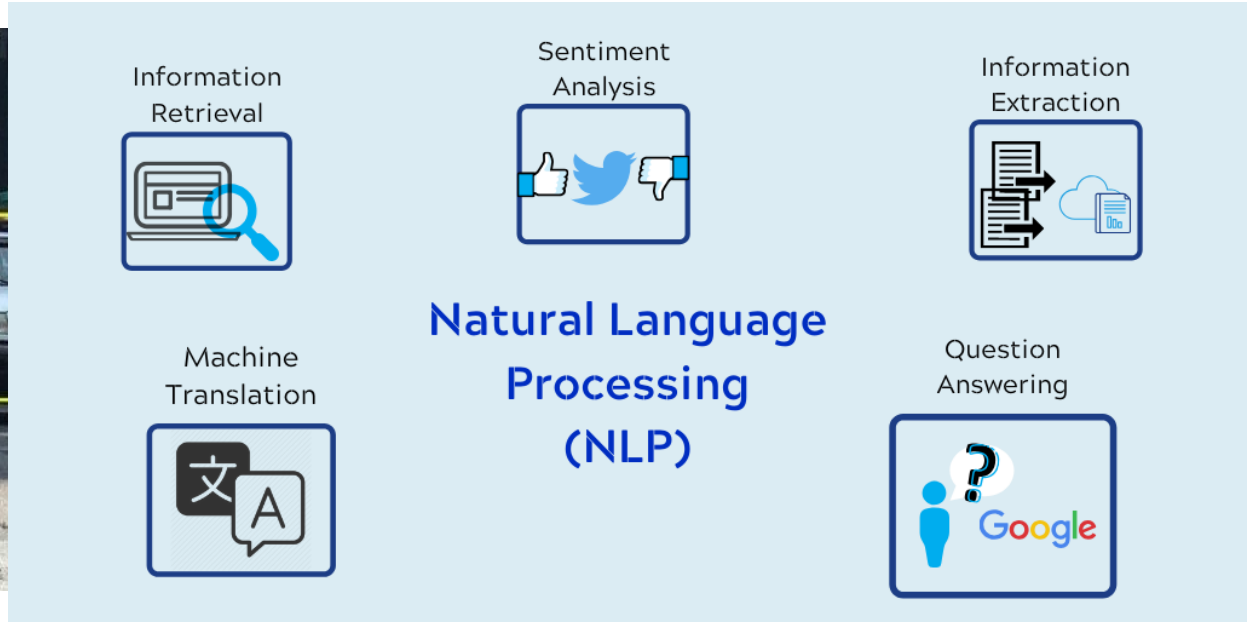
Machine Learning (ML)

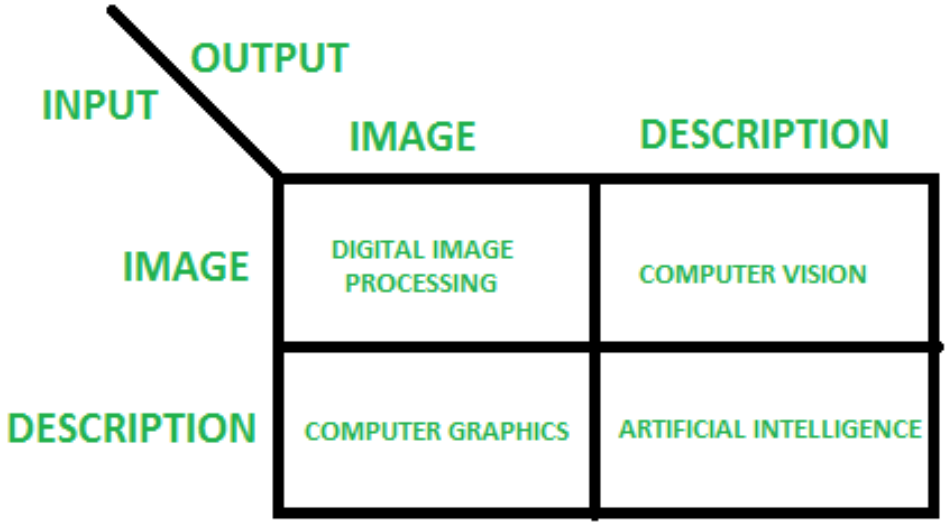
- Basic idea!
  - Enable computer systems to learn from data and experience, just as humans do.
- What is?
  - Branch of artificial intelligence that involves the use of algorithms and/or statistical models to enable computer systems **to learn and improve from experience without being explicitly programmed**.



ML - Notable use cases







ML - Data

- Data is the crucial raw material for Machine Learning models.
- Data is any information or input that is used to train or test a machine learning model.
- Data comes in various forms, such as text, images, audio, or numerical values,
- Is organized into datasets that contain many observations (data instances).
- Datasets are used to train machine learning models, which learn to identify patterns and relationships in the data to make predictions on new data.
- The quality and quantity of data used to train a model can greatly impact its performance, making data a crucial component of machine learning.

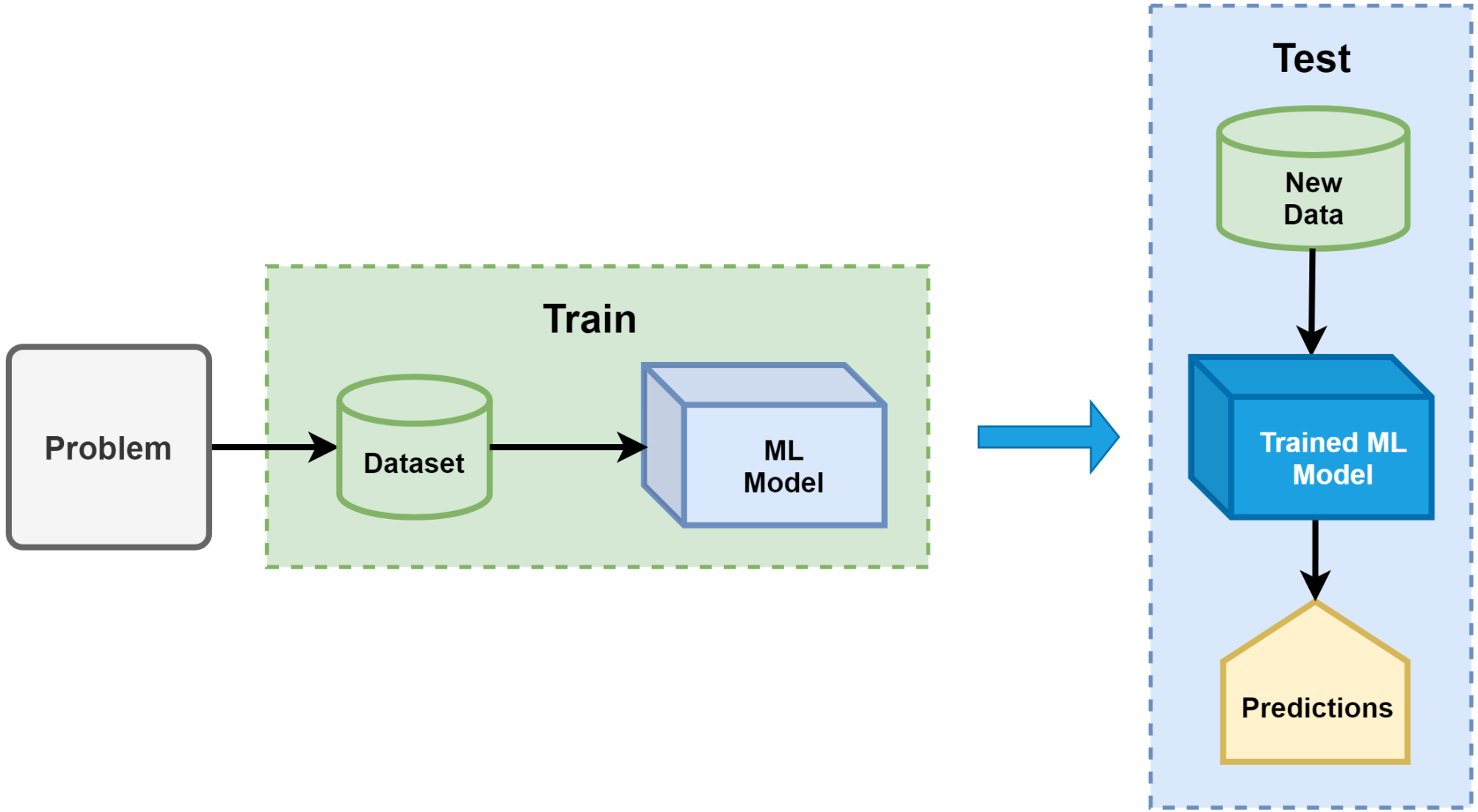
ML - Datasets

- A dataset is a collection of data that is used to train a machine-learning model.
- The dataset consists of data instances (points) each composed of one or several features (a measurable or computable value that describes any system property).
- There is input data (also known as features or independent variables) and output data (also known as labels or dependent variables).
- Input data is used to make predictions or classify data points, while the output data provides the ground truth against which the model's predictions are compared. By training a model on a dataset, the model can learn to recognize patterns and make accurate predictions or classifications on new, unseen data.

ML - The Curse of Dimensionality

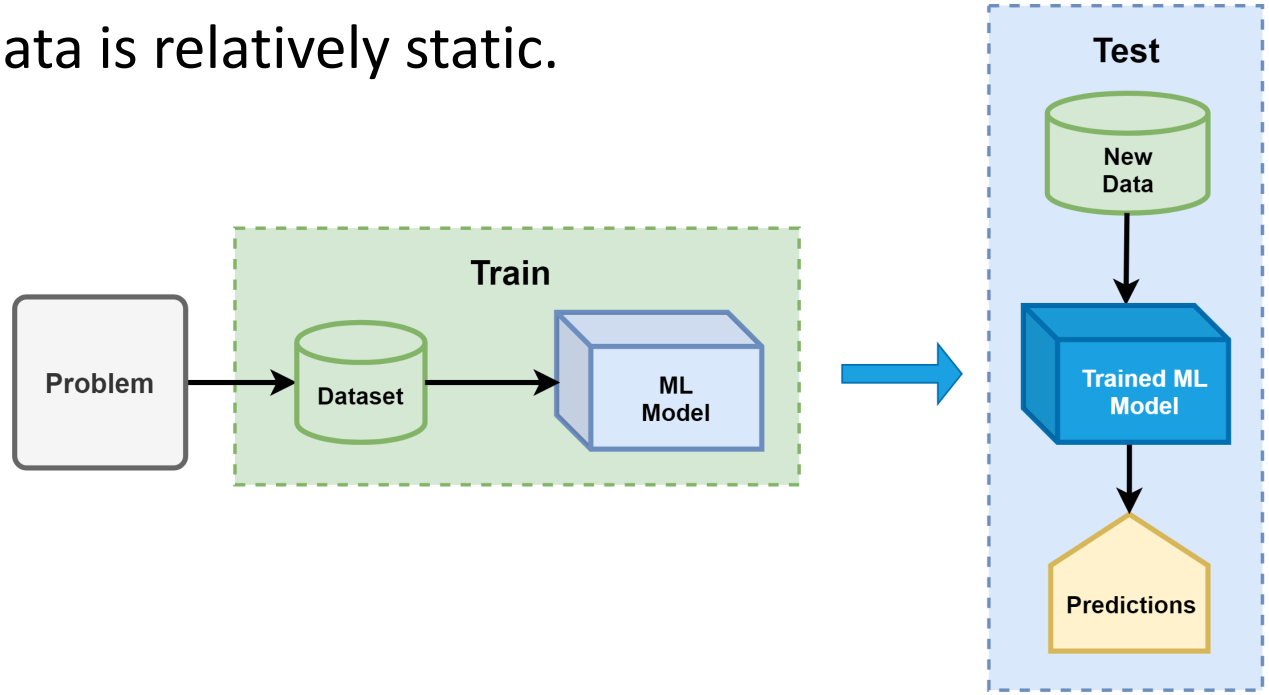
- Refers to the difficulty of accurately modeling and learning from high-dimensional data.
- It's often desirable to work with datasets that have many features or variables. However, as the number of dimensions (e.g. features) in a dataset increases:
  - the amount of data required to effectively learn from the data grows exponentially;
  - It becomes more complex to find meaningful patterns in the data;
  - It becomes more complex to accurately model the relationships between variables, which can lead to overfitting or poor generalization to new data.

ML - *Workflow*


Problem
Train
Dataset
ML
Model
Test
New
Data
Trained ML
Model
Predictions

ML - Type of Learning – Offline Learning

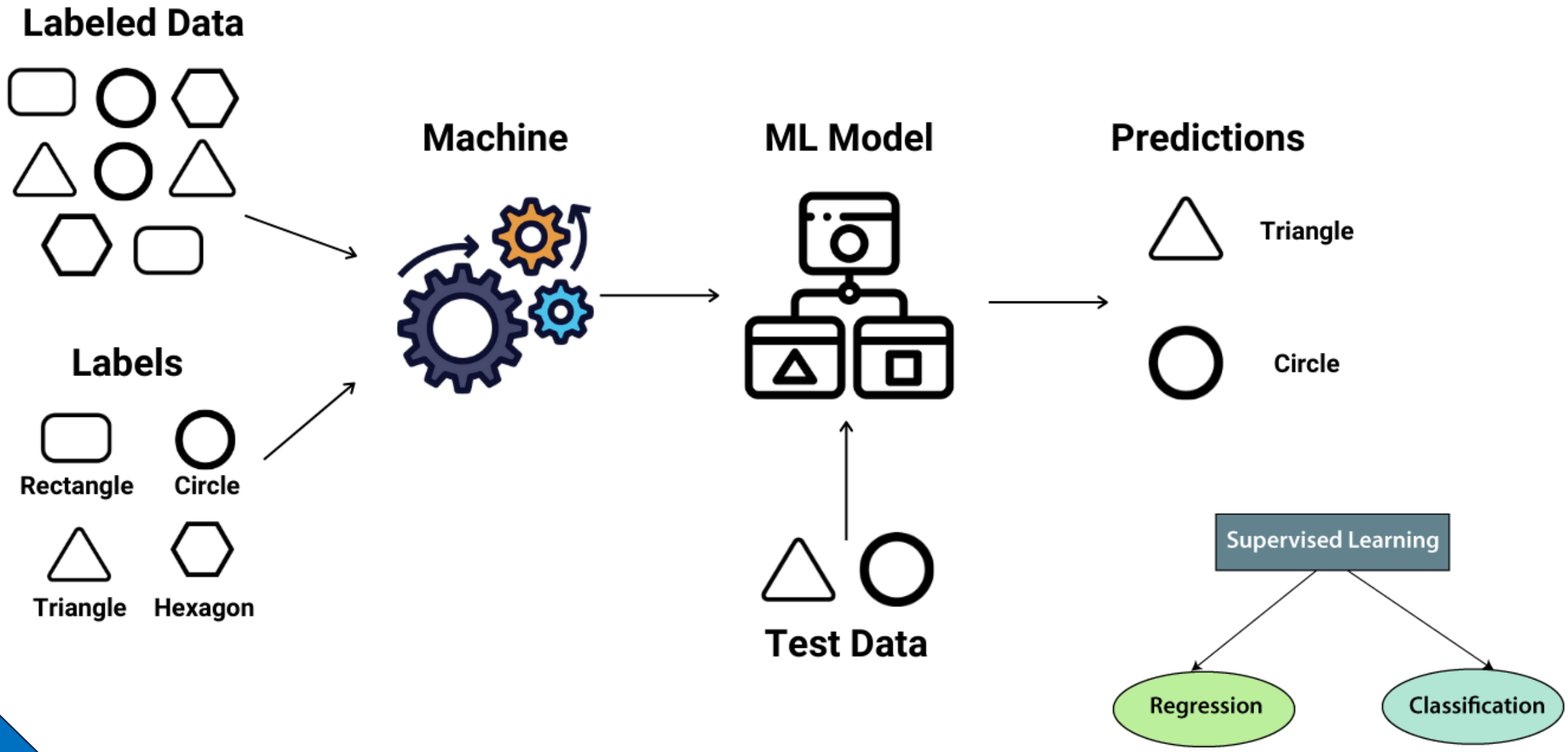
- It’s the conventional and most used approach.
- **Offline (Batch)**
  - Training machine learning models on a fixed dataset (**Batch**) and the trained model is used to make predictions on new data;
  - The model is trained once and then is used repeatedly to make predictions on new data;
  - Used in applications where the data is relatively static.



ML - Type of Learning – Online Learning

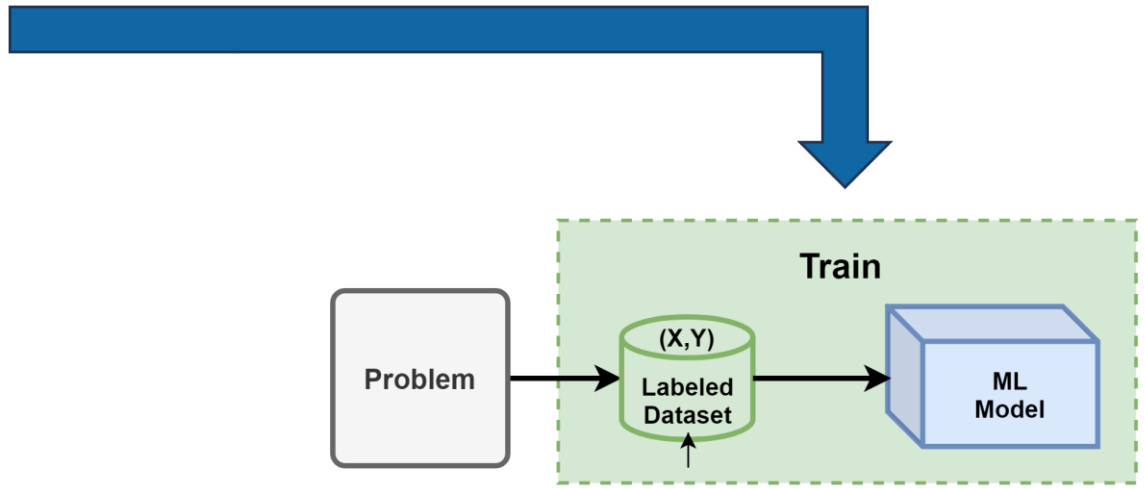
- A.k.a. incremental or streaming ML;
- Machine learning models are updated and improved as new data becomes available;
- Allows models to hopefully improve its accuracy on a specific problem (no concept drift) and to learn and adapt to changing data (with **Concept Drift -** financial trading, online advertising, or recommendation systems);
- Can also be useful in applications where it is not feasible to store all the data, such as in streaming data sources.
- Model is initially trained on a small subset of the data, and then, as new data becomes available, the model is updated with the new information;
- Good for:
  - Handling large volumes of data;
  - Problems with changing patterns in the data (concept drift)
  - To provide real-time predictions.
- However:
  - It can also be more complex to implement;
  - Requires careful monitoring to ensure that the model continues to learn effectively over time.

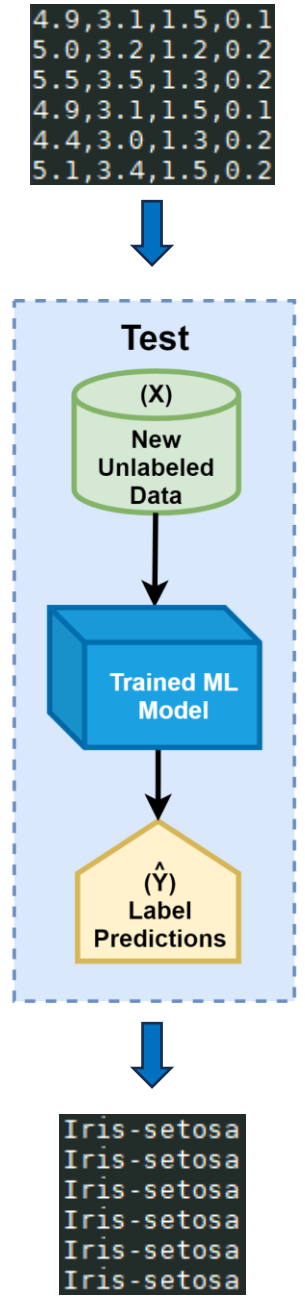
ML – Learning Paradigms - Supervised Learning



ML – Learning Paradigms - Supervised Learning

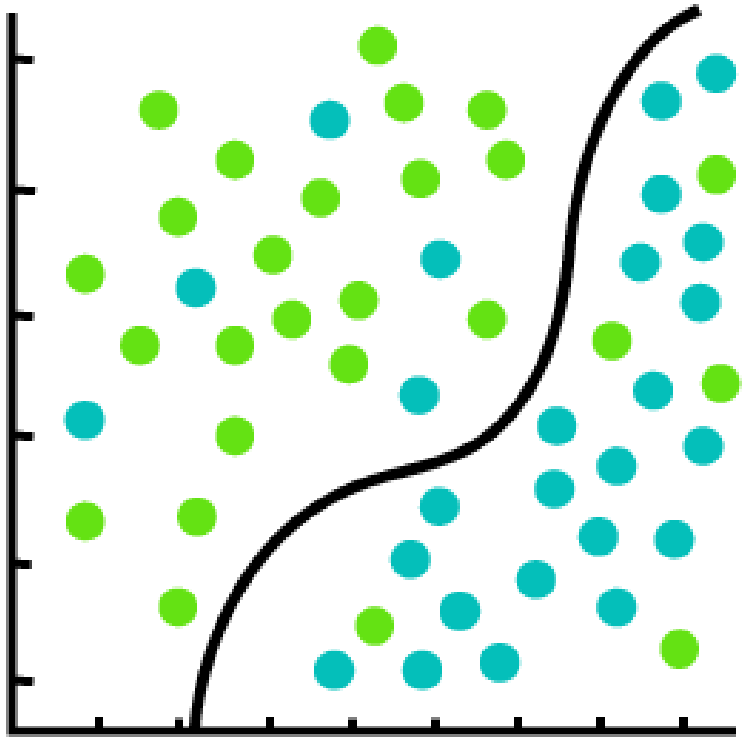
5.0,3.5,1.3,0.3	Iris-setosa
4.5,2.3,1.3,0.3	Iris-setosa
4.4,3.2,1.3,0.2	Iris-setosa
5.0,3.5,1.6,0.6	Iris-setosa
5.1,3.8,1.9,0.4	Iris-setosa
4.8,3.0,1.4,0.3	Iris-setosa
5.1,3.8,1.6,0.2	Iris-setosa
4.6,3.2,1.4,0.2	Iris-setosa
5.3,3.7,1.5,0.2	Iris-setosa
5.0,3.3,1.4,0.2	Iris-setosa
7.0,3.2,4.7,1.4	Iris-versicolor
6.4,3.2,4.5,1.5	Iris-versicolor
6.9,3.1,4.9,1.5	Iris-versicolor
5.5,2.3,4.0,1.3	Iris-versicolor
6.5,2.8,4.6,1.5	Iris-versicolor
5.7,2.8,4.5,1.3	Iris-versicolor
6.3,3.3,4.7,1.6	Iris-versicolor
4.9,2.4,3.3,1.0	Iris-versicolor



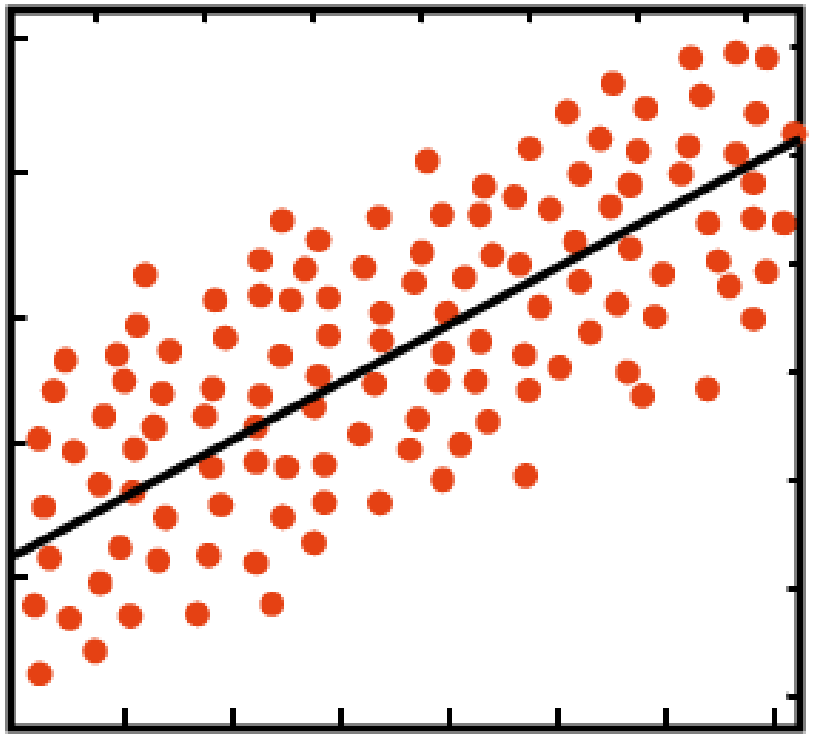


ML – Learning Paradigms - Supervised Learning Problems

- Classification vs Regression

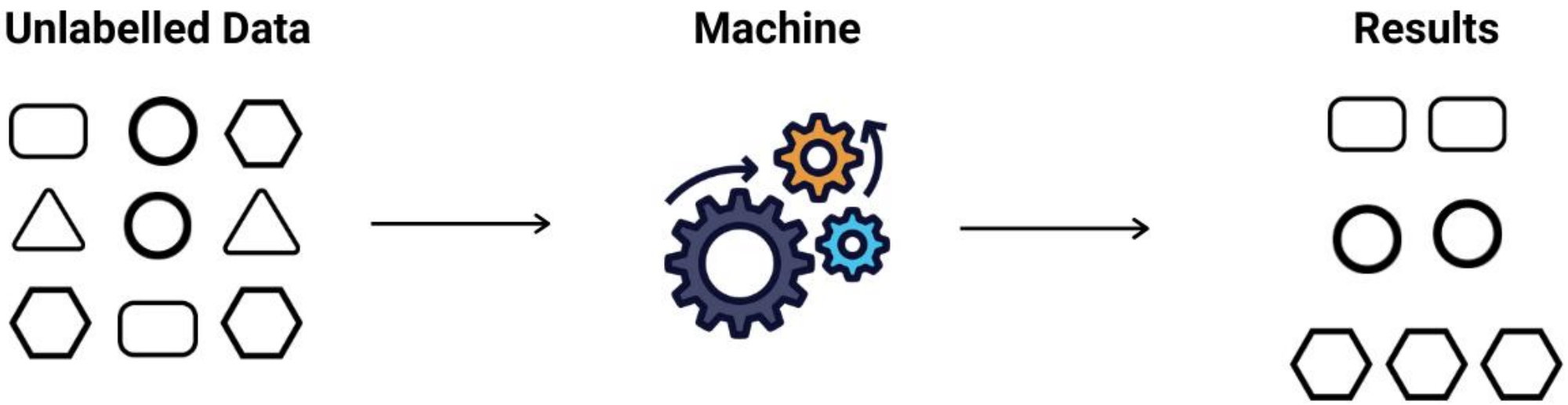


a)

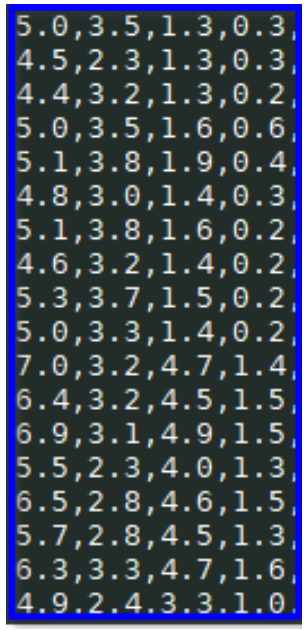


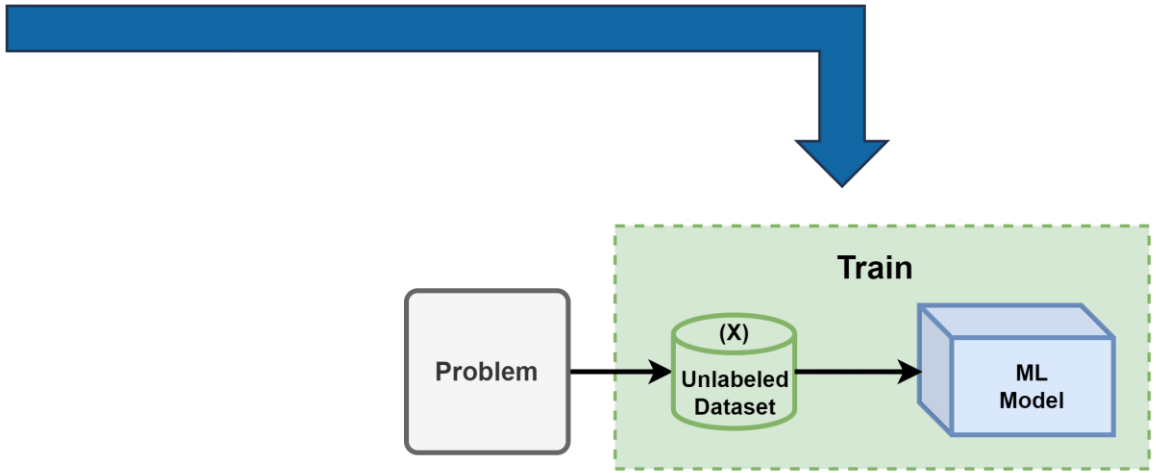
b)

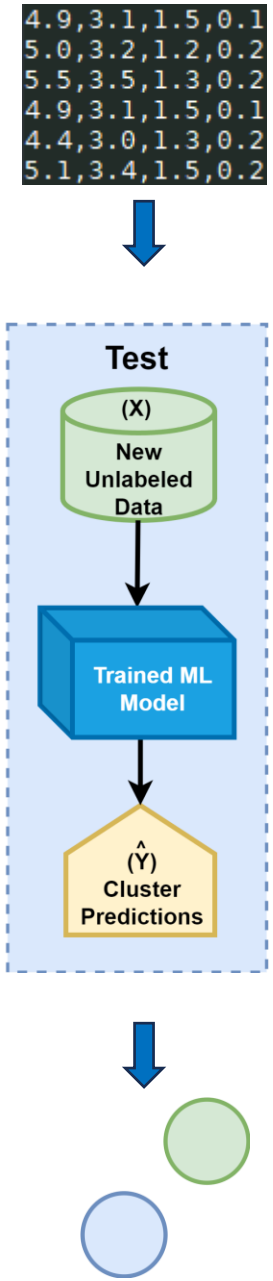
ML – Learning Paradigms - Unsupervised Learning



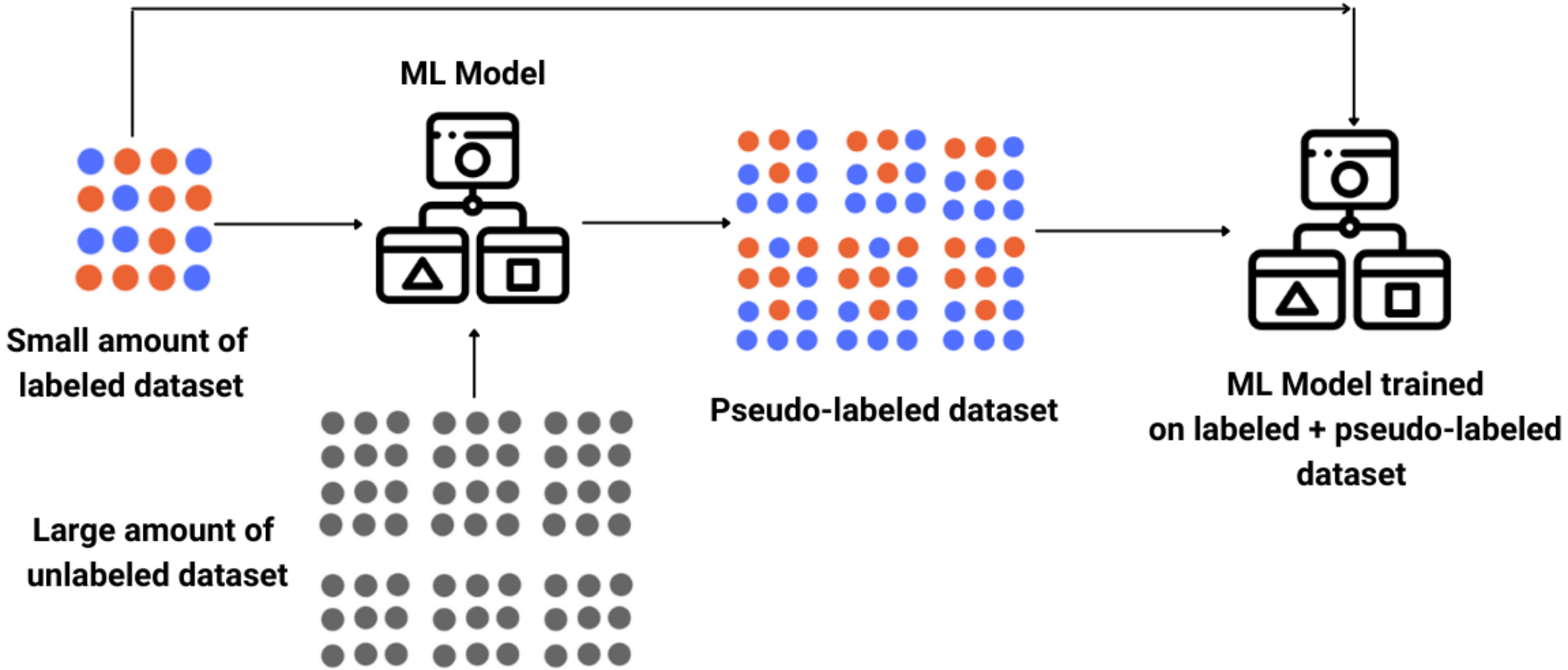
ML - Unsupervised Learning



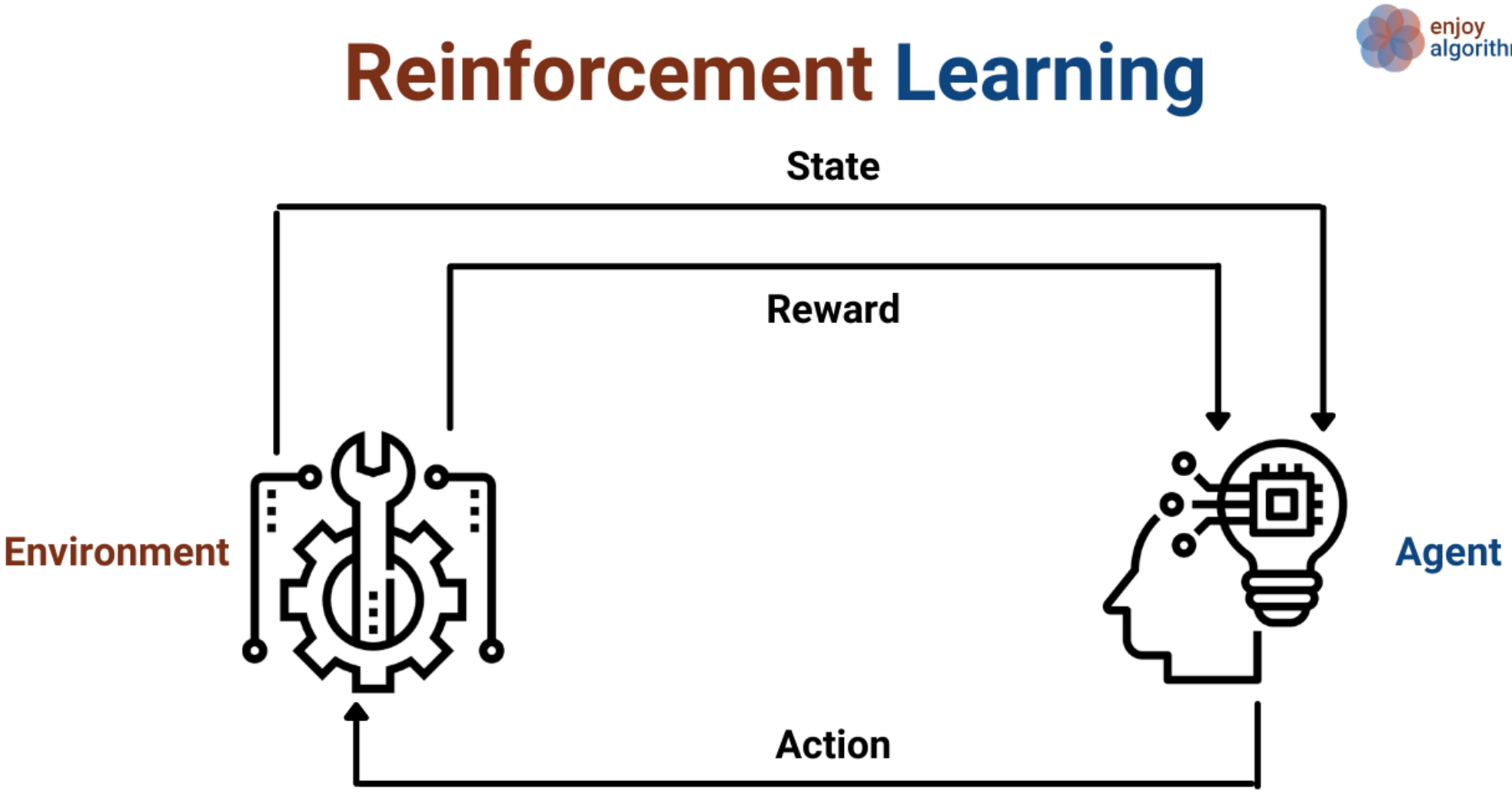




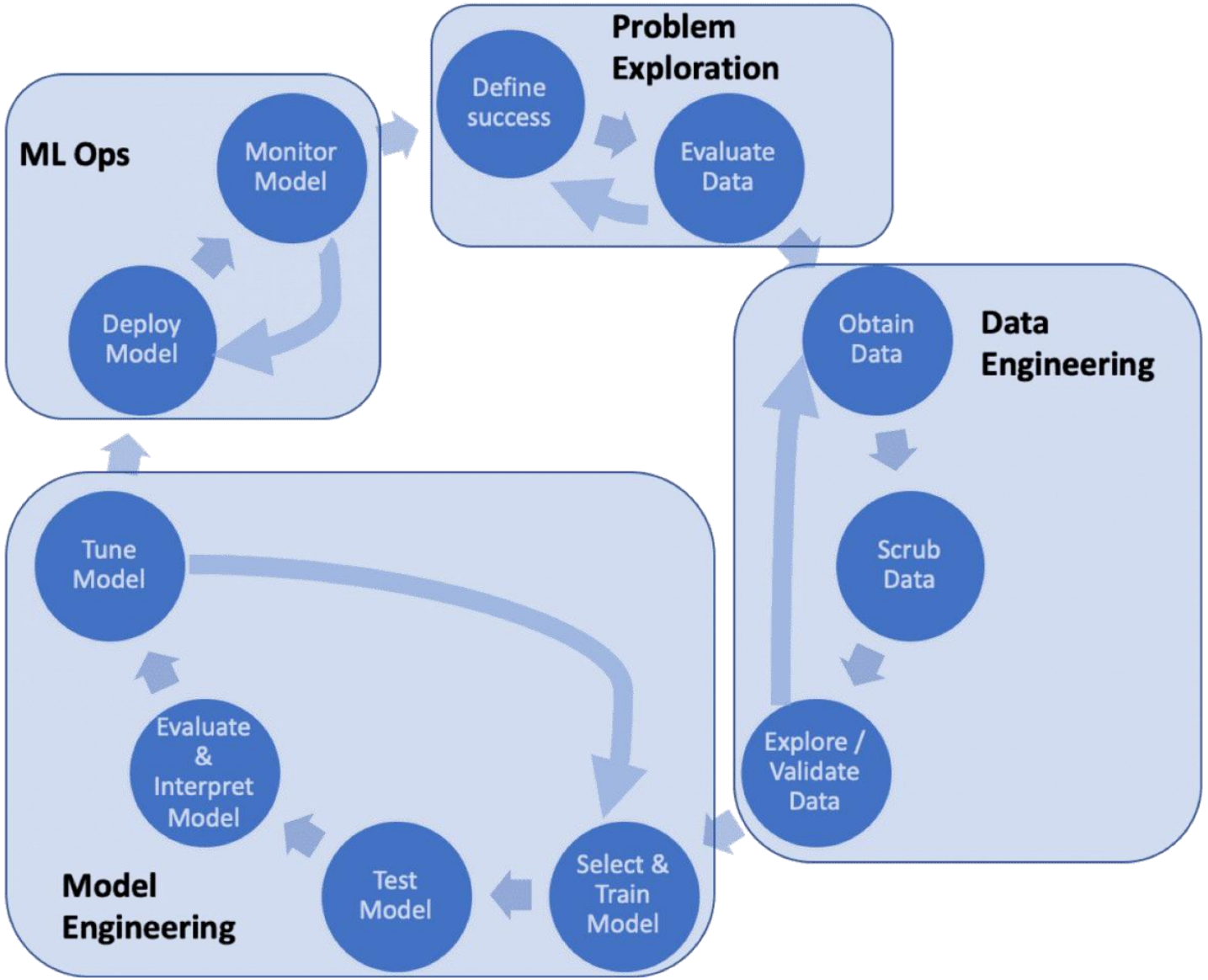
ML – Learning Paradigms - Semi-supervised Learning



ML – Learning Paradigms - Reinforcement Learning



ML - Life Cycle



ML TASKS - Dimensionality Reduction

- The goal is to reduce the number of input variables while preserving the most important information.
- Dimensionality reduction techniques can be used for tasks such as visualization, feature extraction, and data compression.
- The most used dimensionality reduction techniques are:
  - Feature selection;
  - Principal Component Analysis.

ML TASKS - Dimensionality Reduction - **Feature selection**

Feature selection

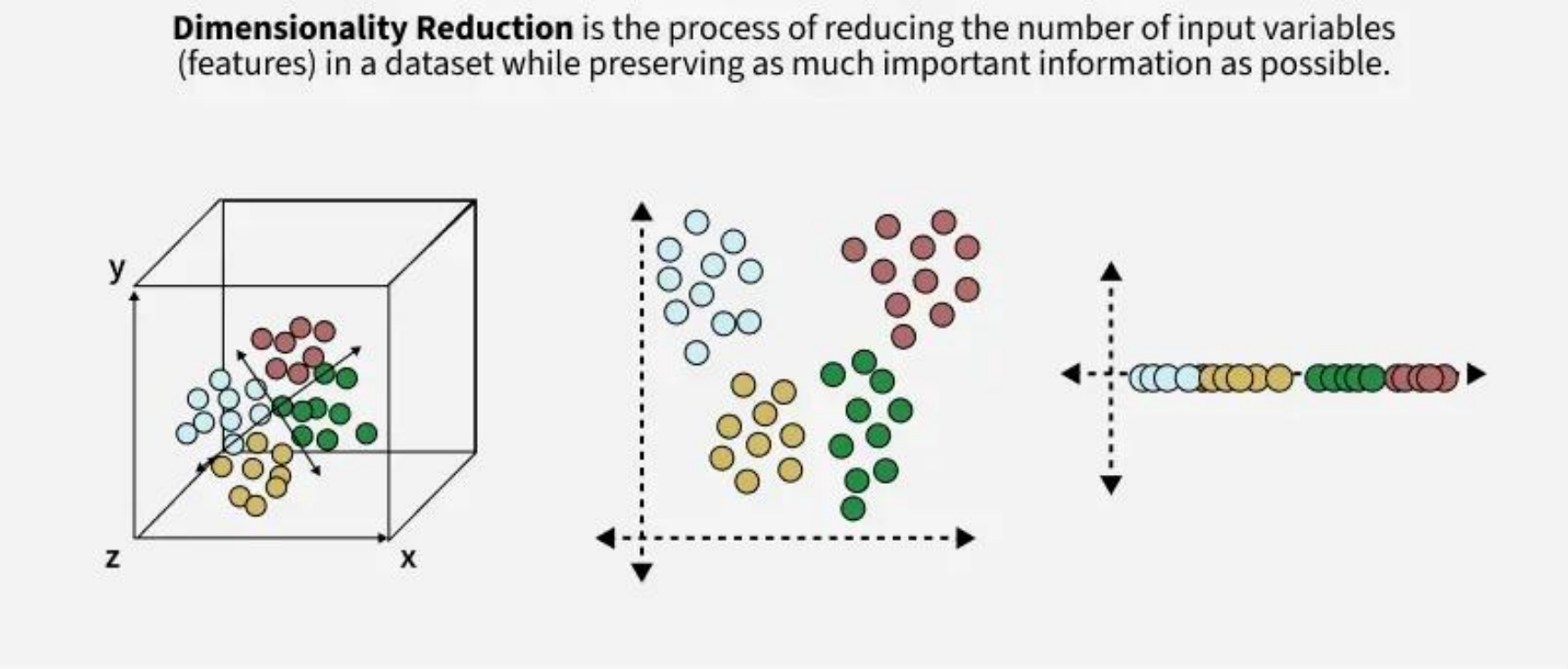
- Techniques used to select the most relevant features or variables from a dataset to use in a model.
- Important because they enable the data dimensionality reduction and this way improve the performance of the model by reducing the risk of overfitting.
- Feature selection algorithms work by ranking the importance of each feature in the dataset based on various criteria, such as correlation with the target variable, information gain, or mutual information. The features are then selected or discarded based on their ranking, with the top-ranked features being selected for use in the model.
- Feature selection can also help improve the interpretability of the model by focusing on the most important features.

ML TASKS - Dimensionality Reduction - **Feature selection**

- Technique used to select the most relevant features or variables from a dataset to use in a model.
- Important because they enable the data dimensionality reduction and this way improve the performance of the model by reducing the risk of overfitting.
- Feature selection algorithms work by ranking the importance of each feature in the dataset based on various criteria, such as correlation with the target variable, information gain, or mutual information. The features are then selected or discarded based on their ranking, with the top-ranked features being selected for use in the model.
- Feature selection can also help improve the interpretability of the model by focusing on the most important features.

ML TASKS - Dimensionality Reduction

- **Principal Component Analysis (PCA)**
- **Linear Discriminant Analysis (LDA)**



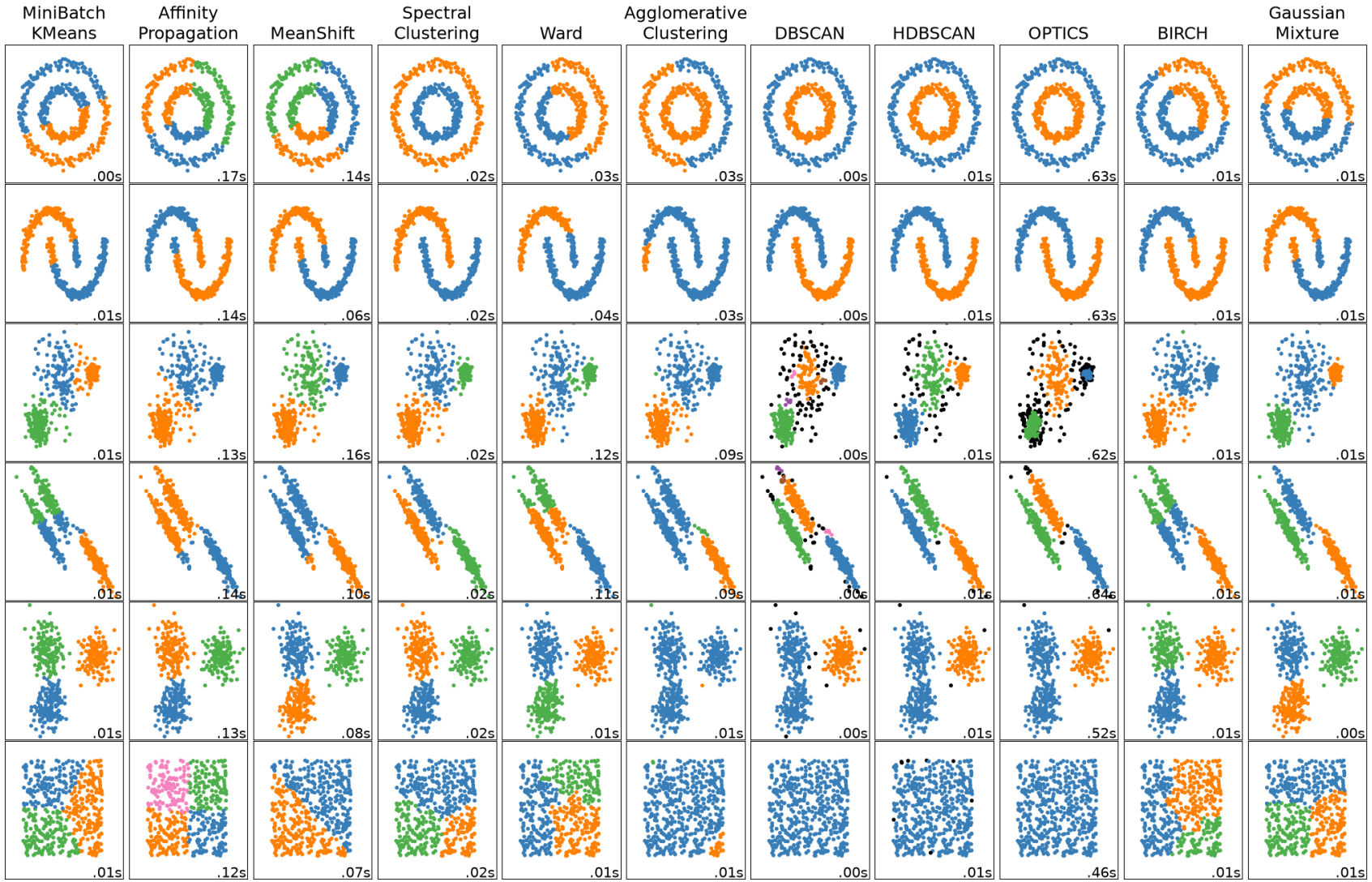
ML TASKS - Dimensionality Reduction - **Principal Component Analysis**

- Principal Component Analysis (PCA) is a classic linear dimensionality reduction method that identifies the directions called principal components in which the data varies the most.
- It works by calculating the covariance matrix of the features and finding its eigenvectors and eigenvalues. The top k eigenvectors with the largest eigenvalues define the new axes that capture the most variance in the data.
- By projecting the original data onto these axes, PCA reduces the number of features while retaining as much information as possible. It's fast, robust and especially effective when the dataset has linearly correlated features. However, it cannot capture complex non linear relationships.

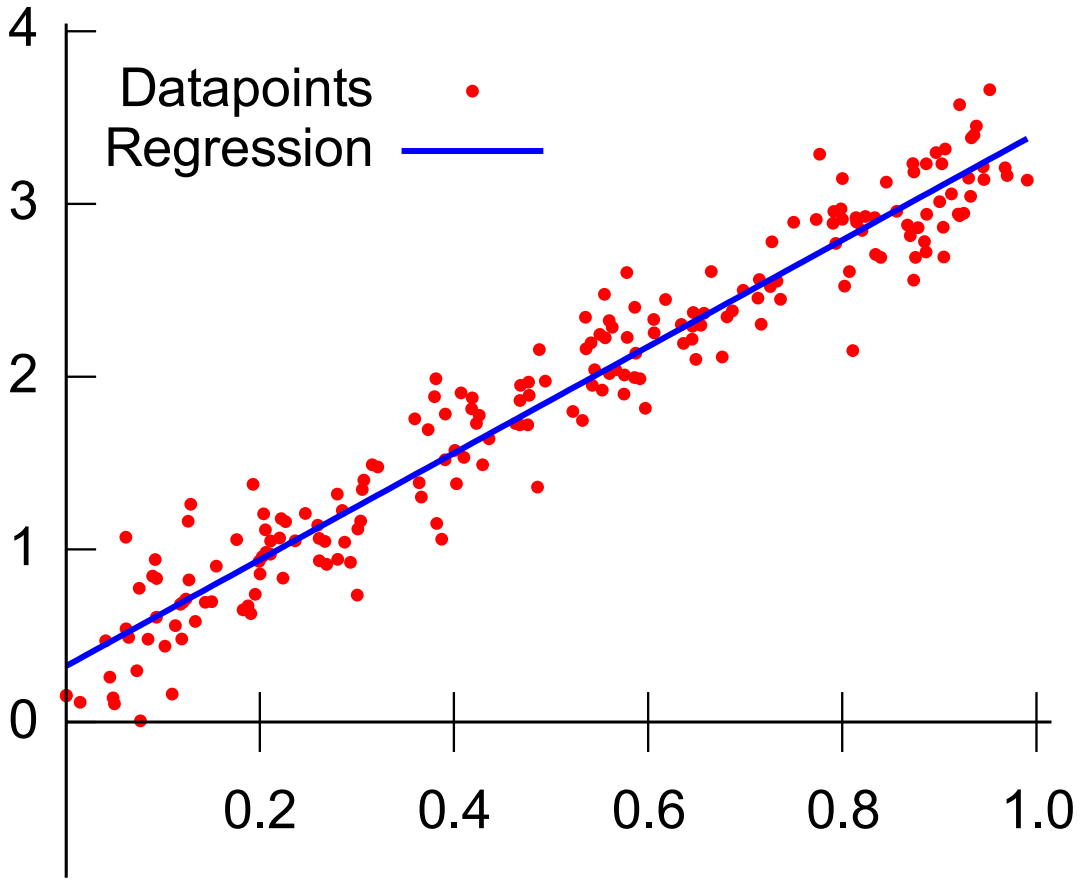
ML TASKS - Clustering



ML TASKS - Clustering



ML TASKS - Regression

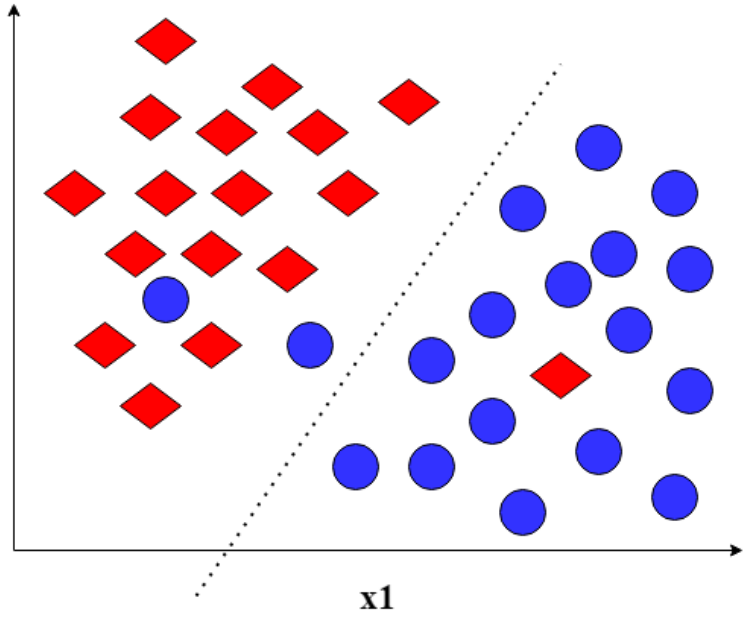


ML Models - **Classification**

- **Classification** is a supervised machine learning technique used to automate tasks done by or with human intervention, that involve sorting data instances into classes or categories.
- The goal of classification is to learn a mapping function that can assign each input data instance into one of several predefined categories.
- In classification, the target variable is categorical (discrete).
- Example: Spam Detection! Each email can be classified as spam or not based on its features.

ML TASKS - Classification - Binary Classification

- **Binary classification** is a type of machine learning where the model makes a simple **yes/no decision** for each input.
- Binary classification = **one question, two answers.**
- **What It Does**
  - Given some data (like an email, image, or patient record), the model answers **one question with two possible answers**:



Examples	Question	Class 0	Class 1
Email	Is this spam?	Not spam	**SPAM**
Medical	Does patient have disease?	Healthy	**DISEASE**
Image	Is this a cat?	Not cat	**CAT**

How It Works (Simple)
Input → Model → Score (0 to 1) → Decision
"Is this spam?" 0.85 → SPAM (if > 0.5)
Score > 0.5 = Class 1 (YES)
Score < 0.5 = Class 0 (NO)

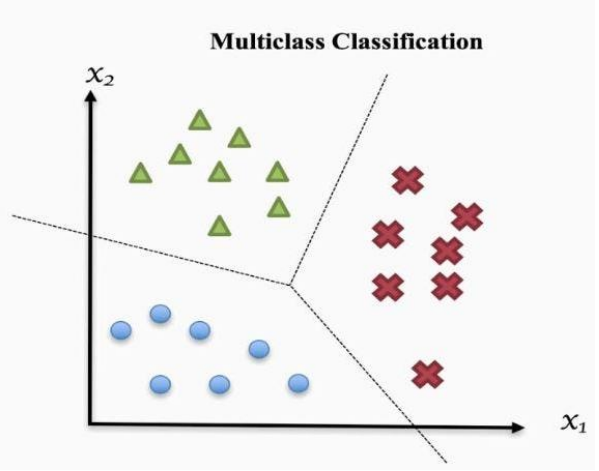
ML TASKS - Classification - Multi-Class Classification

- **Multi-class classification** is machine learning where the model chooses **one answer from 3+ possible categories** (not just yes/no).

Type	Question Example	Possible Answers
Binary	"Is this spam?"	Yes / No
Multi-class	"What fruit is this?"	Apple / Banana / Orange / Pear



- Input: Photo of fruit → Model → Apple: 0.15 | Banana: 0.78 | Orange: 0.05 | Pear: 0.02.
- Choose"Banana" (highest probability).

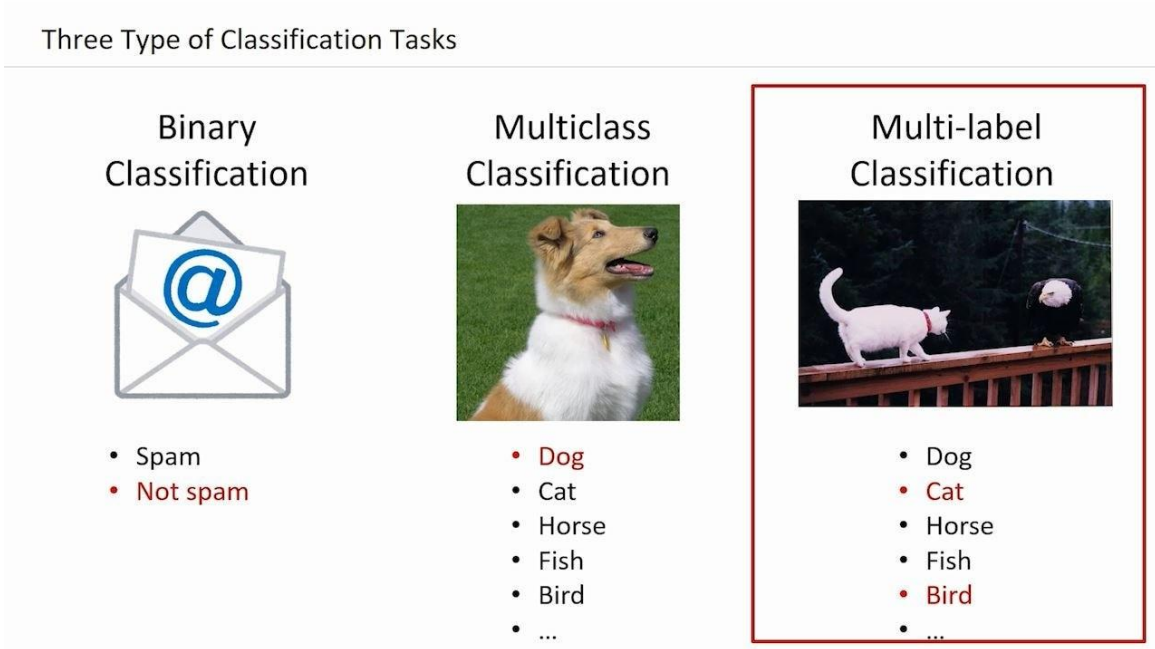


- **Real Examples**
  - **Image recognition**: Cat / Dog / Bird / Fish
  - **Language**: English / Spanish / French / German
  - **Medicine**: Flu / COVID / Allergy / Normal

ML TASKS – Classification – Multi-Label Classification

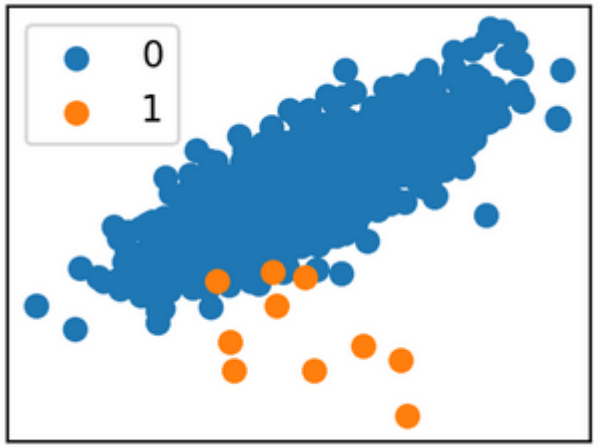
- **Multi-label classification** is machine learning where the model can pick **multiple correct answers at the same time** from several categories.

Type	Question Example	Possible Answers
Binary	"Is this spam?"	Yes / No
Multi-class	"What ONE fruit?"	Apple **OR** Banana **OR** Orange
Multi-label	"What fruits are here?"	Apple **AND** Banana **AND** Orange ✓



- **How It Works**
  - Input: Photo with apples + bananas
  - Model → Apple: 0.92 ✓ | Banana: 0.87 ✓ | Orange: 0.12 ✗
  - Output: ["Apple", "Banana"]** (all > threshold, usually 0.5)
- **Real Examples**
  - **Movie genres**: Action + Thriller + Sci-Fi
  - **Photo tags**: Dog + Park + Sunny
  - **Patient symptoms**: Fever + Cough + Fatigue

ML TASKS – Classification – Imbalanced Classification



- **Imbalanced classification** happens when one class has **way more examples** than the others in your dataset.
- **Exemplo:**
  - Training a spam detector with a 10k emails dataset:
  - ✅ Normal emails: 9,950 (99.5%)
  - ❌ Spam emails: 50 (0.5%)
  - **Problem**: Model learns "always predict Normal" → 99.5% accuracy but **misses ALL spam**.
- Many real world examples:

Scenario	Majority Class	Minority Class	Ratio
Fraud Detection	Normal transactions	Fraud	99.9% vs 0.1%
Cancer Diagnosis	Healthy patients	Cancer	95% vs 5%
Churn Prediction	Stay customers	Leave customers	85% vs 15%

- 98% accuracy looks great... but useless for the important 2%.
- Minority class is usually what matters most (fraud, cancer, churn).

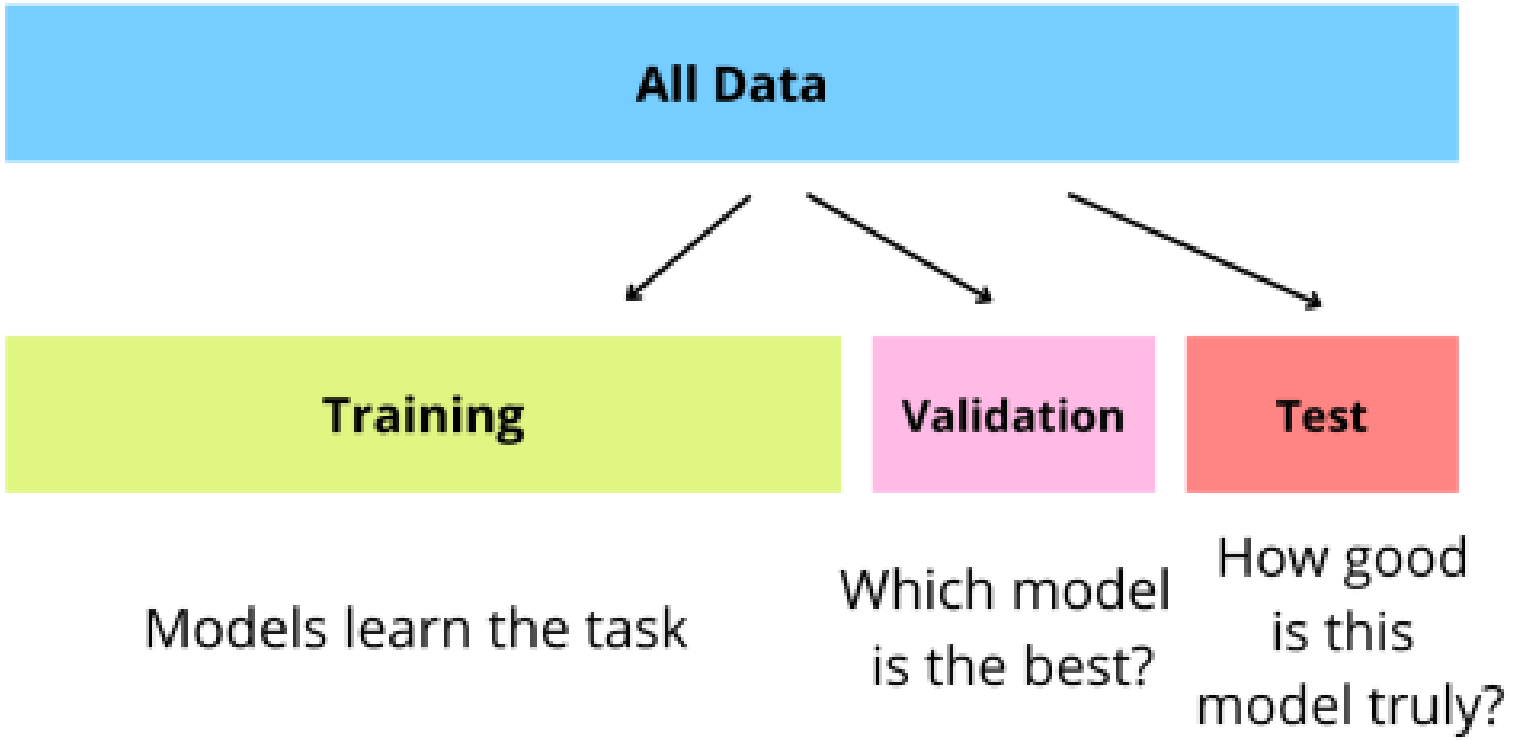
Simple Fix Ideas

- Oversample minority (duplicate rare cases)
- Undersample majority (remove normal cases)
- Use F1-score instead of accuracy (penalizes missing minority)
- Weight classes (make minority errors more expensive)

Bottom line: Imbalanced = "unfair fight" where model cheats by ignoring the rare (but critical) cases.

ML - Model Training And Evaluation

- Train, Validation, and Test Sets

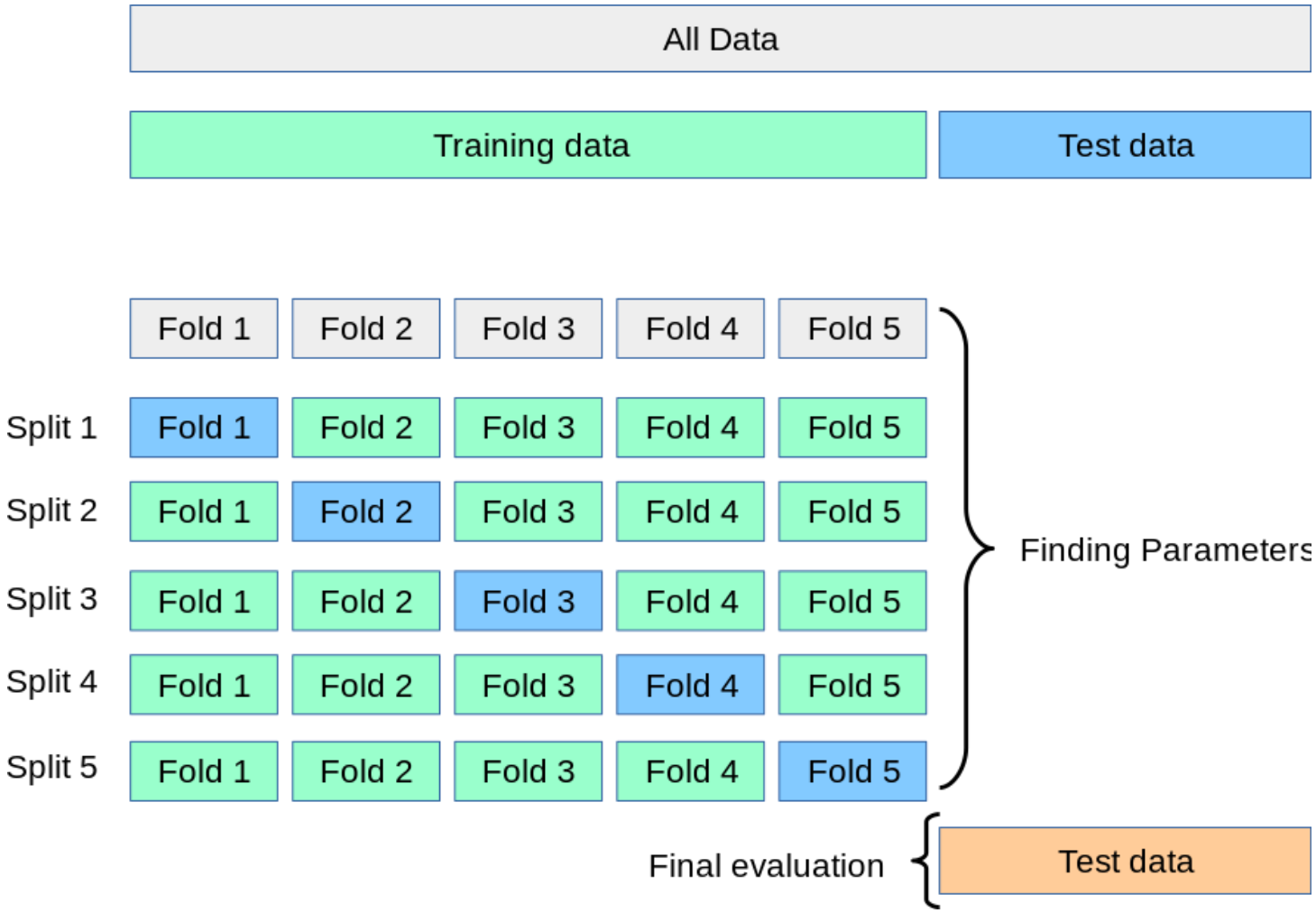


ML - Cross-Validation

- Data is expensive.
- Sometimes the previously referred data partitioning procedure for model training is not sufficient for conveniently tuning the model hyperparameters, testing different properties of the overall dataset, iterating the training process, and overcoming the limitations of a small dataset.
- To overcome this problem, a technique called cross-validation is used to increase the training effectiveness and reduce overfitting manly because it:
  - Helps to estimate the model's performance.
  - Prevents over-reliance on a particular subset of the data.
  - Helps to tune hyperparameters.

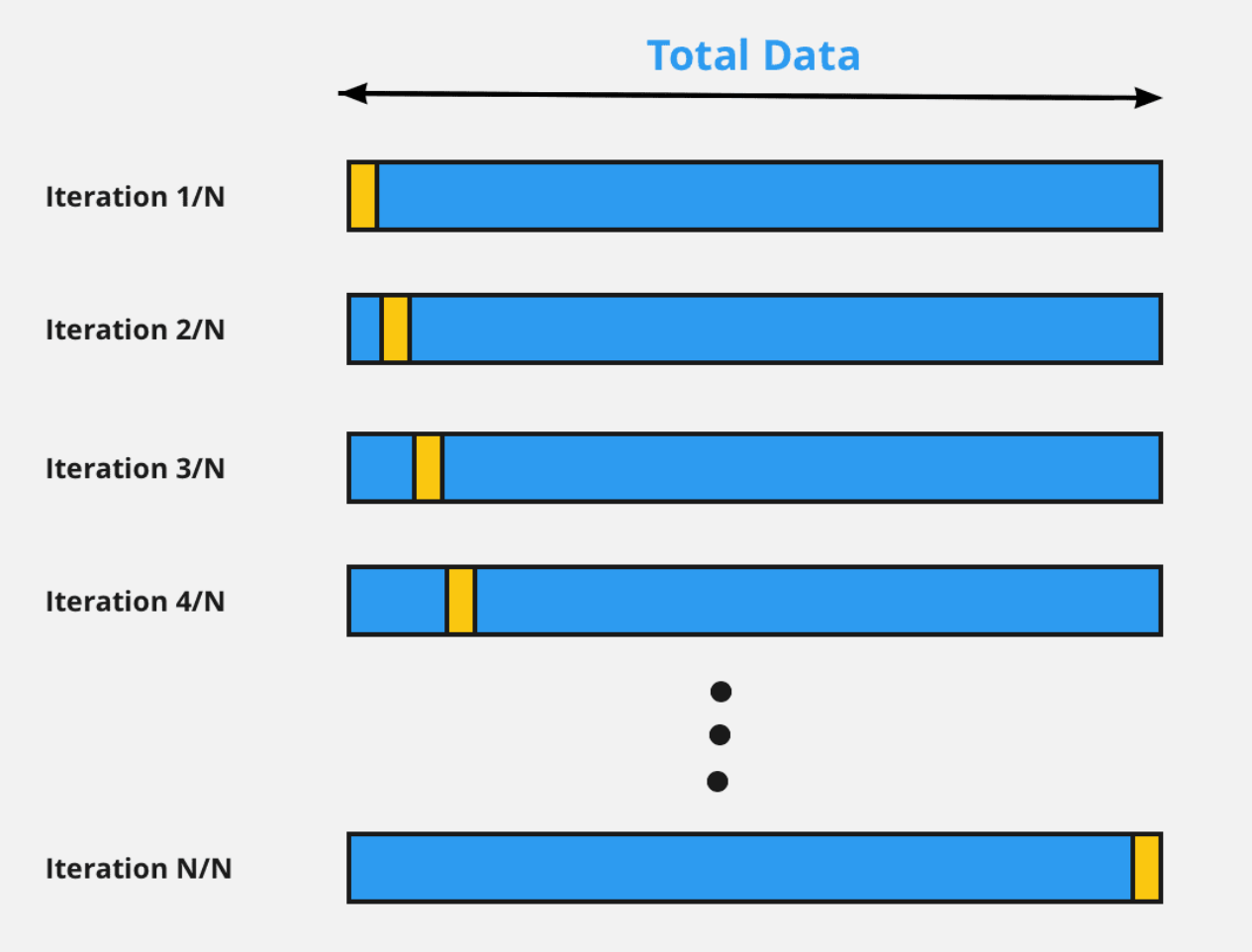
ML - Cross-Validation

- K-fold



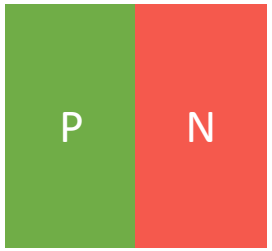
ML - Cross-Validation

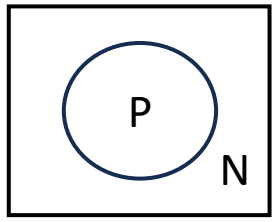
- Leave-One-Out cross-validation (LOOCV)

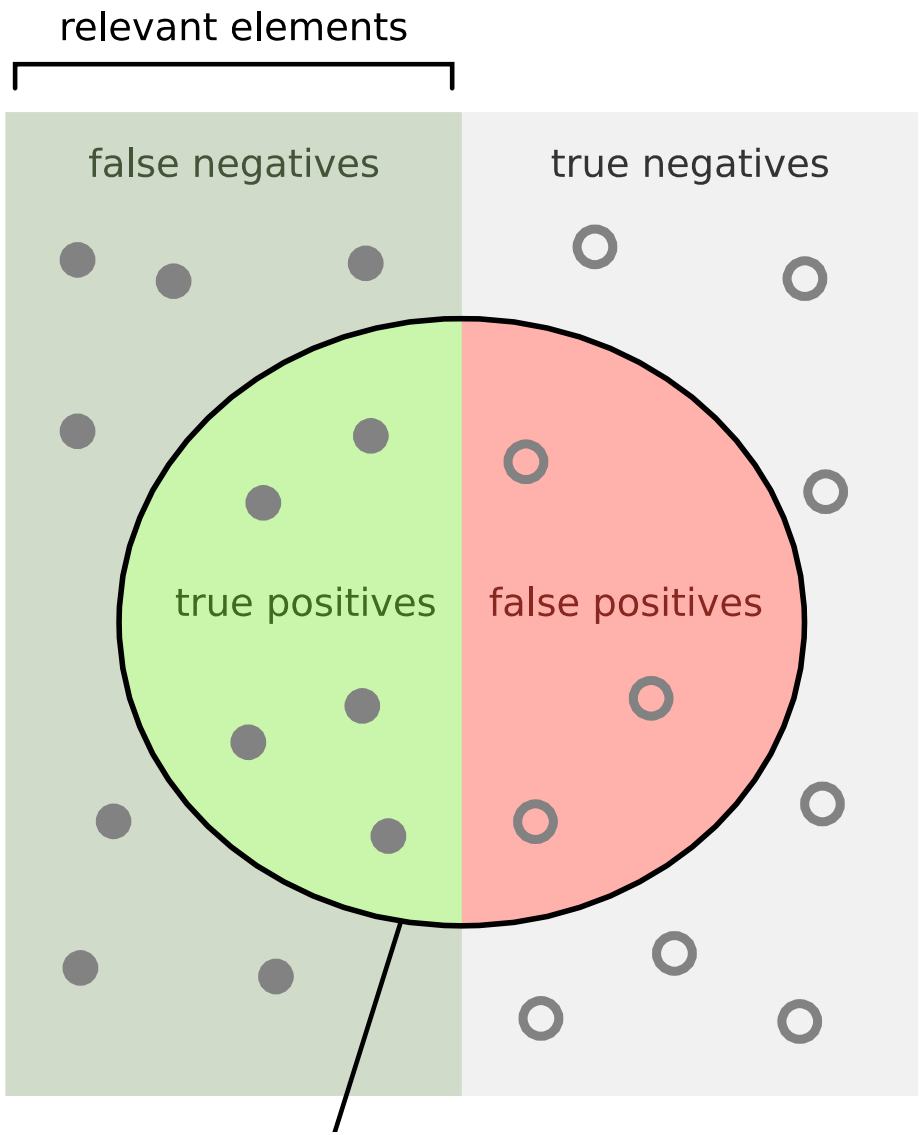


ML – Classification - Performance Metrics

- The performance metrics aim to quantify the accuracy of the classification process, i.e., how accurate predictions were against the real classes. This implies the use of labeled datasets in the testing process of our models.
- At the left (rectangle) we have the positives (relevant elements);
- At the right (rectangle) we have the negatives (non relevant elements);
- The circle represents the classifier decision line. Inside the circle are the elements classified as positives and outside are the elements classified as negatives;







ML – Classification - Performance Metrics

Confusion matrix

- A confusion matrix is a table that summarizes the performance of a classifier model by comparing the predicted labels with the true labels of a set of data points. It is a very useful tool for evaluating the accuracy and performance of a classifier.
- The confusion matrix for a binary classifier has four entries: true positives (TP), false positives (FP), true negatives (TN), and false negatives (FN).

		Actual Class	
		Positive	Negative
Predicted Class	Positive	True positive	False Positive
	Negative	False Negative	True Negative

- From the confusion matrix, several metrics can be computed, including accuracy, sensitivity (recall), specificity, precision, and F1-score. These metrics provide insights into the performance of the classification model and help identify areas for improvement.

ML – Classification - Performance Metrics

Accuracy

- **Accuracy (ACC)** quantifies the percentage of correct predictions divided by the total number of predictions. Accuracy **tells you how many times the ML model was correct overall**.

$$ACC = \frac{Correct\ Predictions}{Total\ Predictions} = \frac{TP + TN}{P + N}$$

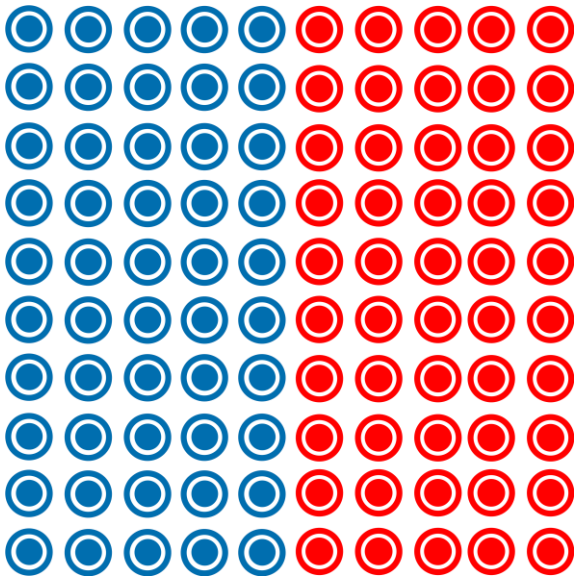
- **Let's see some examples of the calculation of accuracy:**

ML – Classification - Performance Metrics

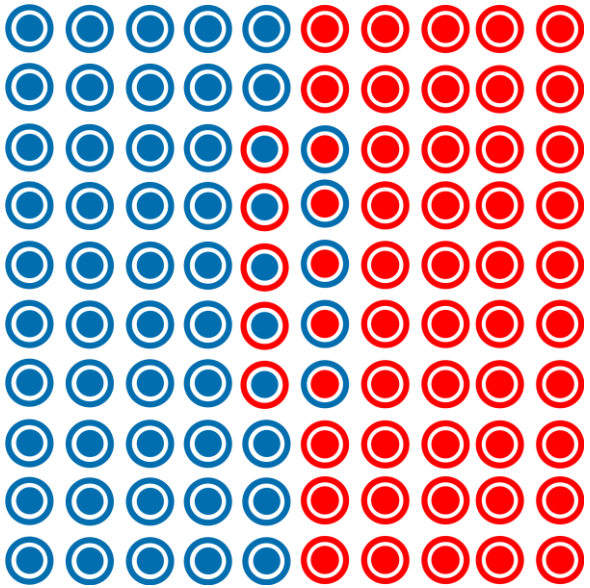
Legend
- Positive classified as positive
- Positive classified as negative
- Negative classified as negative
- Negative classified as positive

Accuracy

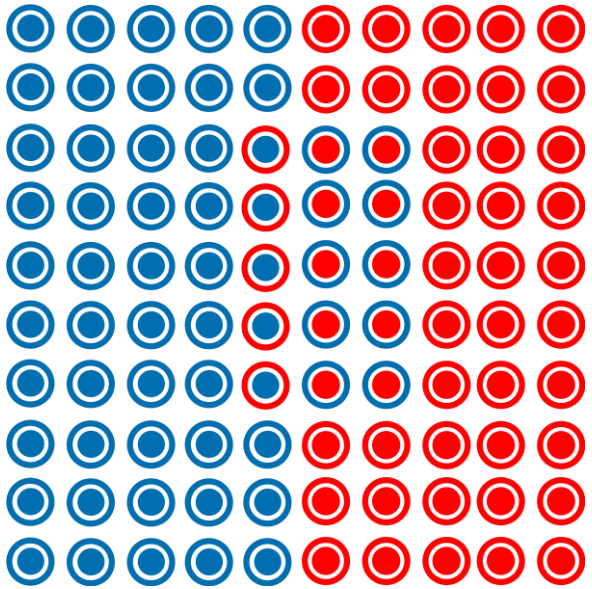
$$ACC = \frac{Correct\ Predictions}{Total\ Predictions} = \frac{TP + TN}{P + N}$$



P = 50 e **N** = 50
TP = 50 e **TN** = 50
Acc = (50+50)/(50+50) = 1
Acc = 100%



P = 50 e **N** = 50
TP = 45 e **TN** = 45
Acc = (45+45)/(50+50) = 0,9
Acc = 90%



P = 50 e **N** = 50
TP = 45 e **TN** = 40
Acc = (45+40)/(50+50) = 0,85
Acc = 85%

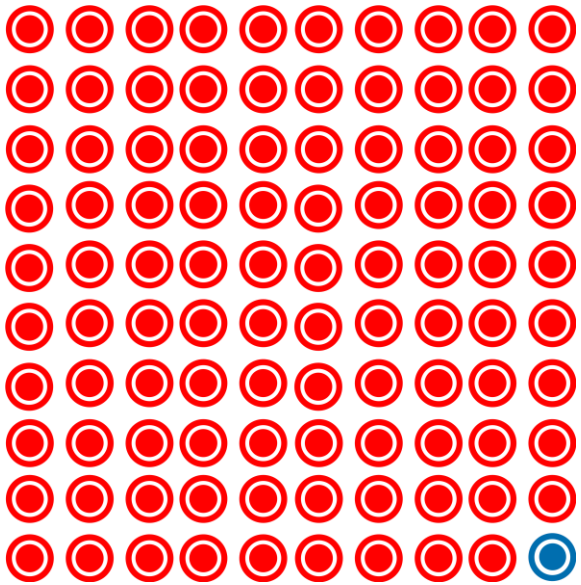
While **accuracy** is a fantastic measure of model performance it **doesn't provide information about the class-wise model performance**. For that, two performance metrics are widely used. **Sensitivity and Specificity**.

ML – Classification - Performance Metrics

Accuracy **how many times the ML model was correct overall**.

$$ACC = \frac{Correct\ Predictions}{Total\ Predictions} = \frac{TP + TN}{P + N}$$

- Additionally, accuracy is not adequate for problems with imbalanced classes.
- E.g. Cancer diagnostic.
  - 100 images where typically at most 1 of them represents the presence of a cancer. What is the accuracy if the model classifies all images are normal?



99% accuracy!

ML – Classification - Performance Metrics

Sensitivity - True Positive Rate (TPR), hit rate or recall

- Is a performance metric that measures the ability of a model to correctly identify positive instances. It measures the proportion of true positives (TP) out of all the actual positive instances in a dataset.

$$TPR = \frac{TP}{P} = \frac{TP}{TP + FN}$$

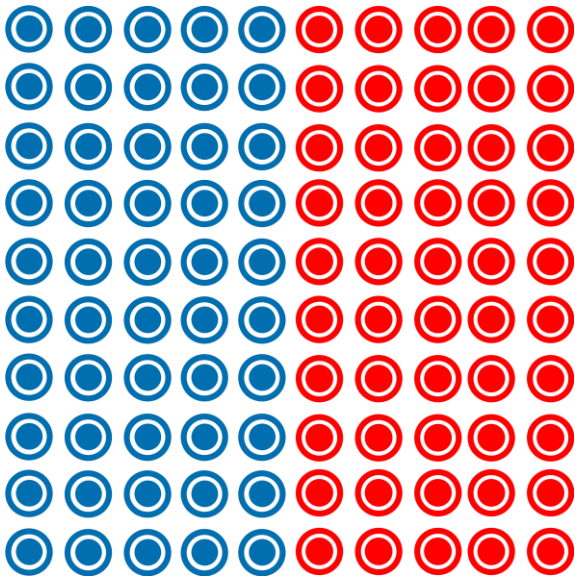
- A high sensitivity value indicates that the model is good at identifying positive instances, while a low sensitivity value indicates that the model is missing a significant number of positive instances.
- Sensitivity is particularly important in applications where missing a positive instance can have serious consequences. For example, in medical diagnosis, high sensitivity is desirable because it means that the model is good at detecting the presence of a disease, even if it leads to a higher rate of false positives.

- **Let's see an example:**

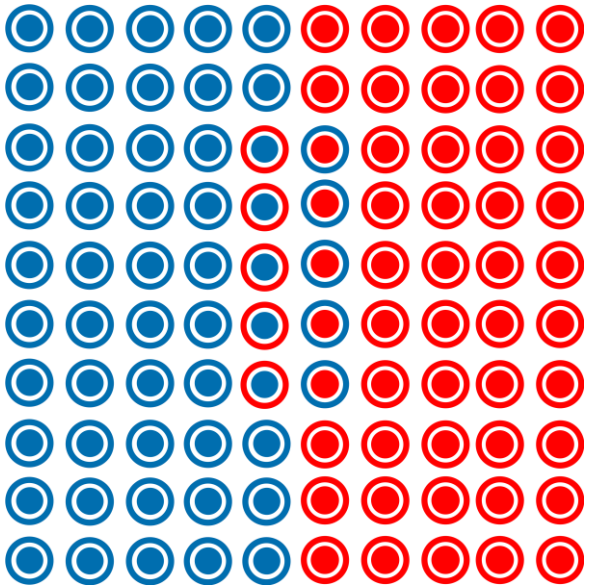
ML – Classification - Performance Metrics

Sensitivity - True Positive Rate (TPR), hit rate or recall

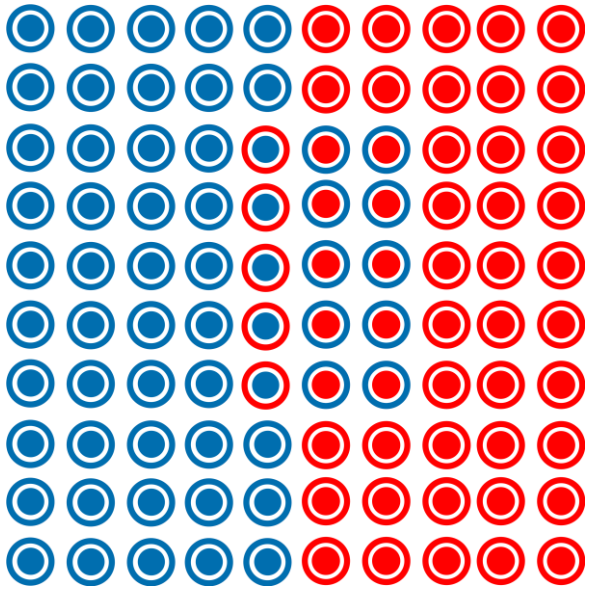
$$TPR = \frac{TP}{P} = \frac{TP}{TP + FN}$$



P = 50 e **N** = 50
TP = 50 e **TN** = 50
TPR = 50/50 = 1
TPR = 100%



P = 50 e **N** = 50
TP = 45 e **TN** = 45
TPR = 45/50 = 45/(45+5) = 0,9
TPR = 90%



P = 50 e **N** = 50
TP = 45 e **TN** = 40
TPR = 45/50=45/(45+5) = 0,9
TPR = 90%

Note: We are detecting the rate at which positive elements are detected

ML – Classification - Performance Metrics

Specificity - True Negative Rate (TNR) or selectivity

- Specificity (TNR – True Negative Rate) is a performance metric that measures the ability of a model to correctly identify negative instances. Specifically, it measures the proportion of true negatives (TN) out of all the actual negative instances in a dataset.

$$\boldsymbol{TNR = \frac{TN}{N} = \frac{TN}{TN + FP}}$$

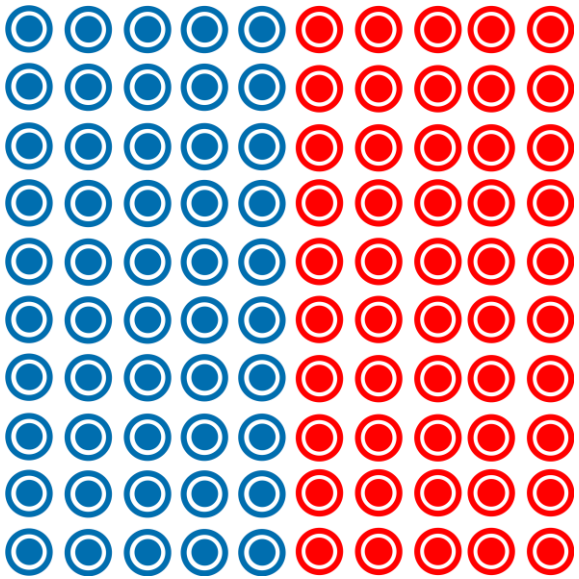
- A high specificity value indicates that the model is good at identifying negative instances, while a low specificity value indicates that the model is incorrectly classifying some negative instances as positive.
- Specificity is particularly important in applications where false positives can have serious consequences. For example, in airport security screening, a high specificity is desirable because it means that the model is good at identifying non-threatening items, even if it leads to a higher rate of false negatives.

- **Let's see an example:**

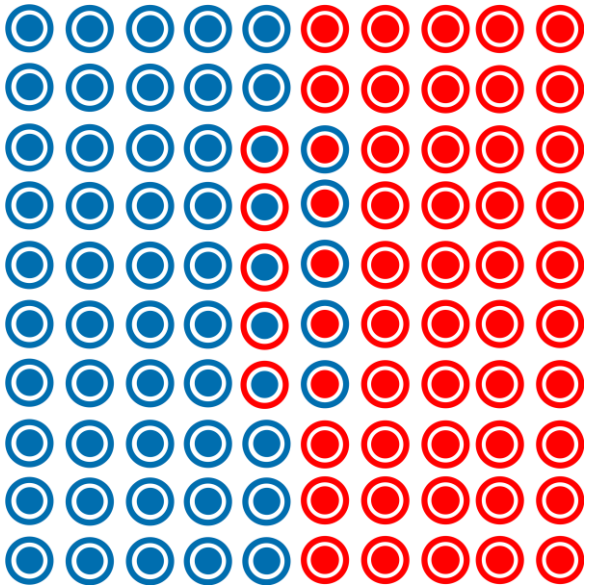
ML – Classification - Performance Metrics

Specificity - True Negative Rate (TNR) or selectivity

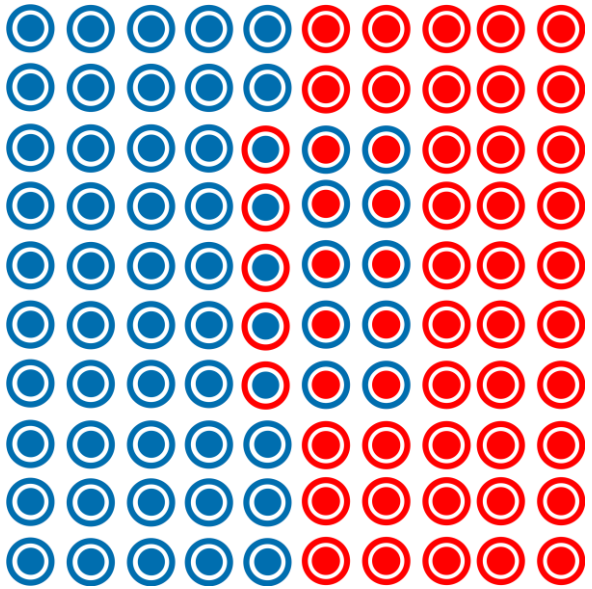
$$TNR = \frac{TN}{N} = \frac{TN}{TN + FP}$$



P = 50 e **N** = 50
TP = 50 e **TN** = 50
TNR = 50/ 50 = 1
TNR = 100%



P = 50 e **N** = 50
TP = 45 e **TN** = 45
TNR = 45/ (45+5) = 0,9
TNR = 90%



P = 50 e **N** = 50
TP = 45 e **TN** = 40
TNR = 40/ (40+10) = 0,8
TNR = 80%

ML – Classification - Performance Metrics

Precision - Positive Predictive Value (PPV)

- Precision (PPV - Positive Predictive Value) measures the proportion of true positive predictions out of all positive predictions made by the model. In other words, it measures how many of the predicted positive instances are actually positive.
- Deals with the quality of positive predictions.

$$PPV = \frac{TP}{TP + FP}$$

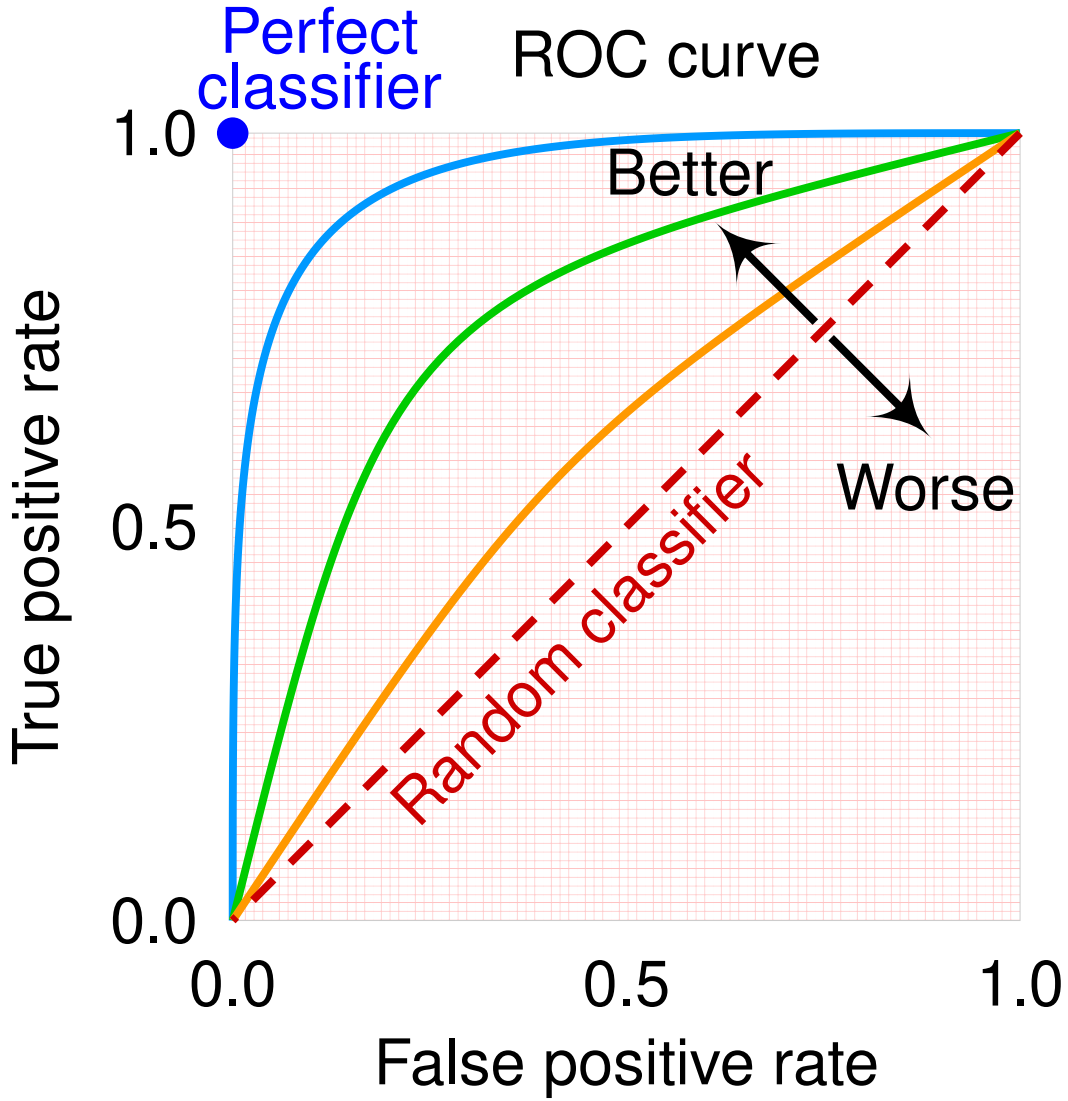
- **More info about this and other related classification measures:**

  [Sensitivity and specificity - Wikipedia](https://en.wikipedia.org/wiki/Sensitivity_and_specificity)

ML – Classification - Performance Metrics

ROC curve

- The receiver operating characteristic (ROC) curve is a graphical representation of the performance of a model in a binary classification problem.
- It is obtained by plotting the true positive rate (sensitivity) against the false positive rate (1 minus specificity) for different classification thresholds (those referred on binary classification slide).
- Each point on the ROC curve represents a different threshold value for the classifier, and the curve is obtained by connecting these points. The closer the curve is to the top left corner of the plot, the better the model's performance.
- The area under the ROC curve (AUC-ROC) is a single number that summarizes the classifier's overall performance across all possible threshold values.
- AUC-ROC values range between 0 and 1, where a value of 0.5 indicates that the classifier performs no better than random, and a value of 1 indicates perfect classification.



ML – Classification – The Bias Variance Trade-off

Definition

- **Bias**

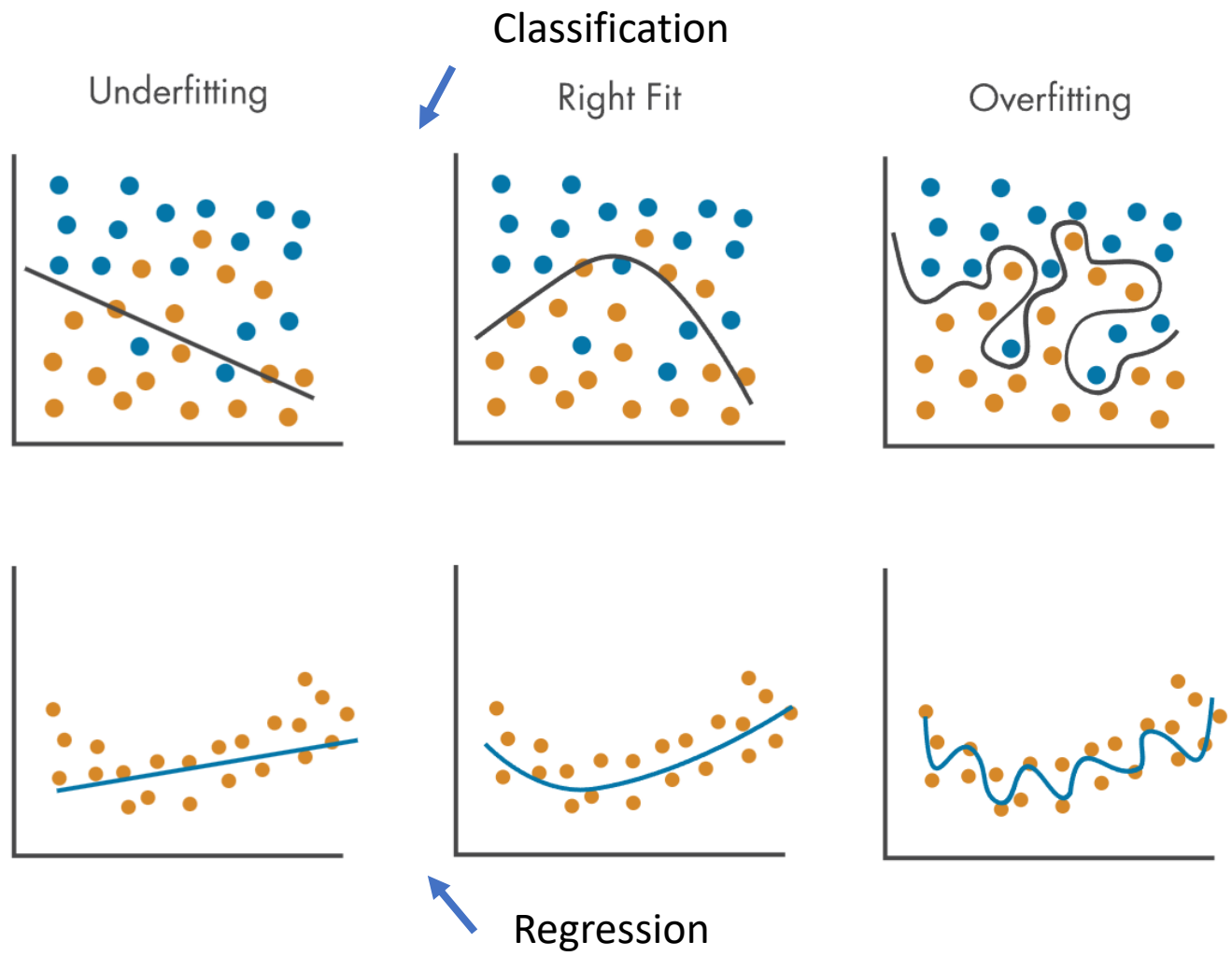
Error that arises from approximating a real-world problem with a simplified model.

A model with high bias is too simple and does not capture the underlying patterns in the data. In other words, it has a systematic tendency to consistently underfit the data.

- **Variance**

Error that arises from the sensitivity of the model to small fluctuations in the training data.

A model with high variance is too complex and fits the training data too closely, including the noise or random fluctuations in the data, leading to poor generalization on new, unseen data. In other words, it has a systematic tendency to overfit the data.



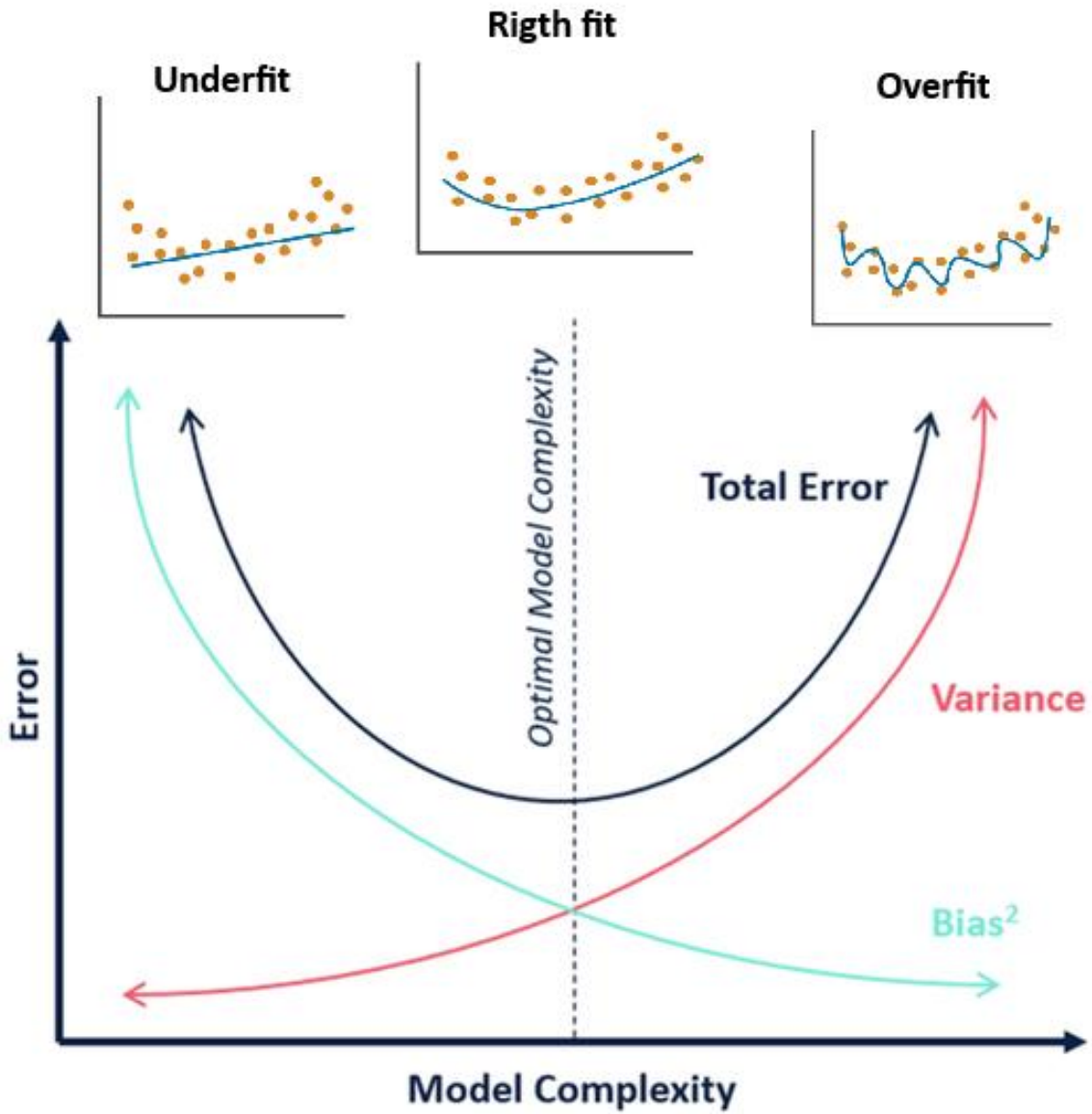
ML – Classification – The Bias Variance Trade-off

- **The bias-variance trade-off**

Refers to the balance between these two types of errors. The goal is to find a model that has low bias and low variance, which means that it captures the underlying patterns in the data without overfitting or underfitting. However, as the model complexity increases, the bias decreases but the variance increases, and vice versa. Therefore, there is a trade-off between the two types of errors, and the goal is to find the optimal balance between them.

Model Error = Bias + Variance + Irreducible error

[Machine Learning Fundamentals: Bias and Variance]



PGSCE-IARSEM

Clustering (ML)

2025/2026

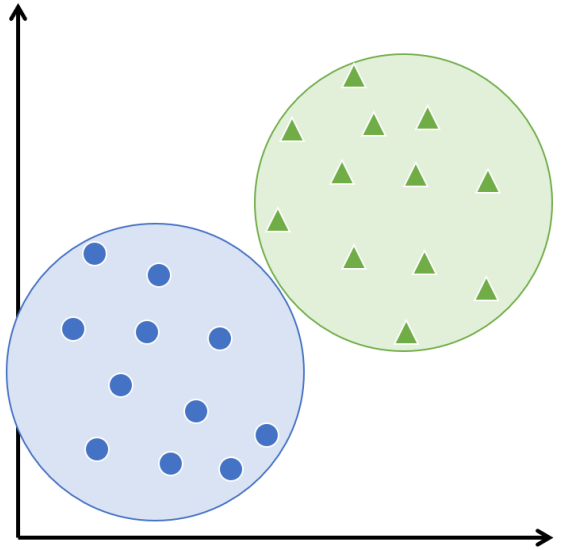
Veríssimo Santos

ML Models - Clustering

- As previously stated:
- **Clustering** is a machine-learning task that involves grouping data points based on their similarity.
- Clustering algorithms automatically discover patterns (groupings in data) without being explicitly told what those groupings should be (**no labels - unsupervised learning**).

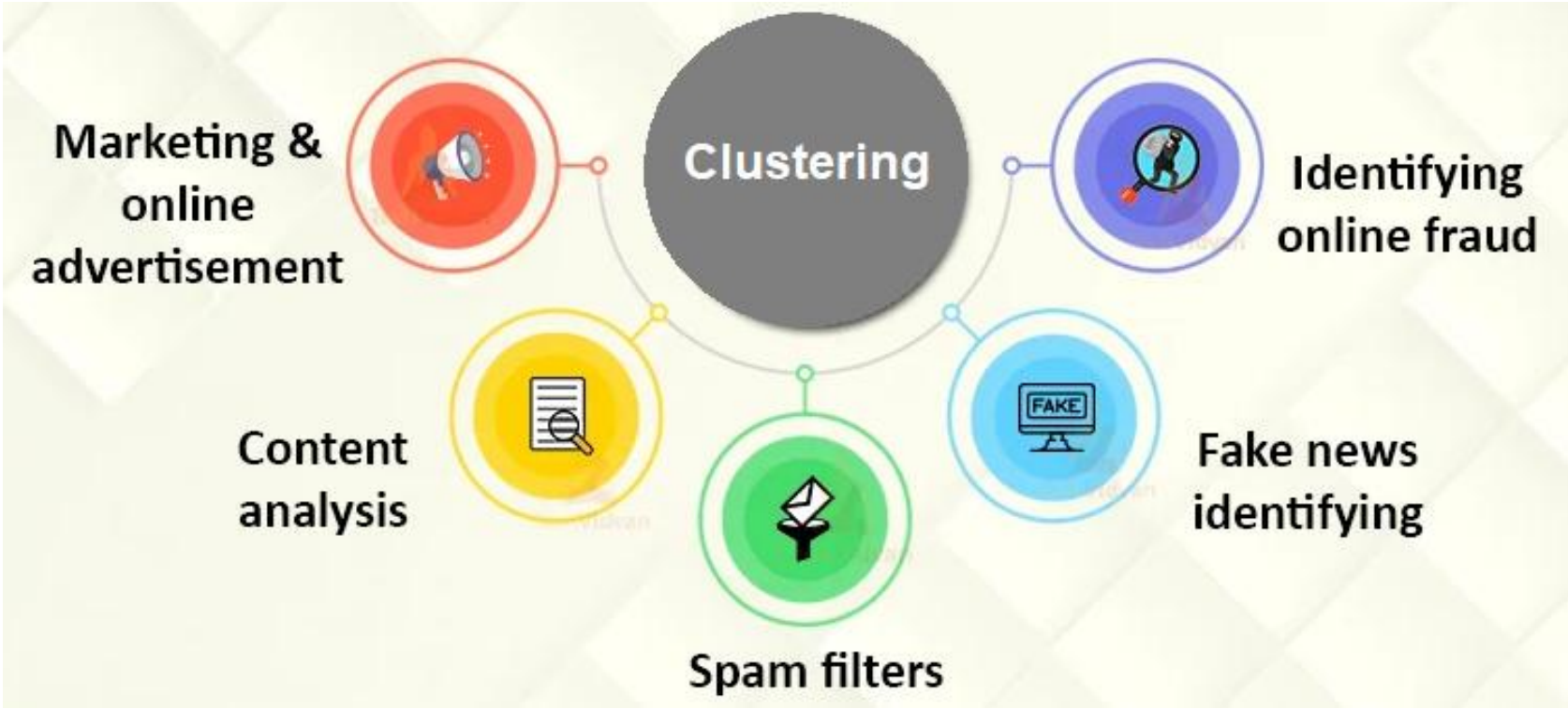
Key Idea:

- Finds **patterns and hidden structures** in the data.
- Groups **similar** data points into **clusters**.
- Unlike classification, **no prior labels exist**.
- The algorithm may use different **distance metrics** or **similarity measures** to determine the degree of similarity between data points. **The goal** is to create internally homogeneous clusters (i.e., data points within a cluster are similar to each other) and externally heterogeneous (i.e., data points from different clusters are dissimilar to each other).



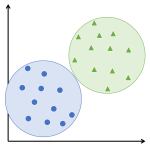
ML Models - Clustering

Clustering applications



Among many others.

ML Models – Clustering Real-World Examples



Business and Marketing

- Customer Segmentation
- Email Marketing
- Market Segmentation

Healthcare

- Disease Diagnosis
- Gene Expression Analysis

Technology and Cybersecurity

- Fraud Detection
- Spam Filtering
- Cyber-Profiling

Social Media and Entertainment

- Streaming Services
- Fantasy Sports Analysis

Urban Planning and Transportation

- Crime Analysis
- Rideshare Optimization

Document Processing

- Text Clustering

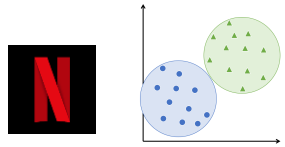
Sports Science

- Player Analysis

Retail Optimization

- Delivery Optimization

ML Models – Clustering Key applications



Clustering has been extensively used by Netflix to enhance its operations and user experience.

Key applications:

1. Content Categorization

Netflix K-means and hierarchical clustering to group similar movies and TV shows

Uses attributes such as genre, cast, director, country, rating, and description

Categorize content into distinct clusters, enabling better organization and discovery of shows on the platform

2. Recommendation Systems

Netflix uses clustering to create personalized recommendations.

By grouping users into clusters based on viewing preferences, Netflix can tailor suggestions for each cluster.

It has established 1,300 recommendation clusters that analyze user data such as viewing history, searches, and ratings to recommend content quickly and effectively.

3. Content-Based Recommender Systems

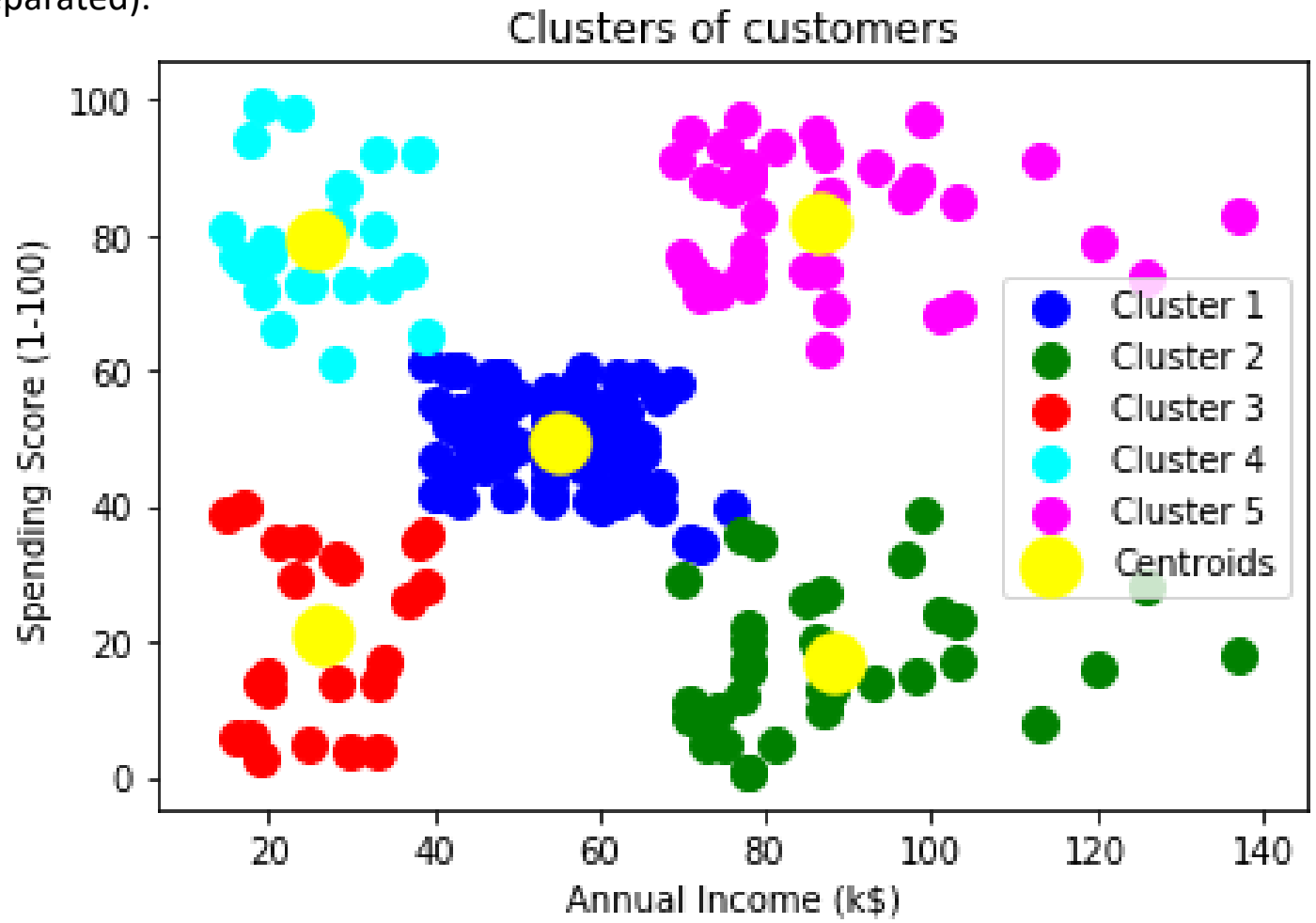
Clustering is integrated with techniques like cosine similarity to build content-based recommendation systems.

These systems suggest shows or movies similar to what a user has already watched by identifying patterns within clusters of content.

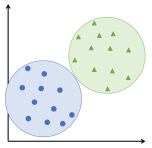
ML Models - Clustering - Key Concepts

What Defines a Good Cluster?

- Homogeneity (points in the same cluster are similar).
- Separation (different clusters are well-separated).



ML Models - Clustering - Key Concepts



Approaches to Clustering

1. **Partitioning-based** [K-Means]
2. **Density-based** [DBSCAN]
3. **Hierarchical** [Hierarchical Clustering]
4. **Model-based** [GMM]

ML Models - Clustering - Approaches to Clustering

1. Partitioning-based (e.g., K-Means) – Fixed number of clusters.

- Partitioning-based clustering divides the dataset into a predefined number of clusters, typically by optimizing an objective function like minimizing intra-cluster distances (e.g., sum of squared distances).

How It Works

- **Initialization:** Select k initial cluster centroids randomly (or using heuristics).
- **Assignment Step:** Assign each data point to the nearest cluster centroid based on a distance metric (e.g., Euclidean distance).
- **Update Step:** Recalculate the centroids of clusters by averaging the points assigned to each cluster.
- **Repeat:** Iterate the assignment and update steps until convergence (e.g., when centroids no longer change significantly, or a maximum number of iterations is reached).

Examples

- **K-Means:** Minimizes within-cluster variance.
- **K-Medoids (PAM):** Uses medoids (actual data points) as cluster centers, making it more robust to outliers.

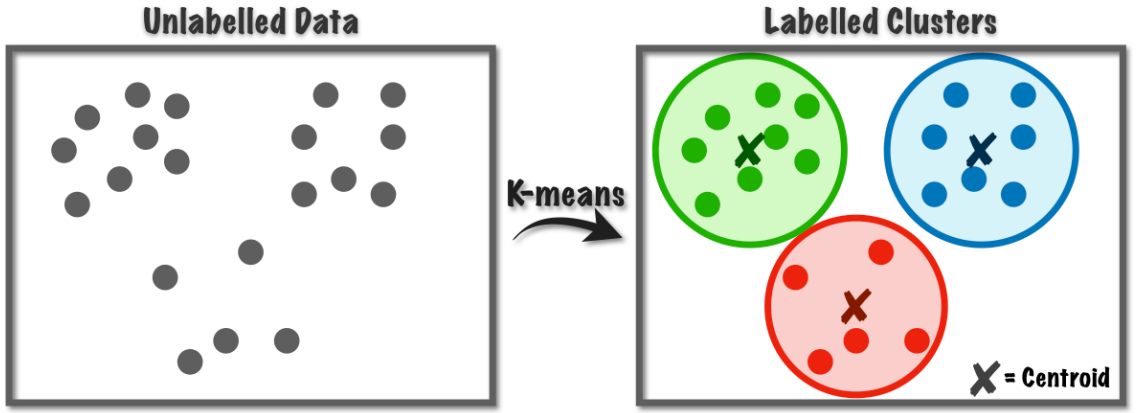
Strengths

- Simple and computationally efficient for large datasets.
- Works well with spherical, evenly-sized clusters.

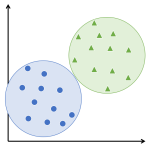
Weaknesses

- Requires the number of clusters (k) to be specified in advance.
- Sensitive to initial centroid placement and outliers.
- Struggles with non-spherical or overlapping clusters.

[K-Means]



ML Models - Clustering - Approaches to Clustering



2. Density-based (e.g., DBSCAN) – Detects arbitrarily shaped clusters

- Density-based clustering groups data points based on regions of high density, identifying clusters as dense regions separated by areas of lower density.

How It Works

- Identify "core points" that have at least minPts neighbors within a radius eps (density threshold).
- Expand clusters by connecting core points and their neighbors.
- Label points that are not part of any dense region as noise or outliers.

Examples

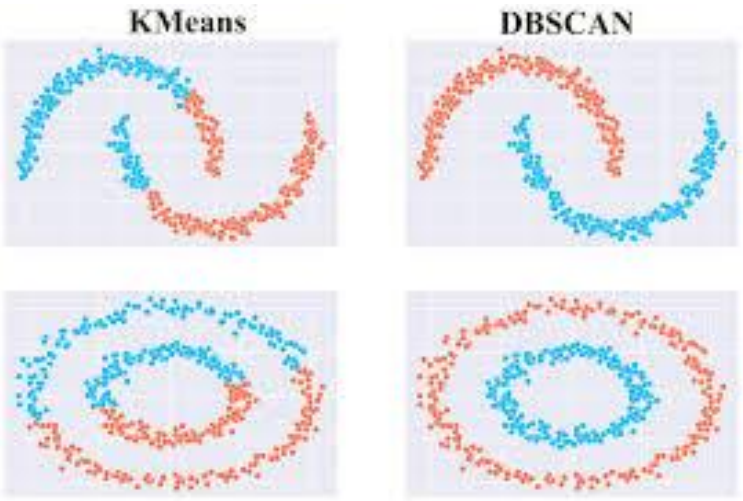
- DBSCAN (Density-Based Spatial Clustering of Applications with Noise): Defines clusters based on density thresholds (eps and minPts).
- OPTICS (Ordering Points To Identify Clustering Structure): Extends DBSCAN to handle varying densities.

Strengths

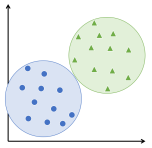
- Can detect arbitrarily shaped clusters (e.g., crescent-shaped or elongated).
- Robust to noise and outliers.
- Does not require specifying the number of clusters in advance.

Weaknesses

- Performance depends heavily on the choice of eps and minPts.
- Struggles with datasets where densities vary significantly across regions.



ML Models - Clustering - Approaches to Clustering



3. Hierarchical (e.g., Agglomerative Clustering) – Tree-like cluster structures

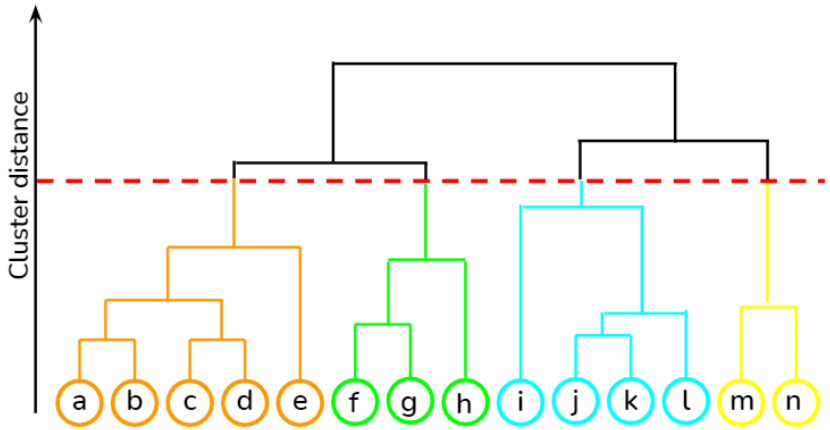
- Hierarchical clustering builds a hierarchy of clusters either by merging smaller clusters into larger ones (agglomerative) or by splitting larger clusters into smaller ones (divisive).

How It Works

- Start with each data point as its own cluster (agglomerative) or all data points in one cluster (divisive).
- Merge or split clusters iteratively based on a similarity measure (e.g., distance between centroids, single linkage, complete linkage).
- Represent the hierarchy as a dendrogram, where cutting the tree at a specific level gives the desired number of clusters.

Examples

- Agglomerative Hierarchical Clustering
- Divisive Hierarchical Clustering



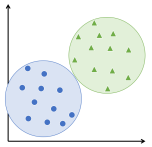
Strengths

- Does not require specifying the number of clusters upfront.
- Produces a dendrogram that provides insights into hierarchical relationships between data points.

Weaknesses

- Computationally expensive for large datasets (time complexity: O(n2)O(n2)).
- Sensitive to noise and outliers.
- Requires a linkage criterion (how the cluster are merged to find the final solution), which can influence results significantly.

ML Models - Clustering - Approaches to Clustering



4. Model-based (e.g., Gaussian Mixture Models, GMM) – Probabilistic approach

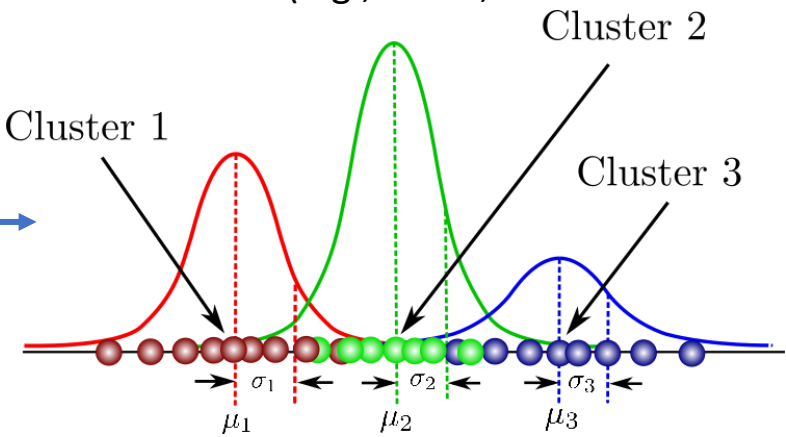
- Model-based clustering assumes that data is generated from an underlying probabilistic model and fits that model to identify clusters.

How It Works

- Assume that the data is generated from a mixture of probability distributions (e.g., Gaussian Mixture Models).
- Use algorithms like Expectation-Maximization (EM) to estimate the parameters of these distributions (e.g., mean, variance, and mixing coefficients).
- Assign data points to the distribution (cluster) with the highest likelihood.

Examples

- Gaussian Mixture Models (GMMs)
- Latent Dirichlet Allocation (LDA) for text clustering



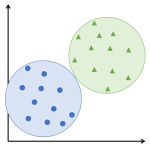
Strengths

- Can model complex cluster shapes using probabilistic distributions.
- Provides probabilistic assignments, allowing soft clustering (a point can belong to multiple clusters with probabilities).

Weaknesses

- Requires assumptions about the underlying distribution (e.g., Gaussian).
- Computationally expensive for large datasets.
- Sensitive to initialization and may converge to local optima.

ML Models - Clustering



Some Distance Metrics

Data points

- Euclidean Distance
- Manhattan distance
- Minkowski distance
- Chebyshev distance

Strings

- Cosine similarity
- Edit distance (NLP)
- Levenshtein distance
- Lee distance
- Hamming distance
- Jaro distance

Probability distributions

- Mahalanobis distance
- Bhattacharyya distance
- Hellinger distance

Sets

- Jaccard index (used in recommendation systems and social media analysis)
- Sørensen–Dice coefficient

Sequences (time series)

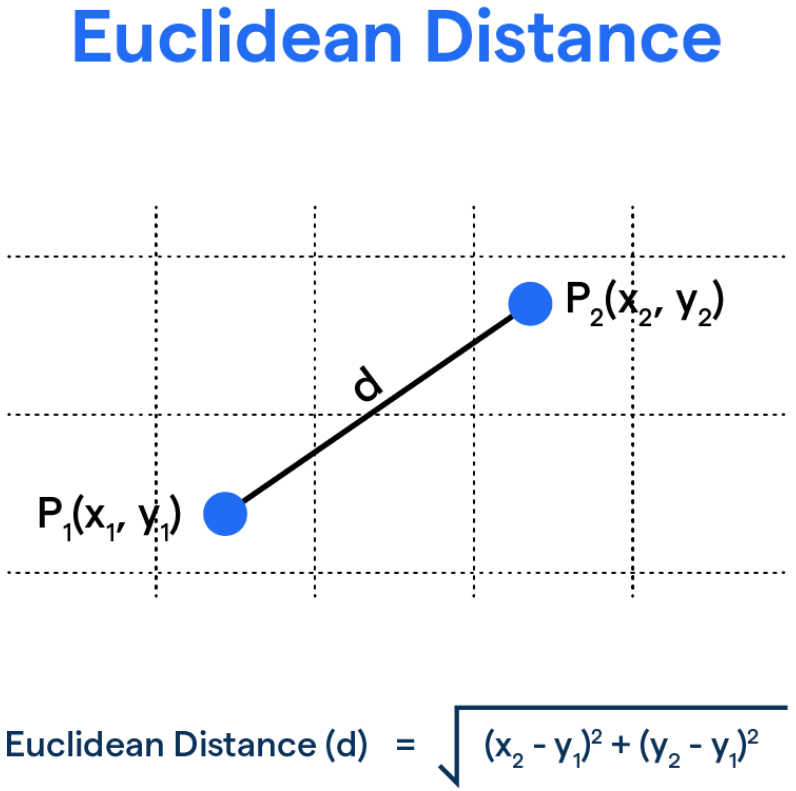
- Dynamic time warping

REF: https://en.wikipedia.org/wiki/Similarity_measure

ML Models - Clustering

Distance Metrics

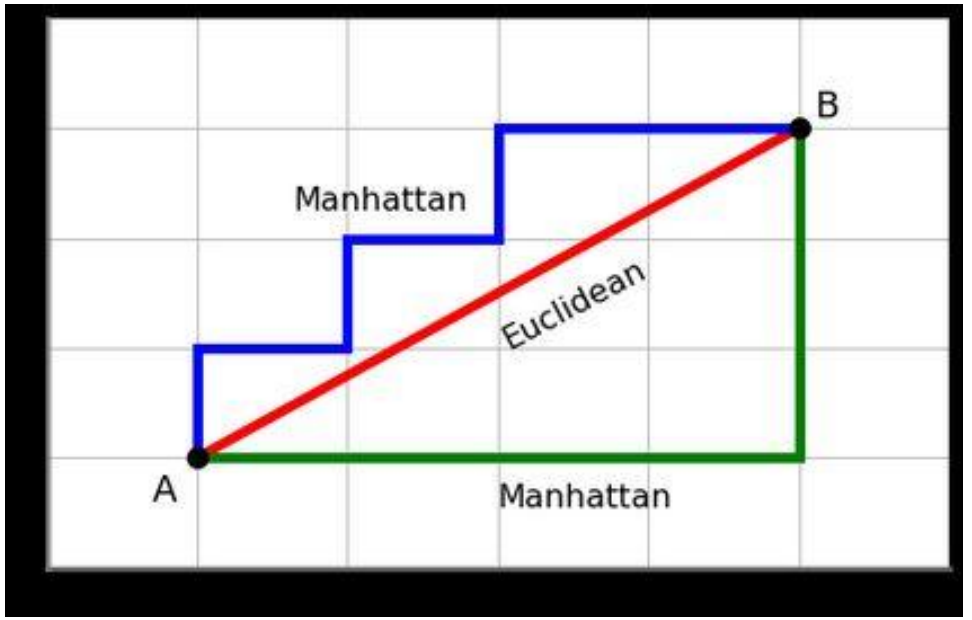
Euclidean Distance: calculates the "straight-line" distance between two data points in n-dimensional space.



ML Models - Clustering

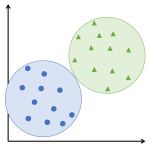
Distance Metrics

Manhattan Distance (a.k.a. taxicab geometry or city block distance): calculates the total distance travelled along each dimension (axis) to get from one point to another. Think of navigating a city grid where you can only move up/down/left/right – Manhattan distance sums the distances travelled in each direction.



$$Manhattan\ distance = |B_x - A_x| + |B_y - A_y|$$

ML Models - Clustering



Distance Metrics

Cosine similarity is a metric used to measure the similarity between two vectors based on the angle between them in a multi-dimensional space.

It is commonly used in text analysis (NLP), recommendation systems, and clustering algorithms.

Vectors are created using words in phrases and the angles between the vectors are quantified.

$$Cosine\ Similarity = \frac{A.B}{\|A\|.\|B\|}$$

A.B - Dot product of vectors A and B.
||v|| -Norma - Length of the vector v.

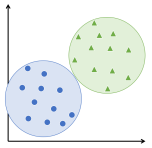
Values Range:

- 1 → Perfectly similar (same direction)
- 0 → Completely different (90-degree angle)
- -1 → Opposite direction (exactly opposite meaning, rare in practical cases)

$$A.B = \sum_{i=1}^{n} A_i . B_i$$

This is simply the **sum of the element-wise multiplications** of the vectors.

ML Models – Clustering Distance Metrics



Distance Metrics

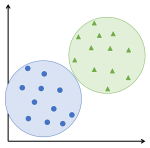
Choosing **the right metric depends on your data and task**.

Distance Metrics best for:

- **Euclidean distance:** Good for numerical data with continuous features.
- **Manhattan distance:** Useful for categorical data or data with uneven scales on different features.
- **Cosine similarity:** Ideal for text data where directionality is more important than magnitude.
  - Cosine Similarity, Clearly Explained! (https://www.youtube.com/watch?v=e9U0QAFbfLI&t=106s)
- **Jaccard similarity:** Well-suited for comparing sets or finding similar items based on shared features.

By understanding these metrics and their strengths, you can select the most appropriate one for your specific clustering application.

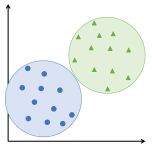
ML Models – Clustering Models



Most relevant clustering algorithms:

- **K-Means Clustering**
- **Hierarchical Clustering**
- **DBSCAN (Density-Based Spatial Clustering of Applications with Noise)**
- **Gaussian Mixture Models**
- **Spectral Clustering**
- **Fuzzy C-Means**
- **Among a few others …**

ML Models - Clustering



K-Means Clustering

K-means clustering is a widely used algorithm for partitioning data points into a specific number of clusters (k). It works by **iteratively assigning data points to the closest cluster center (centroid)** and then **updating the centroid based on the assigned points**. Here's a step-by-step breakdown:

1. Define the number of clusters (k): This is a crucial step that requires some knowledge of your data and the desired outcome. There are various methods to help choose the optimal k, like the elbow method or silhouette analysis.

2. Randomly initialize centroids: The algorithm starts by randomly selecting k data points as the initial centroids. These centroids represent the center of each cluster.



3. Assign data points to closest centroid: Each data point is assigned to the cluster with the nearest centroid based on a distance metric, commonly Euclidean distance.

4. Recompute centroids: Once all data points are assigned, the centroids are recalculated as the average of the data points belonging to their respective clusters.

5. Repeat steps 3 and 4: This **iterative process** continues until a **stopping criterion is met**. In each iteration, data points are reassigned to the closest centroid based on the updated centroid positions, and then the centroids are recalculated based on the new assignments.

Stopping criteria - Common stopping criteria:

No significant change in centroids: If the centroids barely move between iterations, it suggests the clusters have stabilized.
Maximum number of iterations: A predefined maximum number of iterations can be set to avoid the algorithm running indefinitely.

ML Models – Clustering - K-Means

Example



How many clusters? 1, 2, 3, 4, 5, 6, ...

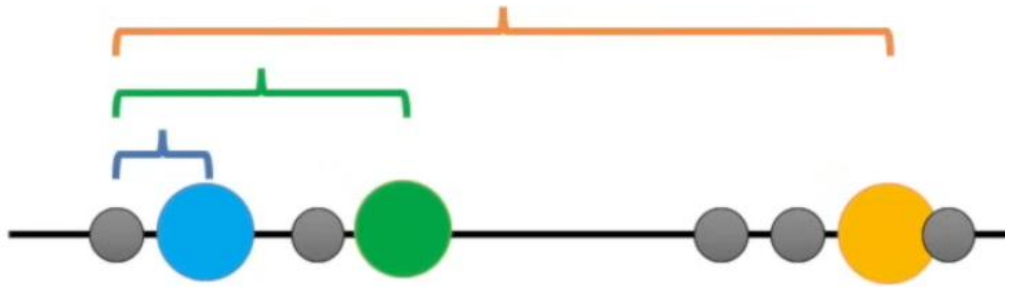
By eye we would assume something like:



Let's select K=3 and initialize the centroids (randomly)



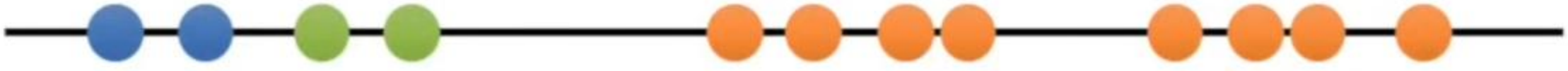
For each data point we measure its distance to the cluster centroid and the point shall be assigned to the cluster with lower distance.



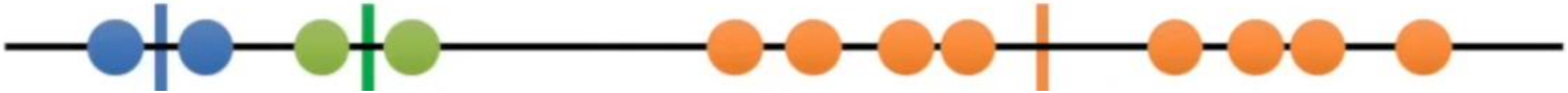
ML Models – Clustering - K-Means

Example

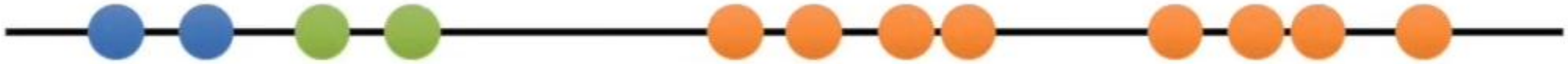
- After all point have been assigned to the clusters.



- We can recalculate the cluster's centroids:



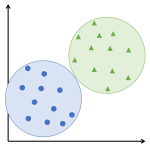
- And iterate this process until convergence is reached (no points change clusters within a iteration).
- Which results in this:



- Pretty lame when compared to our initial guess:



ML Models – Clustering - K-Means



Example

Our algortihms didn’t worked very well. This was due to poor initialization. This is a issue that can happen in K-Means.

Most used K-means initialization techniques:

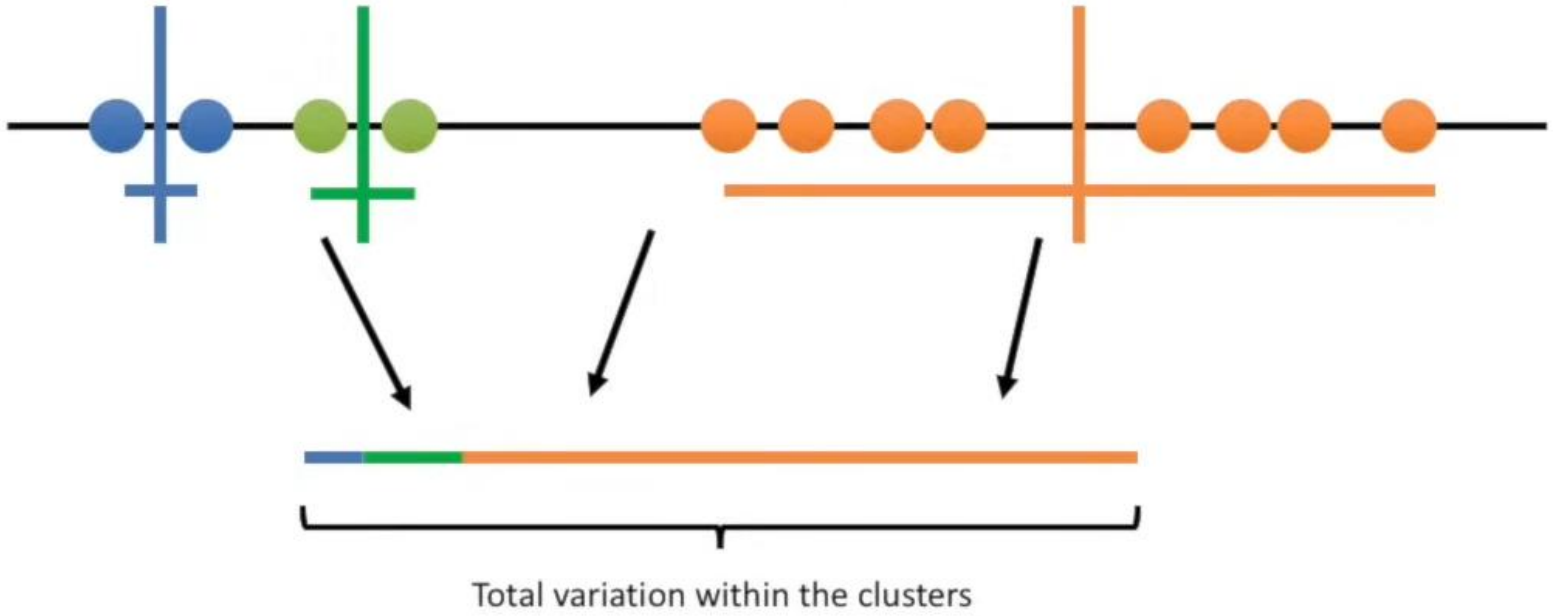
Initialization Method	Description	Advantages	Disadvantages
Random Selection	Simplest approach, randomly selects k data points as initial centroids.	Easy to implement, avoids getting stuck in local optima.	Can lead to suboptimal clustering results depending on the random selection.
K-Means++	More sophisticated method, iteratively selects centroids with probability based on distance to existing centroids.	Promotes better spread of initial centroids, often leading to faster convergence and potentially better clustering results.	Slightly more complex to implement compared to random selection.

ML Models – Clustering - K-Means

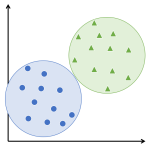
Example

How can we access the quality of the clusters?

For example by summing up the variation between datapoint within each cluster.



ML Models – Clustering - K-Means



Example

And we can choose “the best” cluster variation for this dataset.



Note that “the best” is a highly debatable metric and is chosen for each problem.

As human we instinctively know that the following clustering solution is very good:

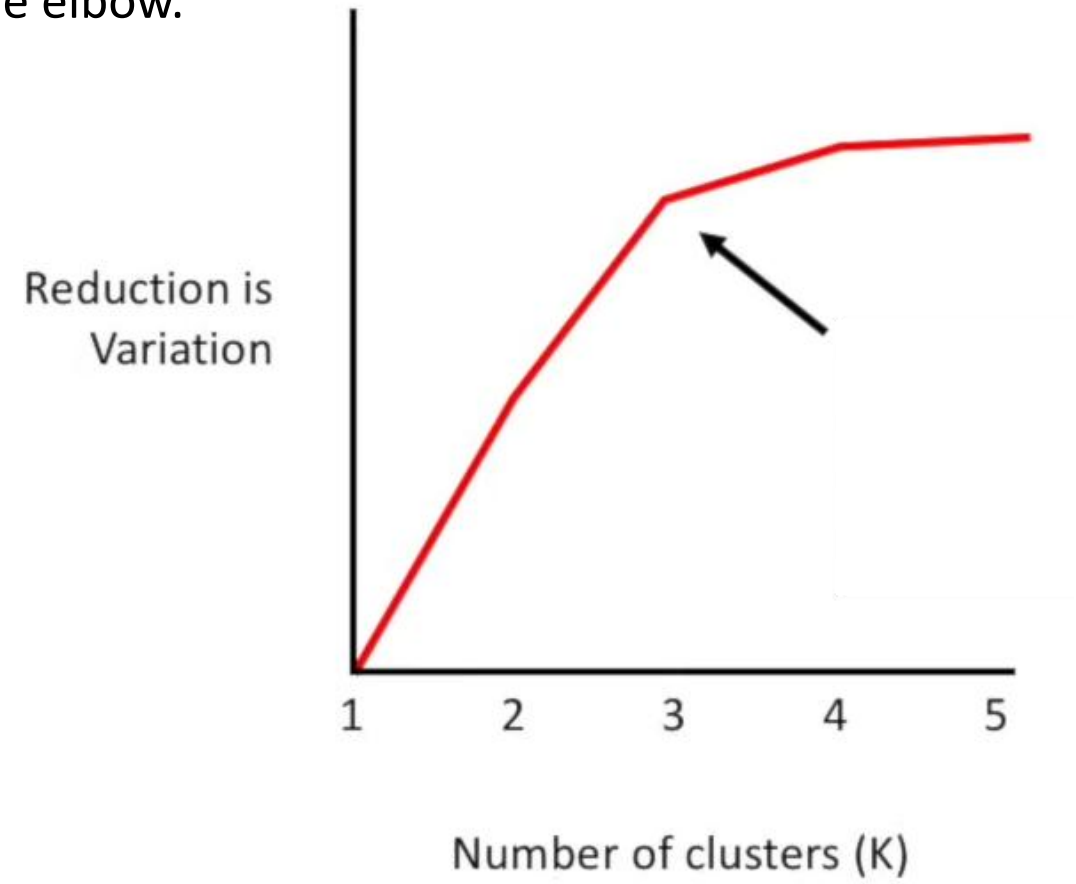


In some problems data might not be so easily visualizable. So there an automated way to choose K? YES. Using the elbow method.

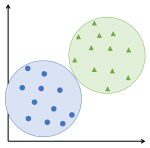
ML Models – Clustering - K-Means

The Elbow Method

- At the elbow there is a significant decrease in the reduction of the total variation of the cluster.
- We just need to create the plot and find the elbow.



ML Models - Clustering



Clustering Algorithms Ranked by Relevance

Rank	Algorithm	Key Feature
1	K-Means	Fast, easy, but assumes spherical clusters.
2	Hierarchical Clustering	No need to predefine K, good for small datasets.
3	DBSCAN	Detects noise and non-spherical clusters.
4	GMM	Handles overlapping clusters well.
5	Mean Shift	Finds dense regions automatically.
6	OPTICS	Improved DBSCAN for variable densities.
7	Spectral Clustering	Works well for graph-based clustering.
8	Affinity Propagation	No need for K, but high memory usage.
9	BIRCH	Good for big data clustering.
10	Fuzzy C-Means	Soft clustering with probabilistic memberships.

PGSCE-IARSEM

Regression (ML)

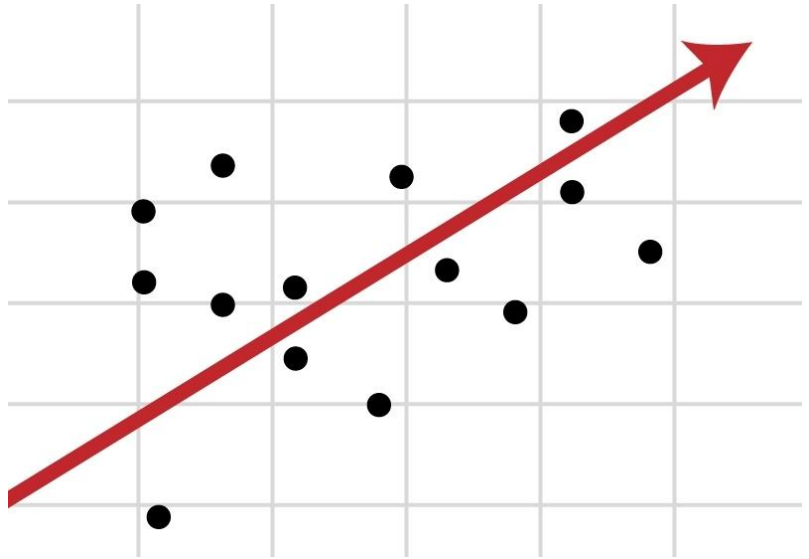
2025/2026

Veríssimo Santos

ML Models - Regression

Regression is a supervised machine learning technique used for modeling the relationship between a dependent variable (response variable) and one or more independent variables (also known as predictor variables or features) to make predictions.

Regression models are used to analyze and understand the relationship between variables and to make predictions about new data. The goal is to find a function that best represents the relationship between the input variables and the output variable, which can be used to predict the value of the output variable for new input data.



ML Models - Regression

Regression analysis uses

- Predict
- Causation
- Trend analysis
- Forecasting
- Optimization

ML Models - Regression

Common types of regression analysis

- **Simple linear regression;**
- **Multiple linear regression;**
- **Polynomial regression;**
- **Ridge regression;**
- **Lasso regression;**
- **Elastic net regression.**

PGSCE-IARSEM

Classification (ML)

2025/2026

Veríssimo Santos

ML Models - Classification

- **Classification** is a supervised machine learning technique used to automate tasks done by or with human intervention, that involve sorting data instances into classes or categories.
- The goal of classification is to learn a mapping function that can assign each input data instance into one of several predefined categories.
- In classification, the target variable is categorical (discrete).

ML Models - Classification

- Classification Examples:
- **Email and Text Processing**
  - **Spam Detection:** Classifying emails as spam or not spam is a classic binary classification problem.
  - **Sentiment Analysis:** Determining whether a text expresses positive, negative, or neutral sentiment.
- **Healthcare**
  - **Disease Diagnosis:** Classification models are used to predict whether a patient has a particular disease, such as COVID-19, based on symptoms and test results.
  - **Medical Image Analysis:** Models classify medical images (e.g., X-rays, MRIs) as showing healthy or pathological conditions.
- **Finance**
  - **Fraud Detection:**
  - **Credit Scoring:**

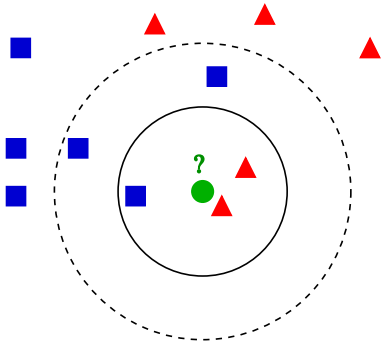
ML Models - Classification

A few of the main classification algorithms:

- k-Nearest Neighbors
- Support Vector Machine
- [X]Decision Trees
- [X]Logistic regression
- [X]Artificial Neural Networks
  - Shallow Neural Networks
  - Deep Neural Networks

[The Bias Variance Trade-Off]

ML Models – Classification - **k-Nearest Neighbors**



- Easy to understand and implement.
- Can be used in classification and regression tasks.
- KNN relies on the assumption that similar data points tend to have similar labels or values.
- To classify a new data point, finds the most similar data points from the training data (its neighbors) and predicts the new point's class based on the majority of its neighbors' classes (classification) or the average of its neighbors' values (regression).

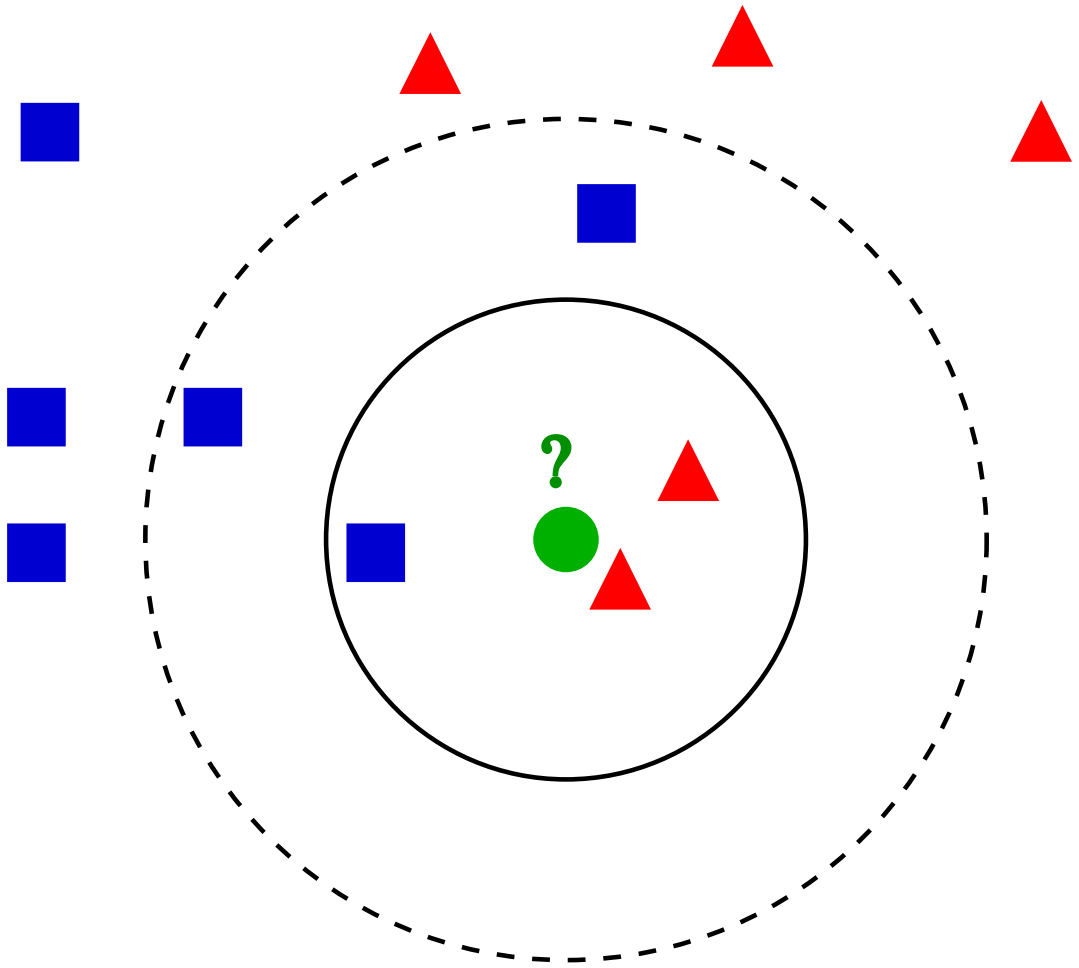
ML Models – Classification - **k-Nearest Neighbors**

KNN - Algorithm

1. **Choose K:** Define the number of neighbors (K) to be considered for the prediction. This is a crucial step as it can impact the model's performance.
2. **Calculate Distances:** The algorithm calculates the distance between the new data point and all the data points in the training set. Common distance metrics *Euclidean*, *Manhattan*, *Hamming*.
3. **Find Nearest Neighbors:** Based on the calculated distances, it identifies the K closest data points (neighbors) to the new data point.
4. **Make Predictions:**
   - Classification, KNN predicts the class label that appears most frequently among the K nearest neighbors.
   - Regression, it predicts the value by averaging the values of the K nearest neighbors.

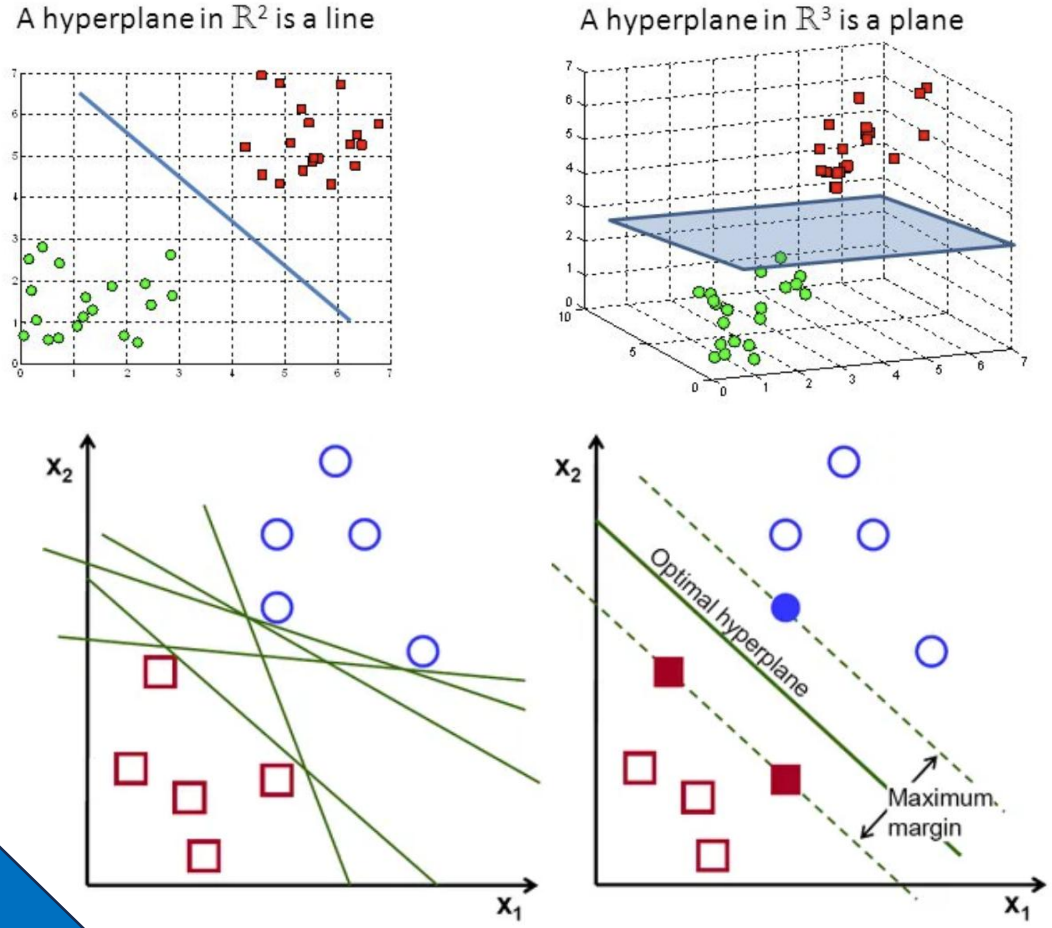
ML Models - Classification - k-Nearest Neighbors

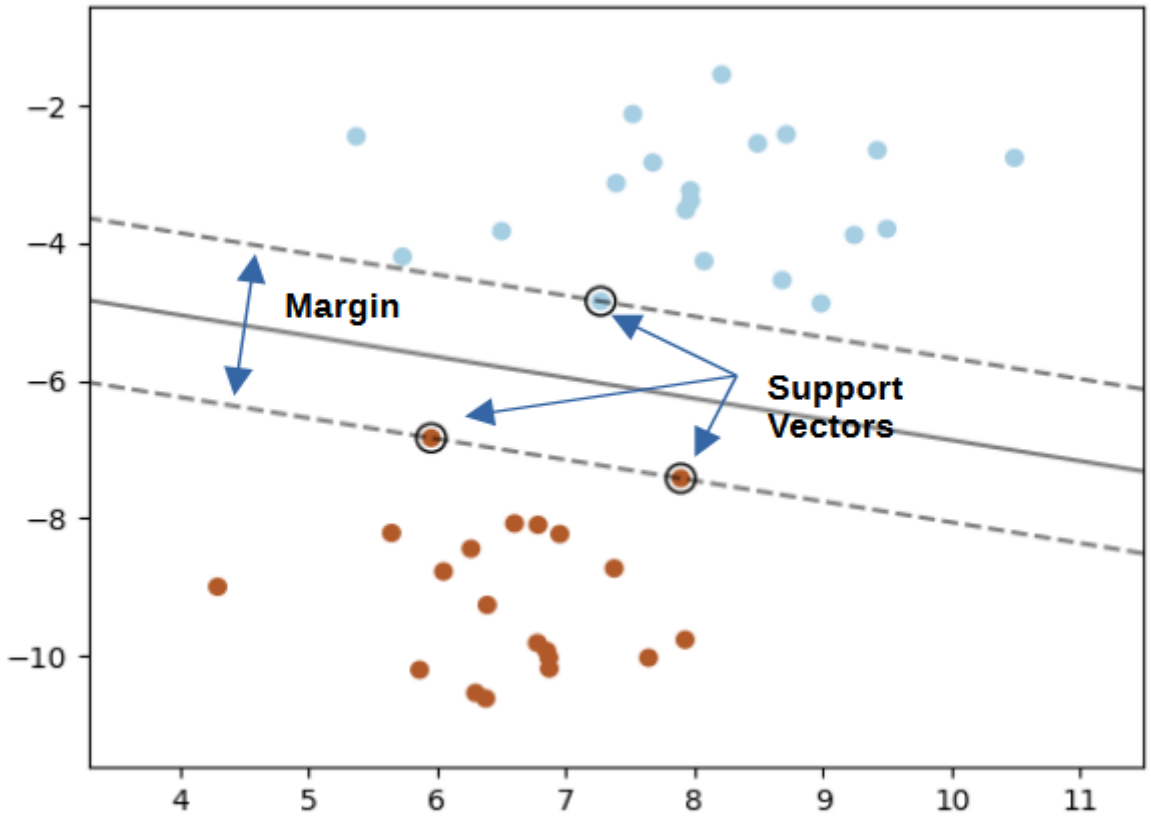
k-Nearest Neighbors - Example



ML Models – Classification - **Support Vector Machine**

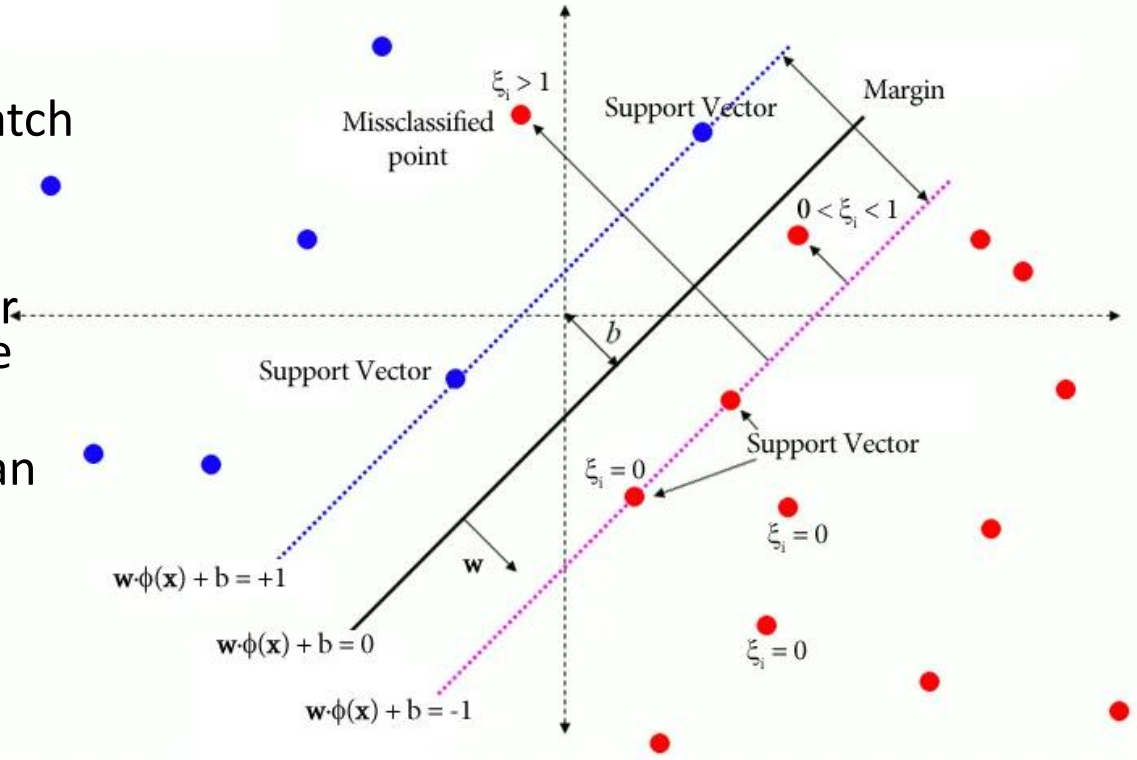
- SVMs work by finding the optimal hyperplane in a high-dimensional space that best separates the different class's data points.





ML Models - Classification - **Support Vector Machine**

- Can be used in classification and regression tasks.
- Not suited for very large datasets since the training time increases significantly as the number of data points grows.
- But what if the classes are not totaly separable? Does the hard margin work for these problems? NO.
- We need to softening the constraints to accommodate real-world complexities.
- Meet **The Soft Margin**.
- A **slack variable ($\xi \geq 0$)** is used to allow some mismatch of data points during the optimization process.
- For a correctly classified points, zeta will be zero.
- For a wrong classified point, zeta value will be greater than zero, and that value presents the points that are misclassified.
- With this information calculated for each point we can determine the best solution that minimizes the classification for the training dataset.



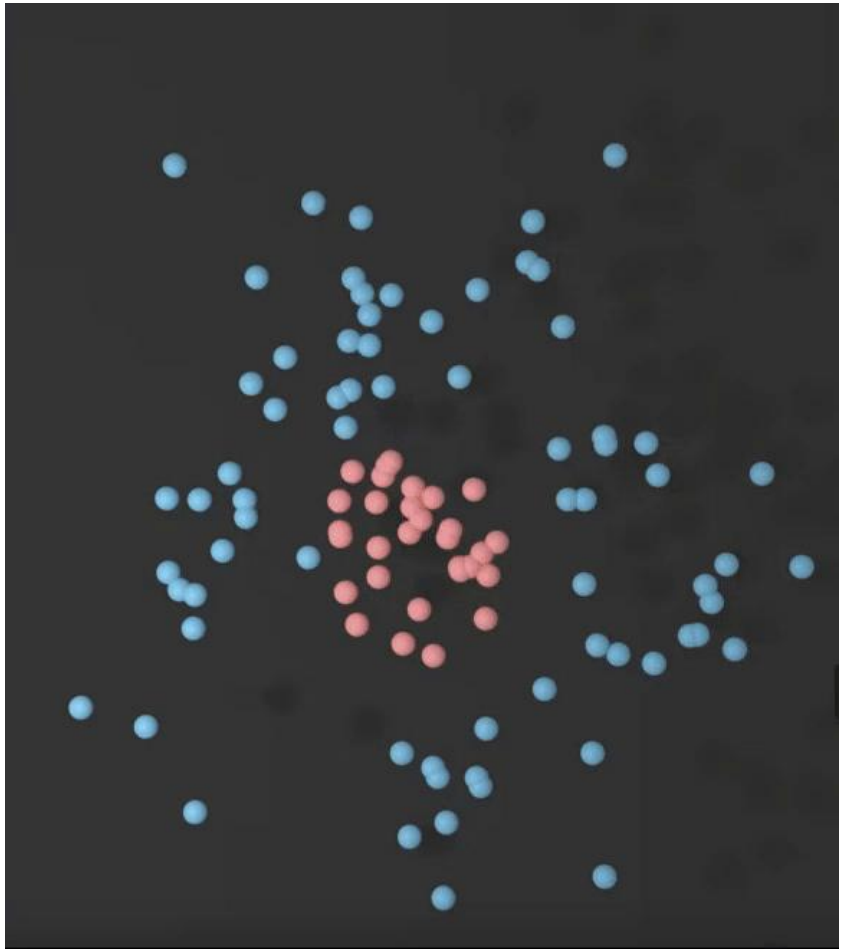
ML Models - Classification - Support Vector Machine

SVM – The Kernel Trick

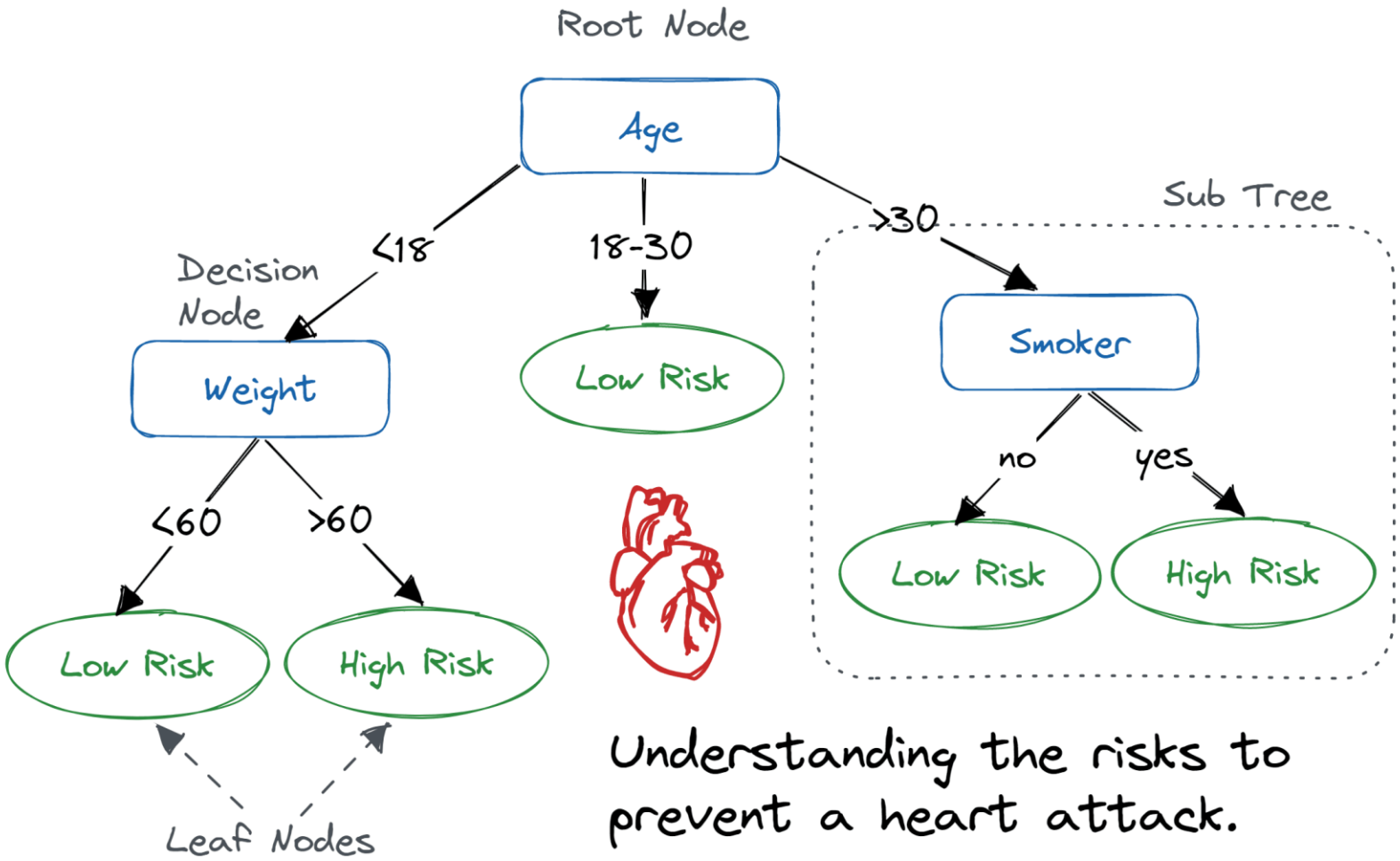
- **SVMs use hyperplanes (linear).**
- Real world datasets frequently have classes that are non-linearly separable.
- To overcome this a clever solution is used:

The kernel Trick

- Special kernel functions (Polynomial, Radial Basis Function) are used to transform the data into a higher-dimensional space in which it becomes linearly separable by a hyperplane

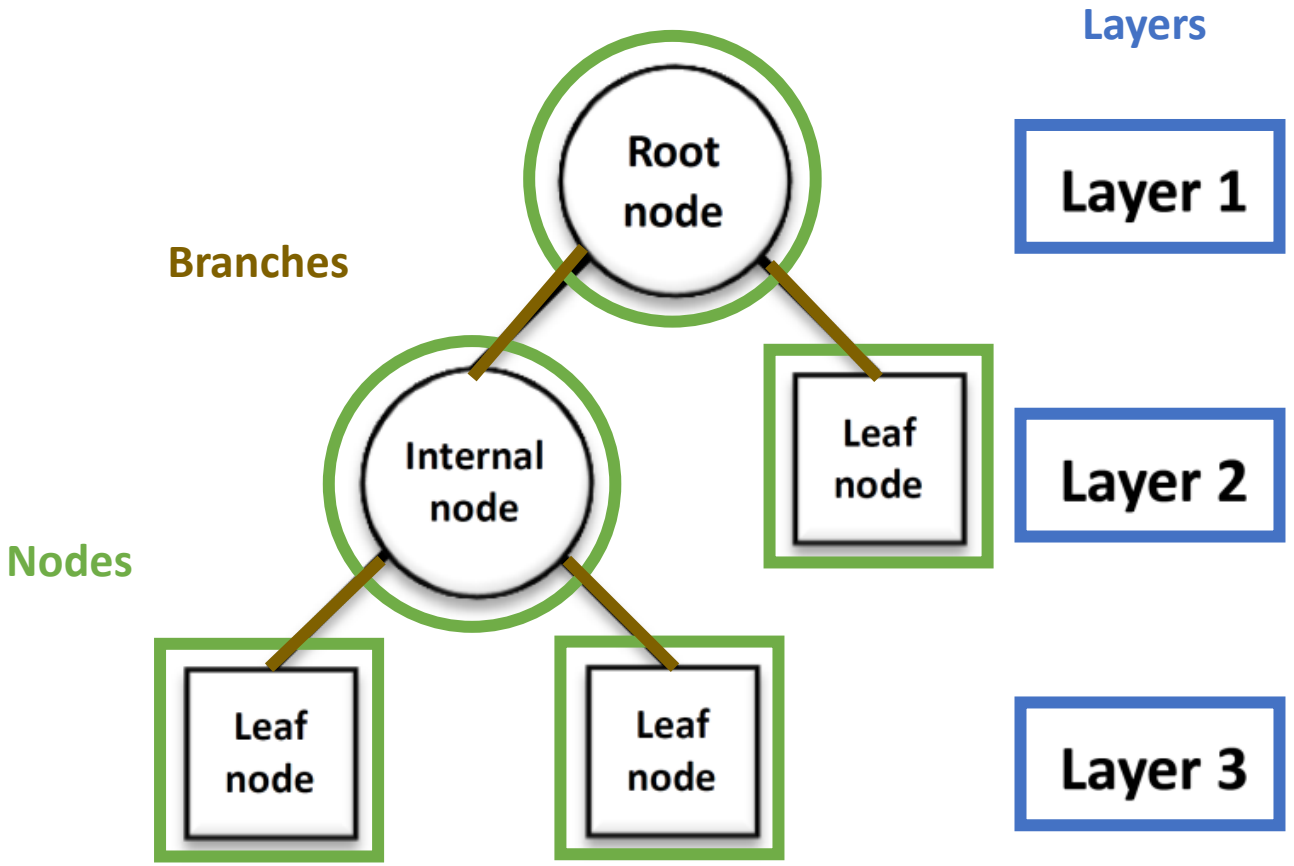


ML Models – **Decision Trees (DT)**



ML Models – Classification - Decision Trees (DT)

DT - Elements

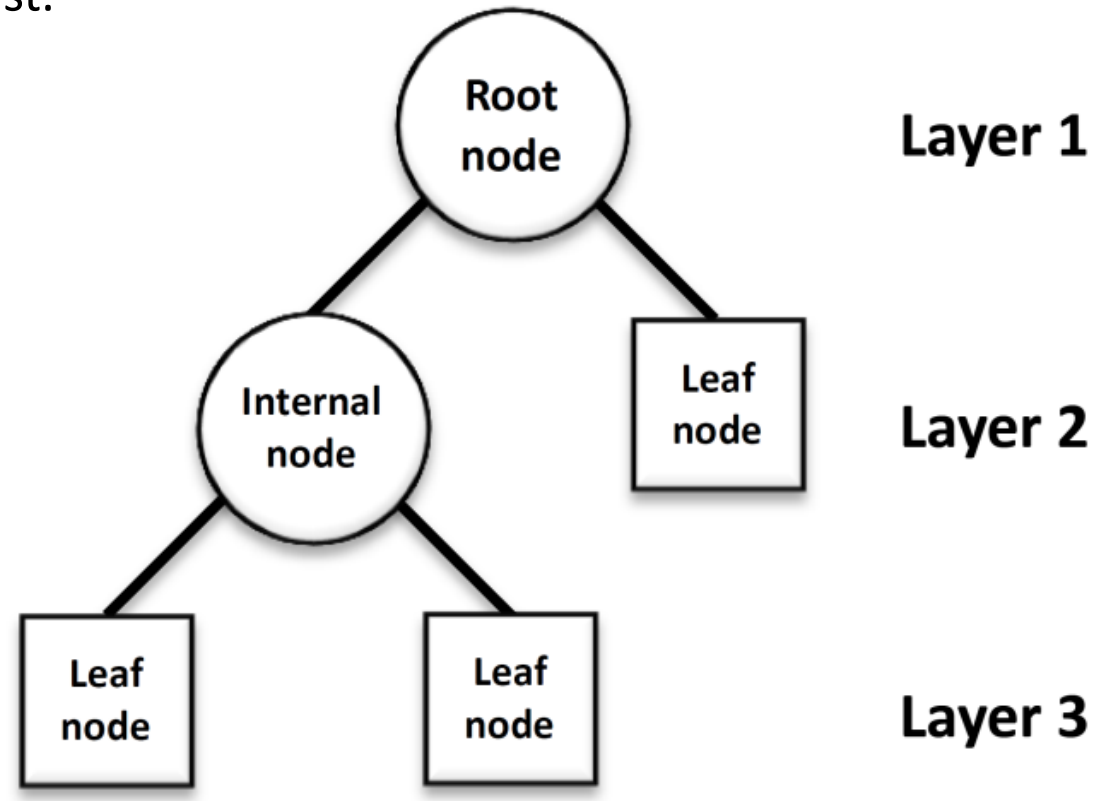


ML Models – Classification - **Decision Trees (DT)**

What is a DT

Is a flowchart flowchart-like structure where:

1. Each **internal node** represents a test or decision on an attribute (feature) of the dataset.
2. Each **branch** represents the outcome of that test.
3. Each **leaf node** represents a final decision or prediction (class label for classification, numeric value for regression).

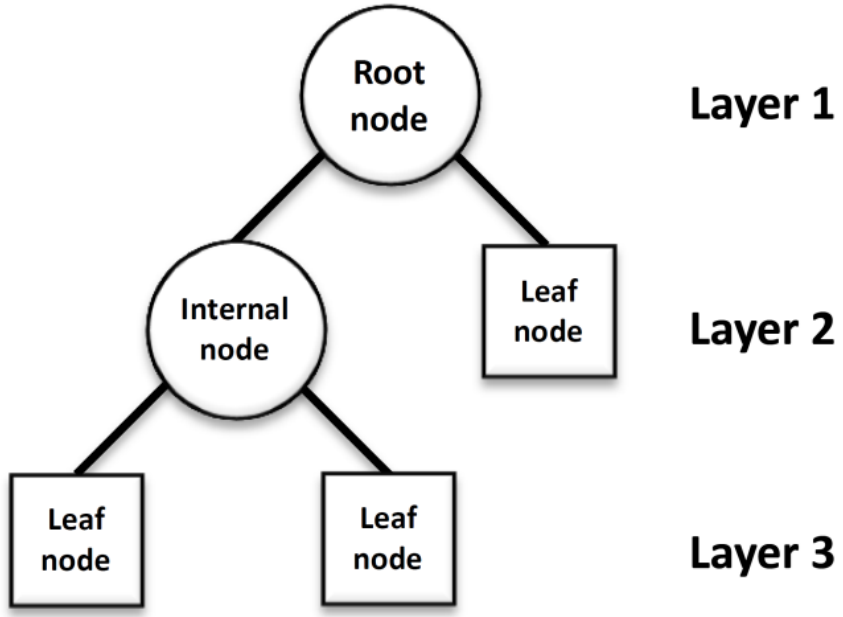


ML Models – Classification - **Decision Trees (DT)**

DT - How it works (Trained)

The prediction process starts at the root node and proceeds down the tree, making a series of comparisons based on the input features;

1. At each node, **a specific feature is tested**, and the sample is directed left or right (binary) depending on the result;
2. This process continues until a **leaf node** is reached, which provides the prediction.



ML Models – Classification - **Decision Trees (DT)**

Decision trees – Types of decisions

- **Binary Splits:** Many decision tree algorithms (like CART) create binary trees where each decision node splits into exactly two branches, often for computational simplicity and to improve certain properties of the tree.
- **Multiway Splits:** Other algorithms, such as **ID3 and CHAID**, have been designed to support nodes with more than two branches, i.e. for **multiway decisions at a single node**. For example, a categorical variable with three possible values can be split into three branches in a single step using these methods. But under certain conditions can support binary splits.
- The binary splits are very common.
- Multi-branch splits can be a good option for specific problems.

ML Models – Classification - Decision Trees (DT)

DT - How is trained



Introdução: [Decision and Classification Trees, Clearly Explained!]



Implementação: [Decision Tree Classification in Python (from scratch!)]



Usage in scikit learn: [Decision Tree Classification in Python Tutorial]

ML Models – Classification - **Decision Trees (DT)**

Advantages of Decision Trees

1. **Interpretability and Transparency:** Decision trees provide a clear, visual representation of decision-making processes, making them easy to explain to non-technical stakeholders
2. **Handle Mixed Data Types:** They work with both categorical and numerical data without requiring extensive preprocessing (e.g., one-hot encoding or scaling)
3. **Non-Parametric Flexibility:** No assumptions about data distributions (e.g., normality), allowing them to model complex, non-linear relationships
4. **Robustness to Data Issues:**
    1. Tolerates missing values by treating them as a separate category.
    2. Resistant to outliers due to hierarchical splitting based on majority patterns
5. **Automatic Feature Selection:** Prioritizes impactful features during training, reducing noise from irrelevant variables
6. **Supports Classification and Regression:** Adaptable to both tasks (e.g., predicting categories or continuous values)
7. **No Need for Feature Scaling:** Unlike neural networks or SVMs, decision trees do not require normalized input data

Disadvantages of Decision Trees

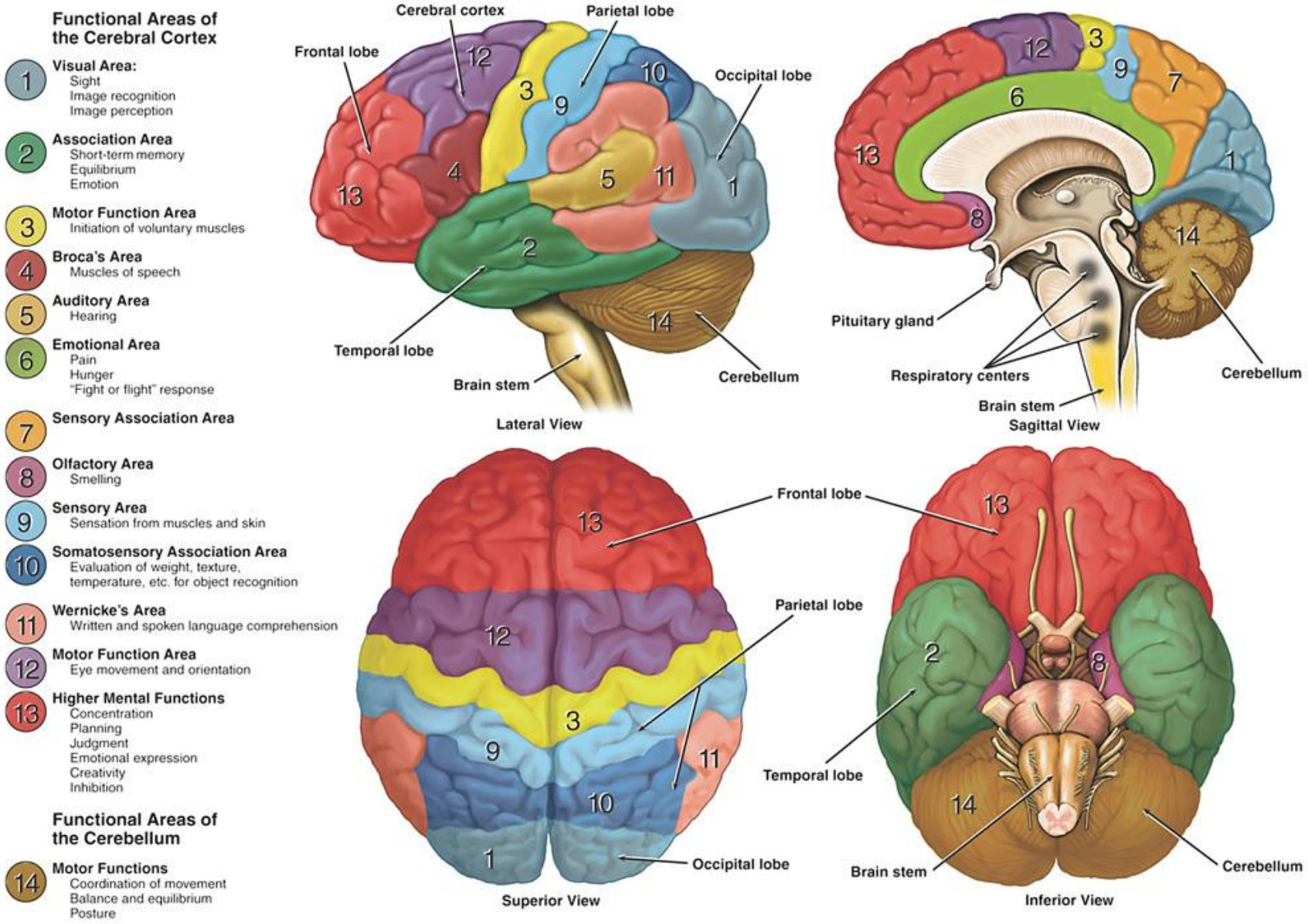
1. **Prone to Overfitting:** Deep trees may memorize noise in training data, leading to poor generalization. Mitigated via pruning or limiting tree depth
2. **Instability:** Small changes in data (e.g., adding/removing samples) can drastically alter the tree structure
3. **Bias Toward Dominant Classes:** Performs poorly on imbalanced datasets unless techniques like class weighting or resampling are applied
4. **Computational Cost:** Training becomes expensive with large datasets or high-dimensional features, often requiring feature reduction
5. **Greedy Splitting:** Uses locally optimal splits at each node, which may miss globally optimal solutions
6. **Non-Smooth Predictions:** Creates step-like decision boundaries, which can be problematic for regression tasks requiring smooth outputs
7. **Scalability Limitations:** Performance degrades with extremely large datasets due to exponential growth in possible splits.



ML Models – Classification - **Decision Trees (DT)**

- **Vamos fazer isto: Build a Decision Tree in Minutes using Weka (No Coding Required!)**
- **Fazer download do *Breast Cancer Dataset* e abri-lo no Weka.**
- **Conversor de format csv para arff → (Aprender sobre ficheiros arff)**
- **Abrir o ficheiro do dataset num editor de texto e ler a informação disponível sobre o mesmo.**
- **Discutir o dataset?**
  - **Qual a variável de saída?**
  - **Existem correlações?**
- **Procurar mais informação sobre o dataset na internet.**
- **No Weka selecionar a Árvore de Decisão J48 e treiná-la.**
- **Observar a saída no Weka e depois visualizar a árvore.**
- **Vamos podar a árvore e repetir o ponto anterior.**

ML Models – Classification - **Artificial Neural Networks**

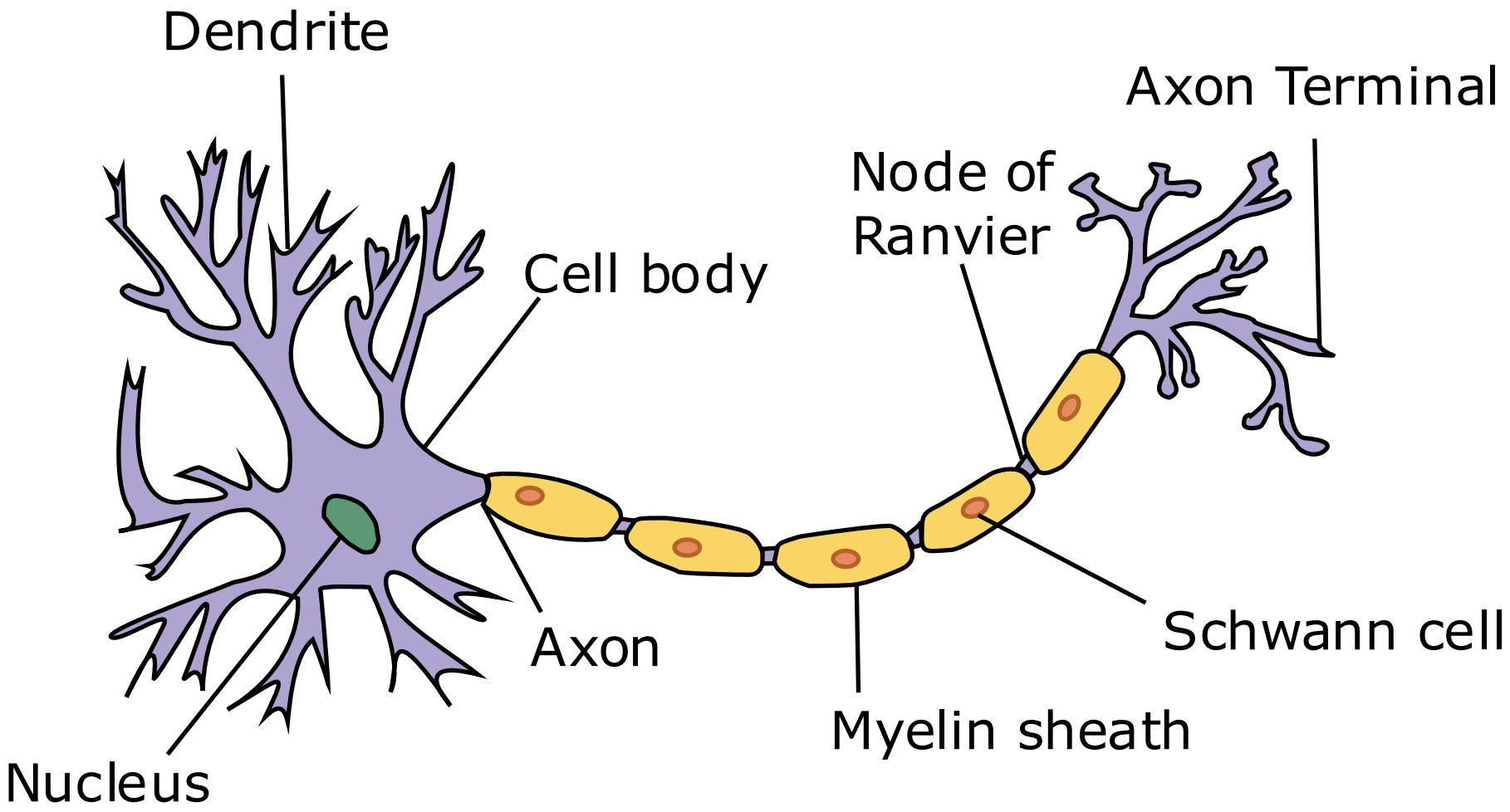


ML Models – Classification - **Artificial Neural Networks**

- O cérebro integra e processa informações do sistema sensorial, coordena atividades voluntárias e involuntárias, e é responsável pelas funções cognitivas superiores como memória, linguagem, inteligência, comportamento e raciocínio.
- O funcionamento cerebral depende da atividade de bilhões de neurônios, que comunicam entre si por meio de sinapses, formando circuitos neurais complexos.
- Os neurônios transmitem sinais elétricos e químicos, liberando neurotransmissores em resposta a estímulos.
- O cérebro também apresenta plasticidade, ou seja, capacidade de reorganizar suas conexões em resposta a experiências, aprendizagem e lesões. Essa plasticidade é fundamental para o desenvolvimento, adaptação e recuperação de funções ao longo da vida.

ML Models – Classification - **Artificial Neural Networks**

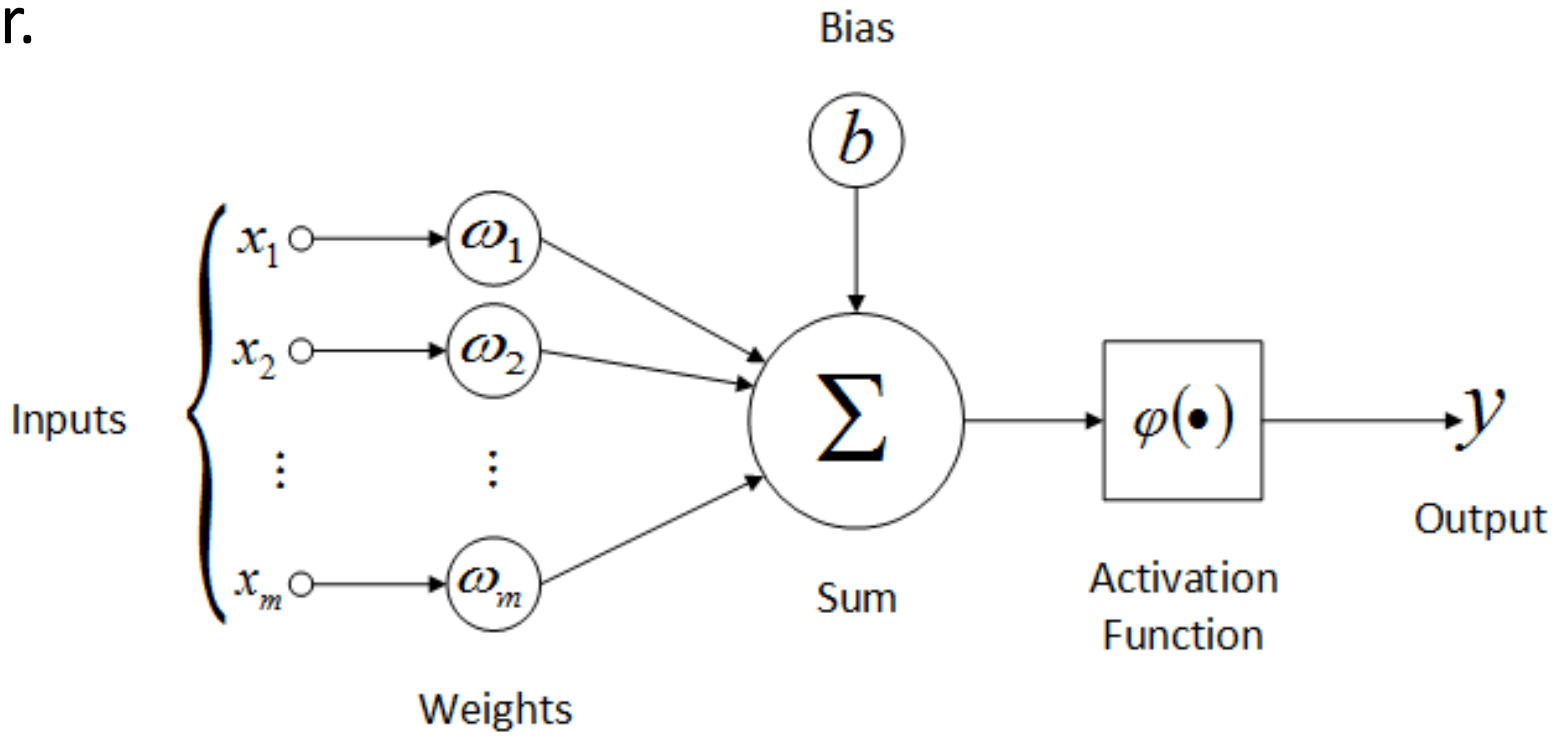
- The biological neuron



ML Models – Classification - Artificial Neural Networks

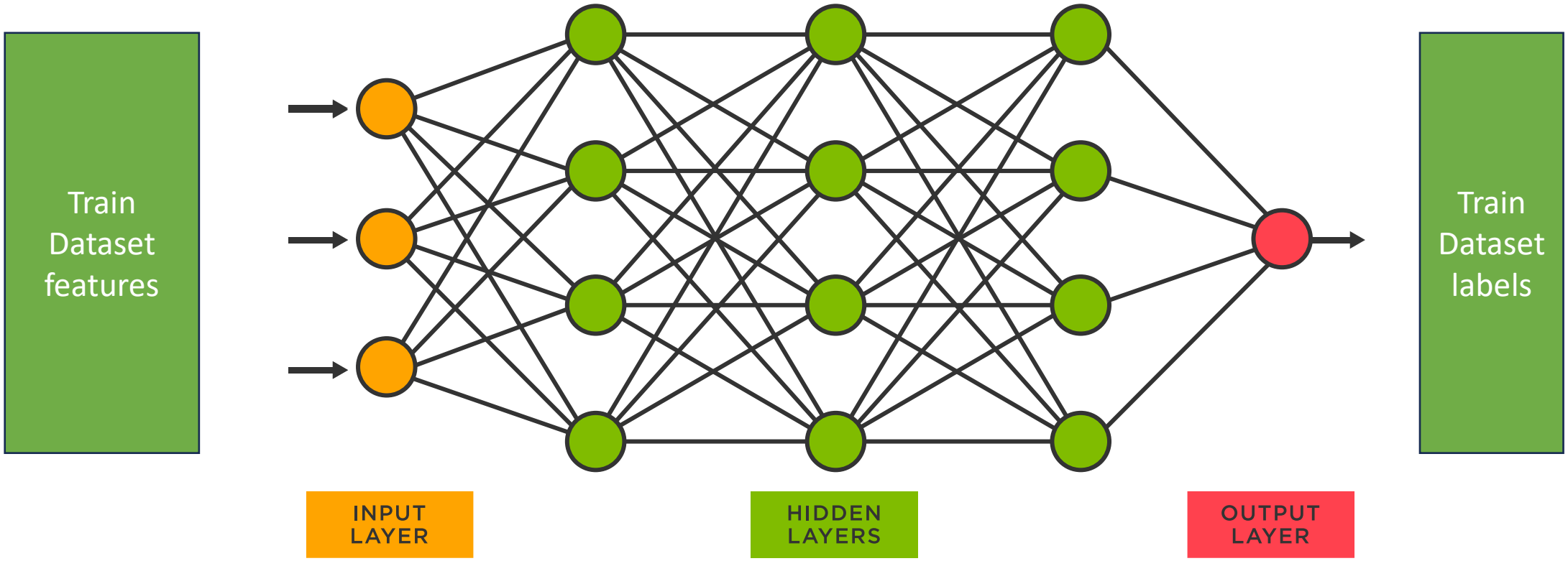
The artificial neuron

- The artificial neuron is a computational unit that is inspired by the structure and function of biological neurons
- Introduced in 1943 by Warren Sturgis **McCulloch and** Walter Harry **Pitts**, Jr.



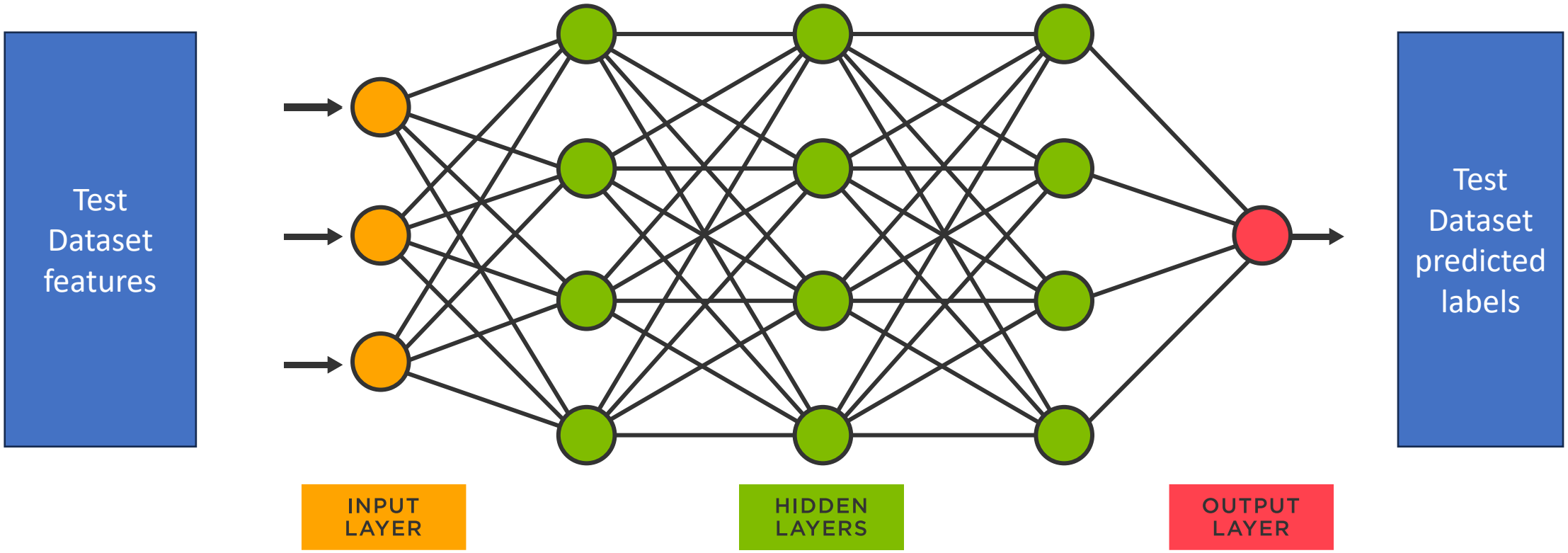
ML Models – Classification - **Artificial Neural Networks**

- ANN Train



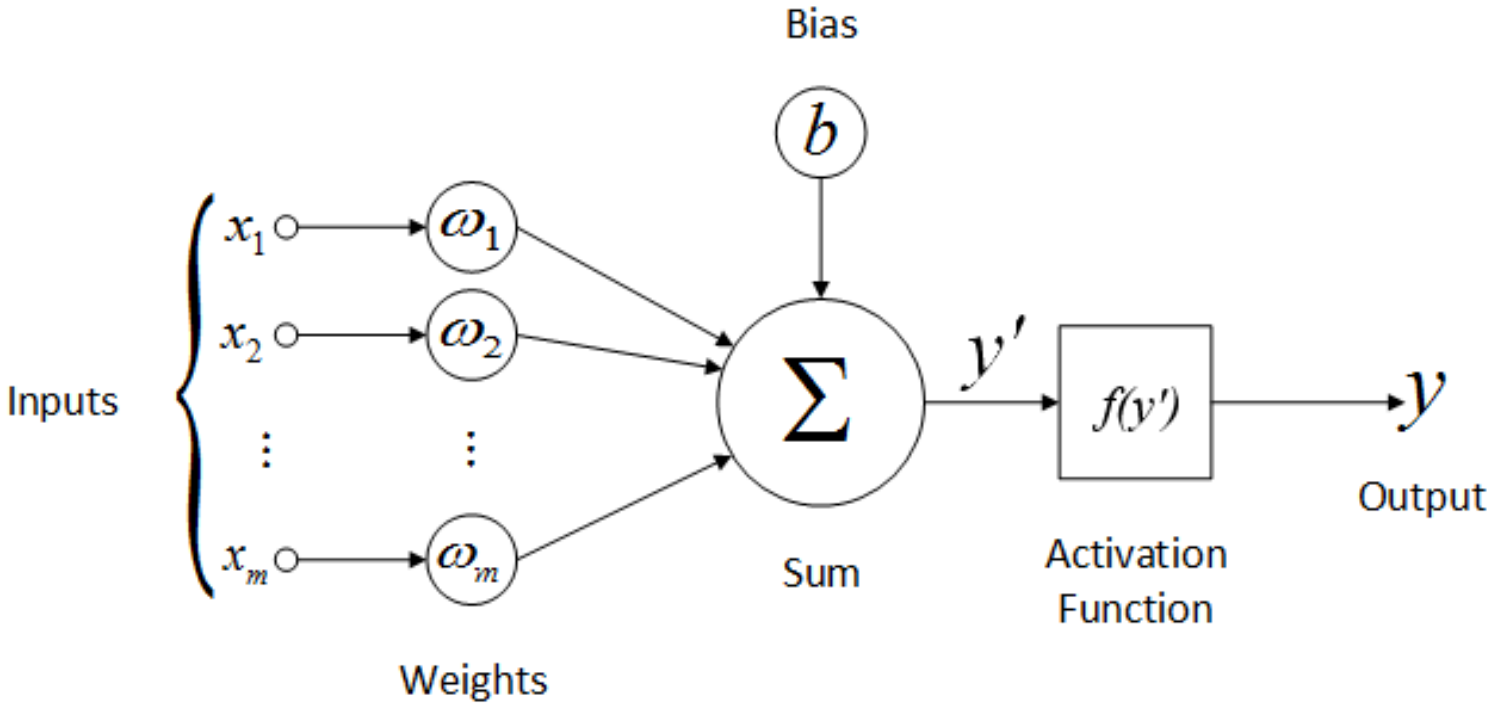
ML Models – Classification - **Artificial Neural Networks**

- ANN Test

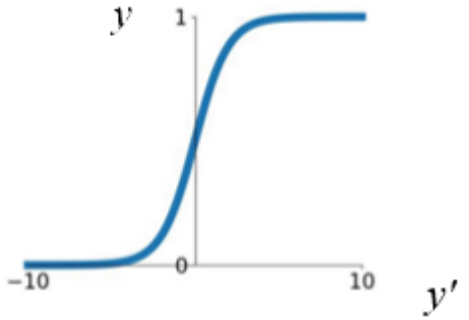


ML Models – Classification - Artificial Neural Networks

- **The artificial neuron – Working Principle**



$$Y' = \sum_{i=1}^{m} (x_i w_i) + b$$



ML Models – Classification - Artificial Neural Networks

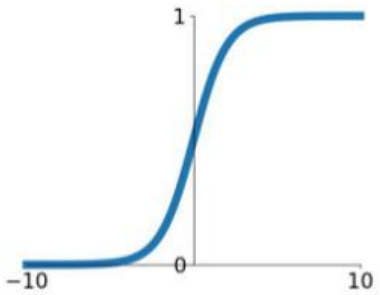
Activation Functions

- The output of the weighted sum plus bias is passed through an activation function, which introduces non-linearity into the network and allows it to model complex relationships between the inputs and outputs.
- The activation function simulates the biological neuron activation by increasing/decreasing the output by a significative amount for certain areas of the combined signal while it saturates the output (doesn't change) for other areas.
- Activation functions also help to normalize the output of each neuron, making it easier for the subsequent layers to process the information.
- They can also help prevent the vanishing gradient problem, where the gradients in the lower layers of the neural network become too small during backpropagation, making it difficult to update the weights.

ML Models – Classification - Artificial Neural Networks

Activation Functions – Sigmoid

The Rectified Linear Unit (ReLU) function is a piecewise linear function that outputs the input value if it is positive, and zero otherwise. It is a widely used activation function in deep neural networks due to its simplicity and ability to alleviate the vanishing gradient problem. However, it can suffer from the problem of "dead" neurons if the input is negative.

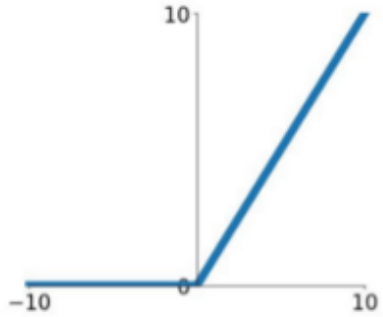


$$f(x) = \frac{1}{1 + e^{-x}}$$

ML Models – Classification - Artificial Neural Networks

Activation Functions – ReLU

The Rectified Linear Unit (ReLU) function is a piecewise linear function that outputs the input value if it is positive, and zero otherwise. It is a widely used activation function in deep neural networks due to its simplicity and ability to alleviate the vanishing gradient problem. However, it can suffer from the problem of "dead" neurons if the input is negative.

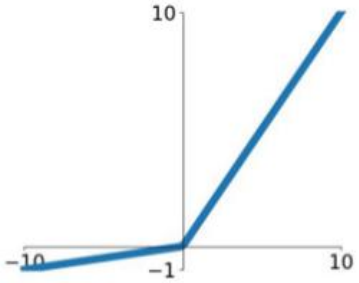


$$f(x) = \begin{cases} x, x > 0 \\ 0, x \leq 0 \end{cases}$$

ML Models – Classification - Artificial Neural Networks

Activation Functions - Leaky ReLU

The Leaky ReLU function is a modified version of the ReLU function that introduces a small slope for negative input values, which helps prevent "dead" neurons. It is a popular choice for deep neural networks.

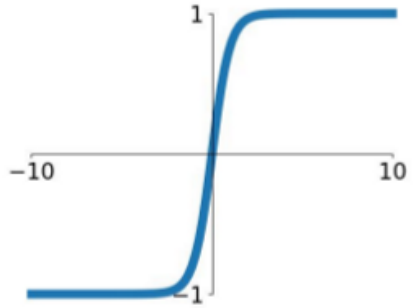


$$f(x) = \begin{cases} a * x, x > 0 \\ 0, x \leq 0 \end{cases} \quad a \rightarrow small\ value\ (e.g.\ 0.1)$$

ML Models – Classification - Artificial Neural Networks

Activation Functions - hyperbolic tangent (tanh)

The hyperbolic tangent (tanh) function is a sigmoidal function that maps any input value to a value between -1 and 1. It is widely used in deep neural networks due to its ability to introduce non-linearity and produce output values that are centered around zero.



$$f(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

ML Models – Classification - Artificial Neural Networks

ANN - Training Procedure

As previously stated, the training procedure aims to adjust the ANN parameters (weights and bias) in order to map the training dataset features to the correspondent training dataset labels.

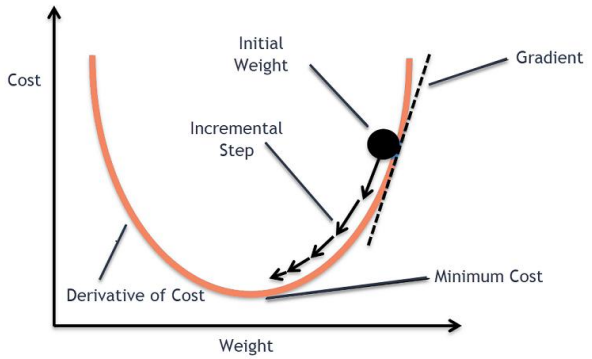
Steps involved:

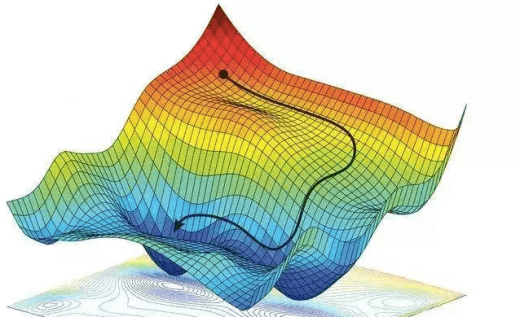
- **Dataset organization**
  - collecting, preprocessing and partitioning
- **Weights Initialization**
  - is the starting point for the training process
  - the network's weights and biases are initialized randomly
- **Forward Propagation**
  - Each input data instance is fed into the network, and the network computes its output.
  - Each neuron in the network computes its output based on its inputs and the current weights and biases values associated with it.
  - The output of one neuron becomes the input to the next neuron, and this process continues until the output of the final neuron is computed.
  - **The output of the network is then compared to the expected output, and the error is calculated.**

ML Models – Classification - Artificial Neural Networks

ANN - Training Procedure

- **Backward Propagation**
  - **The error computed in the previous step is used to adjust the weights and biases of the network.**
  - **The goal is** to minimize the error by iteratively adjusting the weights and biases in the direction that reduces the error.
  - **Backpropagation involves** computing the gradient of the error with respect to each weight and bias in the network.
- **Gradient Descent**
- **Gradient descent** is a commonly **optimization algorithm used in ANN training**.
- It **works by calculating the gradient of the error with respect to each weight and bias and then adjusting them** in the opposite direction of the gradient **to minimize the error**.
- The **learning rate** is a hyperparameter that determines the size of the step taken in the opposite direction of the gradient. A high learning rate can lead to overshooting the minimum, and a low learning rate can lead to slow convergence.

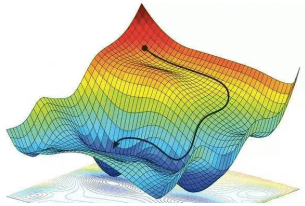




ML Models – Classification - Artificial Neural Networks

ANN - Training Procedure

- **Update Weights and Biases**
  - Once the gradient descent step is complete, the weights and biases of the network are updated using the computed gradients. The weights and biases are updated based on the learning rate and the direction of the gradient. The new weights and biases are used in the next iteration of the training process.
- **Validation**
  - After training the network, it is important to evaluate its performance on a validation set of data. This ensures that the network is not overfitting to the training data and is generalizing well to new data. The validation set is used to tune the hyperparameters of the network, such as the learning rate and the number of hidden layers.
- **Testing**
  - Finally, the network is tested on a separate test set of data to evaluate its performance. This provides a measure of how well the network will perform on new data in the future.



ML Models – Classification - Artificial Neural Networks

Hyperparameters

- **Activation Function:** Introduces non-linearity, enabling the network to learn complex relationships. Common choices include ReLU, sigmoid, and tanh
- **Optimizer Type:** The algorithm used to update network weights (e.g., SGD, Adam, RMSProp). Different optimizers can affect convergence speed and stability
- **Learning Rate Decay/Schedule:** Adjusts the learning rate during training, often decreasing it to allow for finer convergence as training progresses
  - Learning rate: determines the step size at which the model adjusts its weights during training.
- **Regularization Techniques:** Methods such as dropout, L1/L2 regularization, and early stopping to prevent overfitting and improve generalization
- **Number of hidden layers:** determines the number of layers in a neural network.

ML Models – Classification - Artificial Neural Networks

Hyperparameters

- **Number of Neurons per Layer (Width):** Controls the capacity of each layer to represent features. Too few neurons can underfit; too many can overfit
- **Activation Function:** Introduces non-linearity, enabling the network to learn complex relationships. Common choices include ReLU, sigmoid, and tanh
- **Optimizer Type:** The algorithm used to update network weights (e.g., SGD, Adam, RMSProp). Different optimizers can affect convergence speed and stability
- **Regularization Techniques:** Methods such as dropout, L1/L2 regularization, and early stopping to prevent overfitting and improve generalization

ML Models – Classification - Artificial Neural Networks

Hyperparameters

- The choice of hyperparameters can significantly affect the performance of a machine-learning model. Selecting the best hyperparameters for a given task is usually done through a process of hyperparameter tuning, where various values of hyperparameters are tested on a validation set, and the best-performing combination is chosen based on some performance metric.
- Hyperparameter tuning can be done manually, where the modeler tries out different hyperparameter combinations and evaluates their performance, or automatically using tools such as grid search, random search, or Bayesian optimization

PGSCE-IARSEM

Ensembles (ML)

2025/2026

Veríssimo Santos

ML Models - Ensembles

The goal of assembling a musical ensemble is to produce “outstanding music”!

An ensemble is composed of a set of “music performers” whose individual contributions if well combined will produce a much better result than the individual contributions.



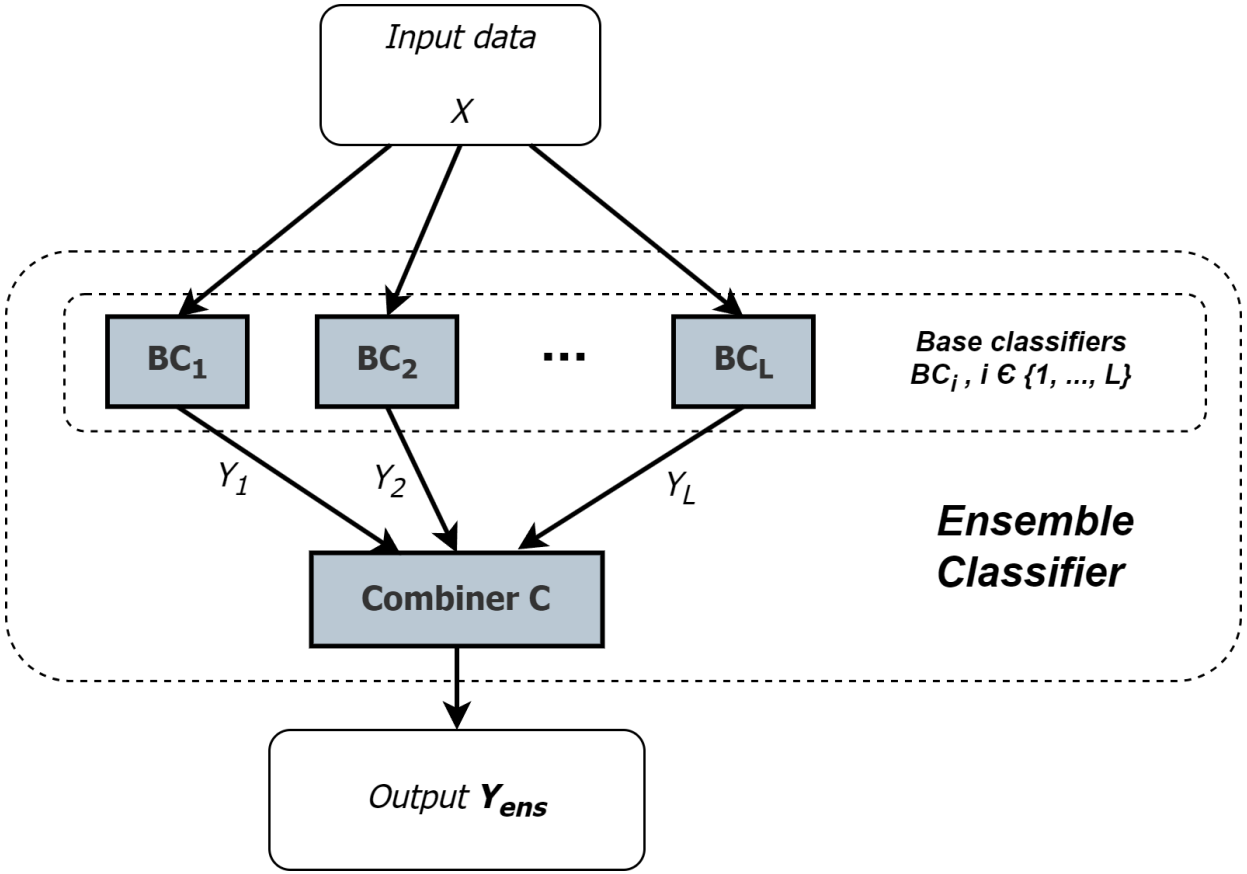
Similarly, in Machine Learning, Ensembles are used to combine individual models into a more powerful one.

ML Models - Ensembles

In Machine Learning an Ensemble is composed of:

- A set of base learners
- A combiner

- The base learners' opinions should be very similar or very diverse?
- The base learners' outputs should be combined by averaging or by selection?



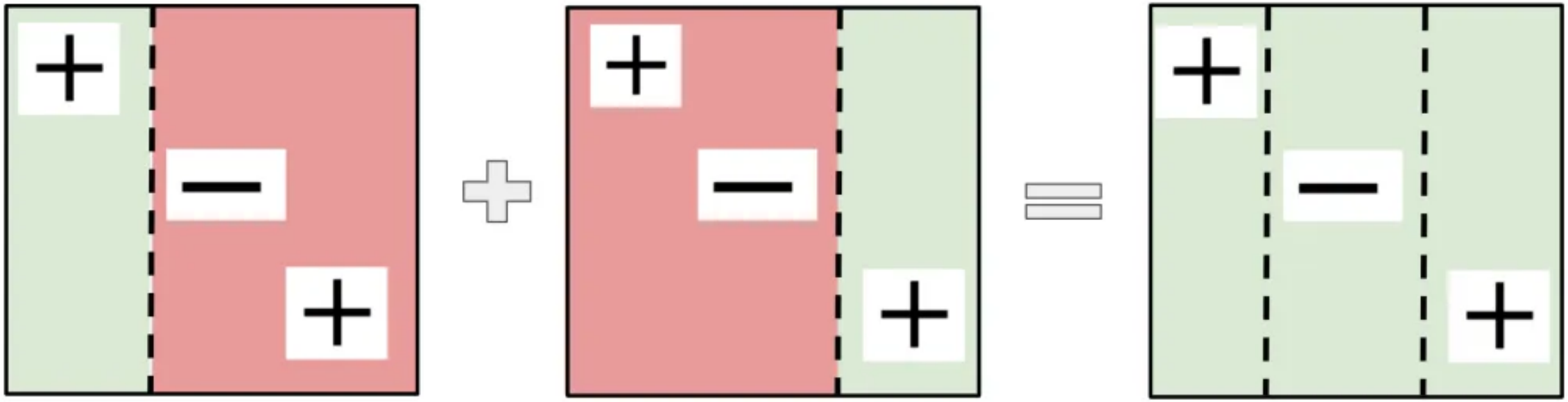
ML Models - Ensembles

Base Learners train vs Combination

- If the base learners are all equal! -> They all make correct or incorrect prediction simultaneously.
  - So, some DIVERSITY (OF OPINION) IS NEDEED.
- If the base learners are very alike, their predictions mostly will agree. When one misses the prediction on one data instance, the predictions of the others will dilute this error!
  - In this case, the best combining strategy is to average the base learners' responses!
- When the base learners are different and their predictions are mutually exclusive. I.e. one tend to predict the outcome correctly when the others fail.
  - In this case a combination strategy by selection is more advisable!

ML Models - Ensembles

Graphical representation of how weak learners can be combined to form a strong learner



Weak Learner
Error Rate: 33%

Weak Learner
Error Rate: 33%

Strong Learner
Error Rate: 0%

© AIML.com Research

- *Dotted lines represents the decision boundary for a weak learner (eg: a decision stump)*
- *+ and – symbols represent the sample points belonging to a binary class classification problem*

ML Models - Ensembles - **Bagging**

